

BIG DATA

Spark Introduction

Prafullata Kiran Auradkar

Department of Computer Science and Engineering

prafullatak@pes.edu

Acknowledgements:

Significant information in the slide deck presented through the Unit 1 of the course have been created by **Dr. K V Subramaniam's** and would like to acknowledge and thank him for the same. There have been some information which I might have leveraged from **Dr. H L Phalachandra's** slide contents too. I may have supplemented the same with contents from books and other sources from Internet and would like to sincerely thank, acknowledge and reiterate that the credit/rights for the same remain with the original authors/publishers only. These are intended for classroom presentation only.

BIG DATA

Overview of lecture – Spark Introduction

- Apache Spark
- Why Spark? – motivation
- Moving to in-memory compute
- Distribute in-memory data
- Putting it all together : Word count
- Spark Use cases

Apache Spark

BIG DATA

What is Apache® Spark™?

- Open source unified engine for large-scale data processing
- Spark started in 2009 as a research project in the UC Berkeley RAD Lab,
- It was built for speed, ease of use and sophisticated analytics
- It is a faster, more general alternative to MapReduce
- Internet powerhouses such as Netflix, Yahoo, and eBay have deployed Spark at massive scale, collectively processing multiple petabytes of data on clusters of over 8,000 nodes.

It has quickly become the largest open source community in big data



With over 1000 contributors from 250+ organizations.

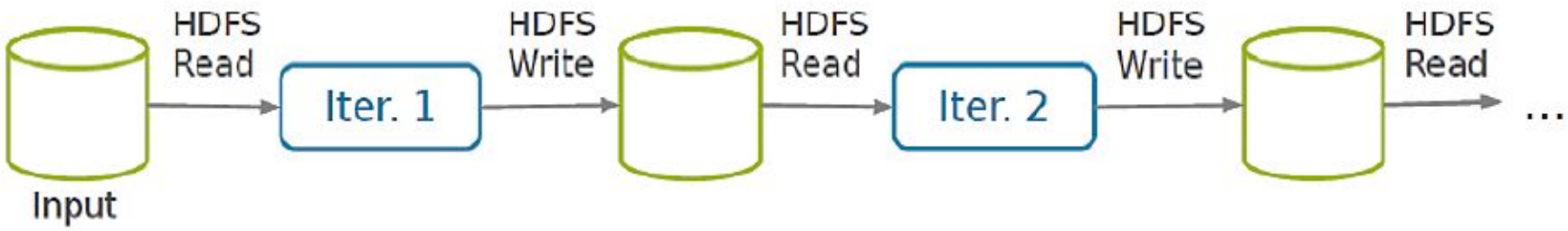
Source: Learning Spark – Lightning-Fast Big Data Analysis (Karau, Konwinski, Wendell & Zaharia, O'Reilly), Ch. 1

Motivation for Spark

MapReduce : Weakness and Limitations

Programming Model

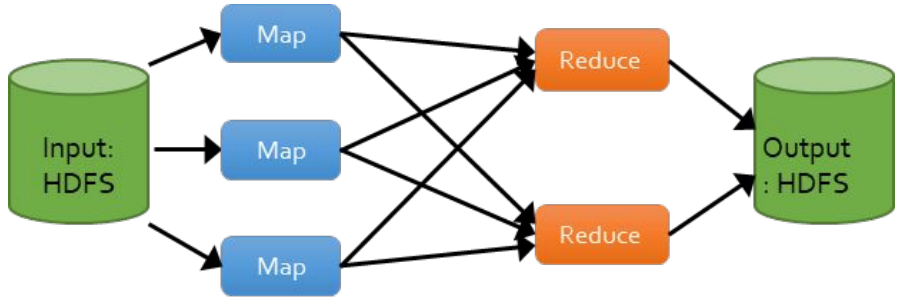
- Hard to express every computation as map + reduce
- Simple tasks need multiple chained MR jobs (e.g. word count sorted by frequency)
- Little control over data types or structure
- No native support for iteration
  - Every iteration reads and writes disk ⟶ expensive overhead
  - Need to design algorithms to reduce the number of iterations.



Every MapReduce iteration reads from and writes back to HDFS → disk I/O dominates runtime.

Mapreduce : Efficiency Pain Points

- High communication cost: map → shuffle → combine → reduce
- Frequent writes of intermediate output to disk
- Main memory barely exploited
- Not workable for real-time stream processing
- Every job re-scans the entire input



An Alternative Model: What Spark Does Differently

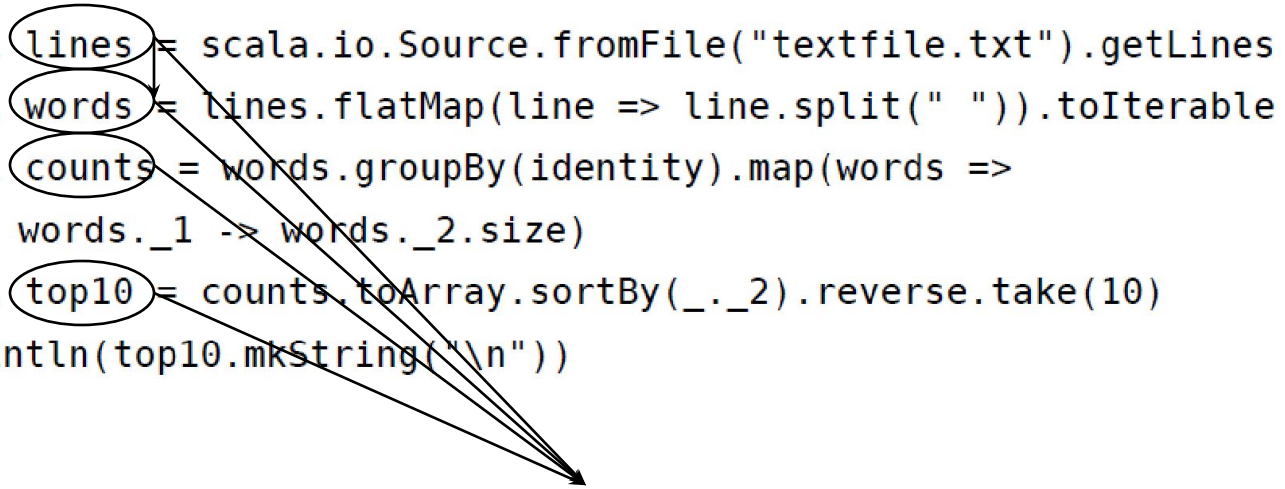
- Acyclic Data Flow
  - Represents a computation as a DAG (Directed Acyclic Graph) of transformations
  - Transformations are combined into a single computation plan
  - Reduces unnecessary intermediate data movement and disk I/O
- Reuse the Working Set
  - Keeps frequently reused data in memory rather than repeatedly writing it to disk
  - The same working set can be reused across multiple computations or iterations
  - Example: PageRank repeatedly updates and reuses the importance vector
- Why This is Better?
  - Less disk I/O
  - Less data movement
  - Faster iterative computation
- Well suited for:
  - Graph processing
  - Machine Learning
  - Interactive analysis with Python/R

In Memory Computation

BIG DATA

Word count in scala

```
val lines = scala.io.Source.fromFile("textfile.txt").getLines
val words = lines.flatMap(line => line.split(" ")).toIterable
val counts = words.groupBy(identity).map(words =>
    words._1 -> words._2.size)
val top10 = counts.toArray.sortBy(_._2).reverse.take(10)
println(top10.mkString("\n"))
```



Each operation creates
A data value that can be kept in
Memory and reused.

```
text_file = sc.textFile("input.txt")  # For local file

# Or for HDFS:

# text_file =
sc.textFile("hdfs://localhost:9000/user/username/input.
txt")

# Split lines into words

words = text_file.flatMap(lambda line: line.split())


# Map each word to (word, 1)

word_pairs = words.map(lambda word: (word, 1))


# Reduce by key (sum counts)

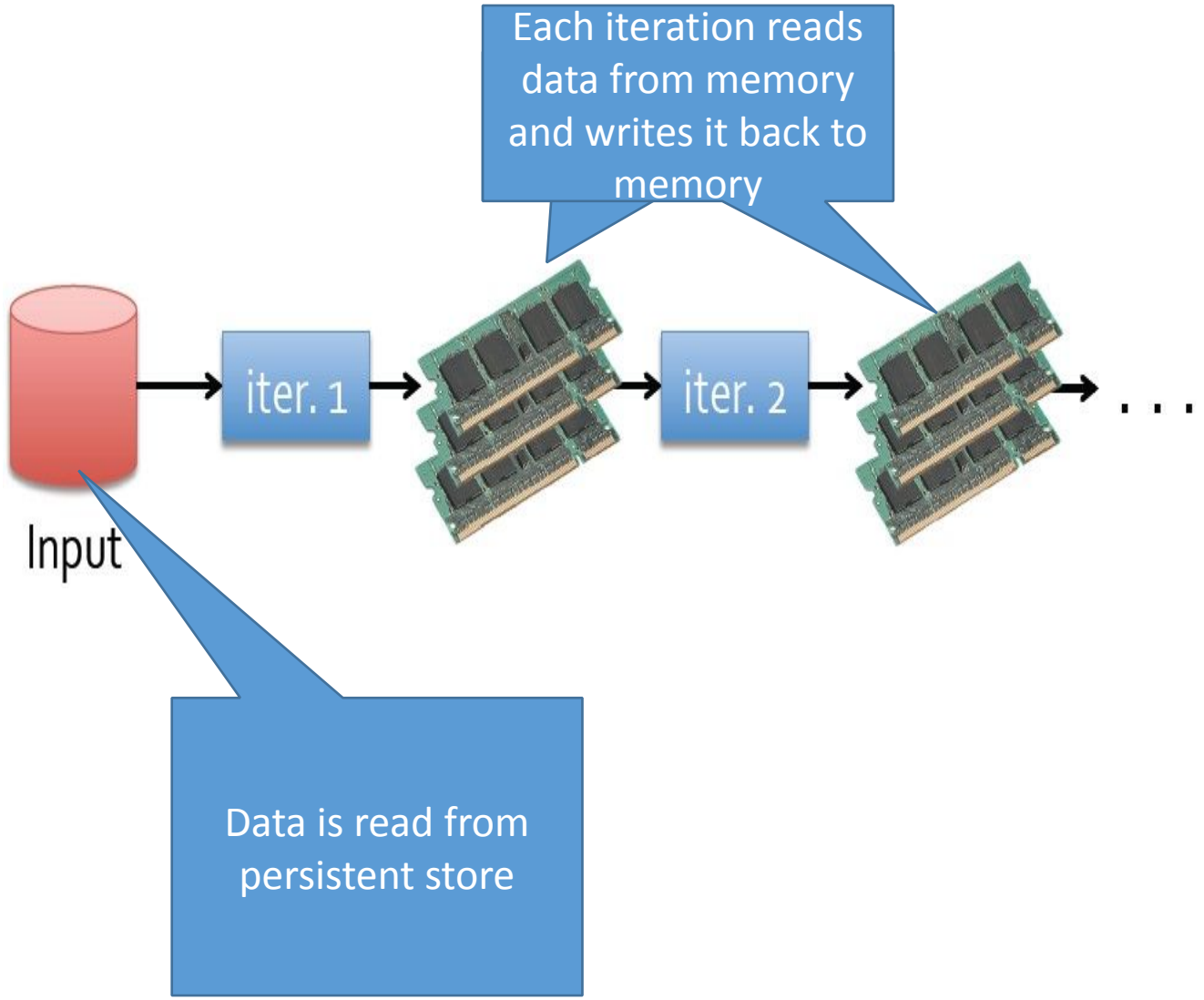
word_counts = word_pairs.reduceByKey(lambda a, b:
a + b)


# Collect results

word_counts.collect()
```

Doing in memory processing – Iterative processing

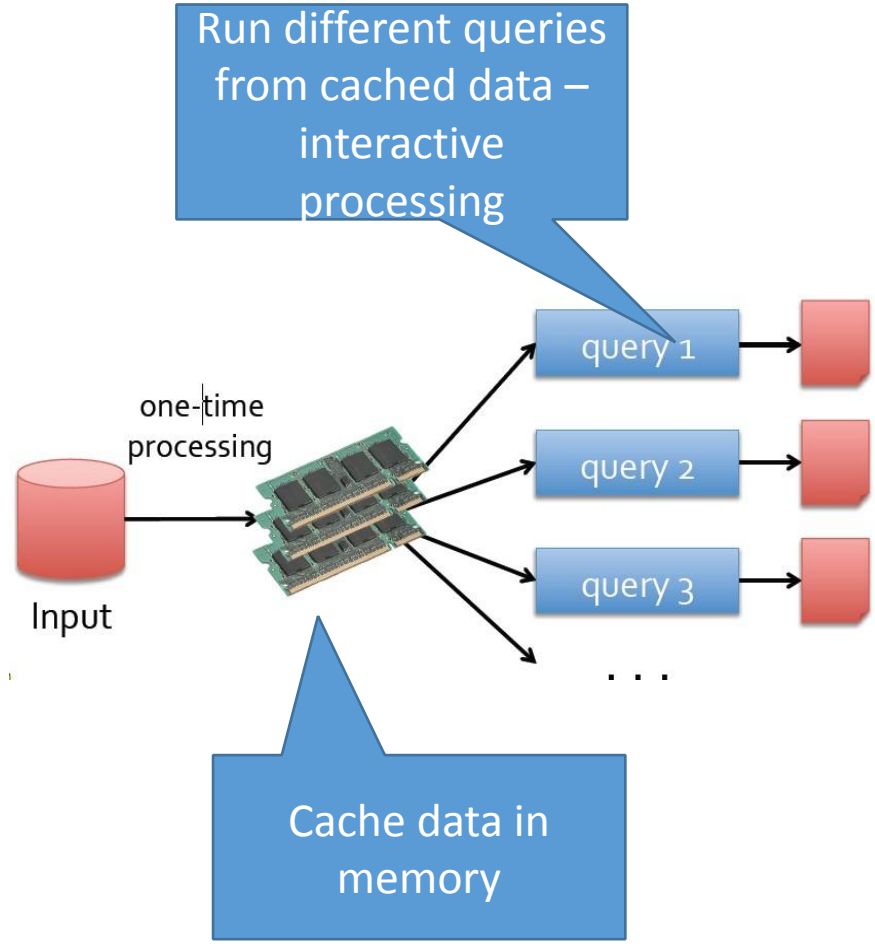
- Data lives in RAM, not on slow disks →processed in parallel
- Each iteration reads from memory and writes back to memory
- Two pillars: RAM storage + parallel distributed processing
- Cheaper RAM has made this economically practical at scale



BIG DATA

Doing in memory processing – interactive processing

- Cache data once, then run many different queries against it
- Great fit for real-time risk management & fraud detection
- Data becomes instantly accessible; computation speed jumps
- Economically viable now that RAM is cheap



Challenges of in memory processing

- How do we distribute the data among the DRAM of the cluster?
- What happens if this memory is not sufficient?
- How do we handle failures because memory is volatile?



Distributed Dataset

What is an RDD?

- Start with a Dataset
  - A dataset is a collection of data that we want to process.
  - In a distributed system, the dataset can be split into partitions and processed across multiple machines.
- Add Lineage
  - Spark keeps track of how the data was created through a sequence of operations.
  - If a partition is lost, Spark can recompute it from its lineage instead of storing another copy.

So,

- RDD = Resilient Distributed Dataset
  - Resilient ⟶ recovers lost data using lineage
  - Distributed ⟶ data is partitioned across machines
  - Dataset ⟶ collection of data being processed

- RDDs are immutable: operations create new RDDs rather than modifying existing ones
- RDDs can also be kept in memory for reuse.

How is lineage information stored

- How Is Lineage Stored?
  - Lineage as a Directed Acyclic Graph (DAG)
  - Spark tracks:
    1. Operations performed on an RDD
    2. RDDs produced by those operations
    3. Dependencies between RDDs

For example:

Input RDD → filter() → map() → reduceByKey() → Output RDD

These dependencies form a DAG.

- If data is lost, Spark follows the lineage to recompute the missing partition.



- What Operations Can We Perform on RDDs?
  - RDD operations are broadly divided into:
    1. Transformations → create a new RDD
    2. Actions → trigger computation and return a result

BIG DATA

Example: Log Processing

Load error messages from a log into memory, then interactively search for various patterns

```
lines = spark.textFile(“hdfs://...”)
errors = lines.filter(startsWithERROR())
messages = errors.map(split(“\t”),2)
cachedMsgs = messages.cache()

cachedMsgs.filter(containsfoo()).count()
cachedMsgs.filter(containsbar()).count()
. . .
```

Courtesy: Zaharia et al , “Spark: Fast, Interactive, Language-Integrated Cluster Computing”, www.spark-project.org

BIG DATA

Example: Log Processing

```
lines = spark.textFile(“hdfs://...”)
errors = lines.filter(startsWithERROR())
messages = errors.map(split(“\t”),2)
cachedMsgs = messages.cache()

cachedMsgs.filter(containsfoo()).count()
cachedMsgs.filter(containsbar()).count()
...
```

Loads a file to an in memory struct called an RDD (think of it as a collection of strings)

Filter function to retain only those lines with an error. Creates another RDD

Applies a function to each element(string) in RDD and produces a new RDD

Keep it in memory as it will be reused

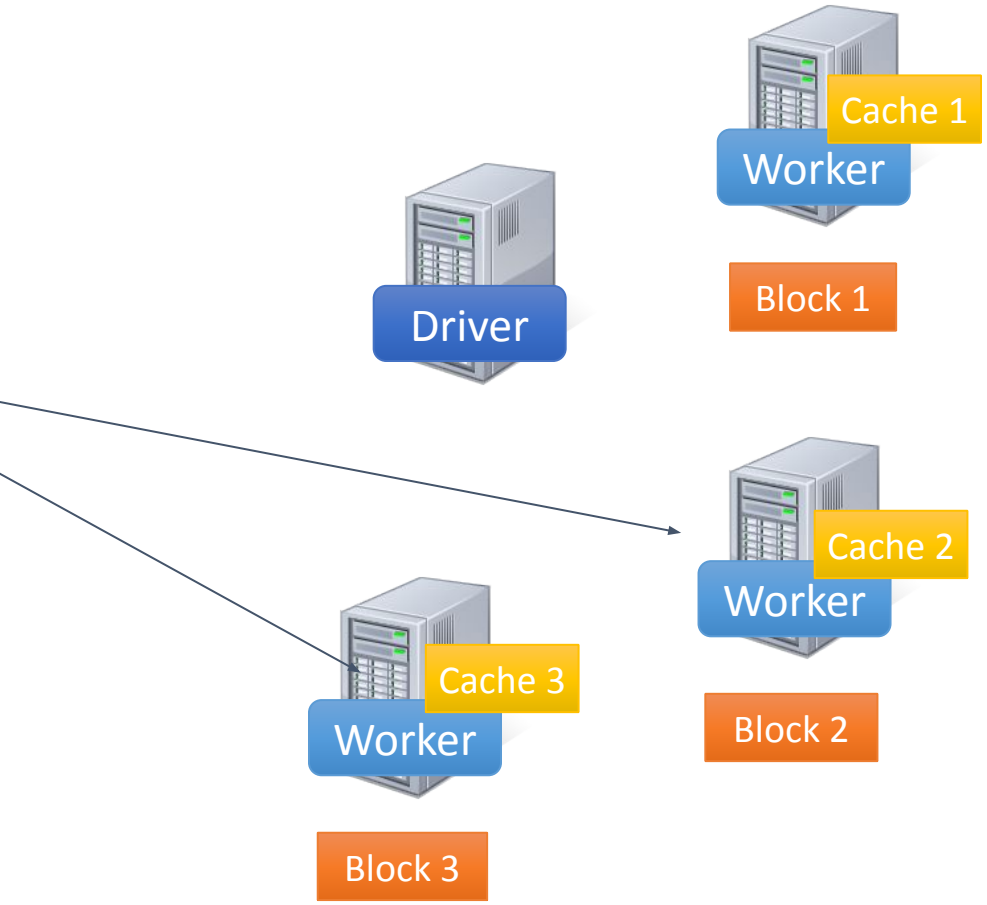
Counts #objects in the RDD

Courtesy: Zaharia et al , “Spark: Fast, Interactive, Language-Integrated Cluster Computing”, www.spark-project.org

Log Processing: Distributing the computation

```
lines = spark.textFile(“hdfs://...”)
errors = lines.filter(startsWithERROR())
messages = errors.map(split(“\t”),2)
cachedMsgs = messages.cache()

cachedMsgs.filter(containsfoo()).count()
cachedMsgs.filter(containsbar()).count()
```



Word Count in Spark

BIG DATA

Word Count in Spark

Create a spark context: tell Spark to create a new job

```
val sc = new SparkContext(new SparkConf().setAppName("Spark Count"))

val tokenized = sc.textFile(args(0)).flatMap(_.split(" "))

val wordCounts = tokenized.map((_, 1)).reduceByKey(_ + _)
```

Read in text file
Split it into words

Each of these is an RDD

Map each word to 1

Reduce by key. Can also use countbykey

https://docs.cloudera.com/documentation/enterprise/5-13-x/topics/spark_develop_run.html

BIG DATA

Spark Use cases



Additional References

- What is Apache Spark? Matei Zaharia
  - https://www.youtube.com/watch?v=p8FGC49N-zM



THANK YOU

Prafullata Kiran Auradkar
Department of CSE
prafullatak@pes.edu



BIG DATA

PySpark Hands on

Prafullata Kiran Auradkar

Department of Computer Science and Engineering

prafullatak@pes.edu

Acknowledgements:

Significant information in the slide deck presented through the Unit 1 of the course have been created by **Dr. K V Subramaniam's** and would like to acknowledge and thank him for the same. There have been some information which I might have leveraged from **Dr. H L Phalachandra's** slide contents too. I may have supplemented the same with contents from books and other sources from Internet and would like to sincerely thank, acknowledge and reiterate that the credit/rights for the same remain with the original authors/publishers only. These are intended for classroom presentation only.

BIG DATA

Overview of lecture

- What is PySpark
- Installation
- PySpark Architecture
- Word count with PySpark and Scala

What is PySpark

BIG DATA

What is PySpark

- PySpark is a Python API
- released by the Apache Spark community to support Python with Spark.
- easily integrate and work with RDDs in Python programming language.
- It has numerous features to work with huge datasets.
- Python has a rich library set.
- majority of data scientists and analytics experts use Python nowadays.

Features of PySpark

1) **Real-time computations:** Because of the in-memory processing in the PySpark framework, it shows low latency.
2) **Polyglot:** The PySpark framework is compatible with various languages such as Scala, Java, Python, and R, which makes it one of the most preferable frameworks for processing huge datasets.
3) **Caching and disk persistence:** This framework provides powerful caching and great disk persistence.
4) **Fast processing:** The PySpark framework is way faster than other traditional frameworks for Big Data processing.
5) **Works well with RDDs:** Python programming language is dynamically typed, which helps when working with RDDs.

Interactive and Batch processing

- Supports the pyspark shell for interactive processing
- And regular batch processing jobs can be run by writing pyspark scripts

PySpark configuration

Downloading and Setting up pyspark

- Download and untar the spark tar file from the spark repository

```
# sudo tar -xzf spark-3.3.4-bin-hadoop3.tgz -C /opt/spark --strip-components=1
```

- It will create unzip it into the directory: `/opt/spark`
- Before starting PySpark, you need to set the following environments to set the Spark path and the Py4j path.

- Pyspark is bundled along with spark

- However, it requires some configuration
  - Setup of proper paths.

BIG DATA

Pyspark configuration

- Needs environment variables to be setup
- First modify .bashrc to include

```
echo 'export SPARK_HOME=/opt/spark' >> ~/.bashrc
echo 'export PATH="$SPARK_HOME/bin:$SPARK_HOME/sbin:$PATH"' >> ~/.bashrc
echo 'export PYSPARK_PYTHON=/usr/bin/python3.10' >> ~/.bashrc
echo 'export PYSPARK_DRIVER_PYTHON=/usr/bin/python3.10' >> ~/.bashrc
source ~/.bashrc
```

Then run so that the env variables are setup

Or, to set the above environments globally, put them in the .bashrc file. Then run the following command for the environments to work.

```
# source .bashrc
```

BIG DATA

Starting pyspark

- Needs environment variable to be setup
- First modify .bashrc to include

- Now that we have all the environments set, let us go to Spark directory and invoke PySpark shell by running the following command –

```
prafullata@prafullata-VirtualBox:~$ start-all.sh
starting org.apache.spark.deploy.master.Master, logging to
oy.master.Master-1-prafullata-VirtualBox.out
localhost: starting org.apache.spark.deploy.worker.Worker,
.spark.deploy.worker.Worker-1-prafullata-VirtualBox.out
prafullata@prafullata-VirtualBox:~$ jps
6578 Jps
6483 Worker
6276 Master
prafullata@prafullata-VirtualBox:~$
```

BIG DATA

Starting pyspark

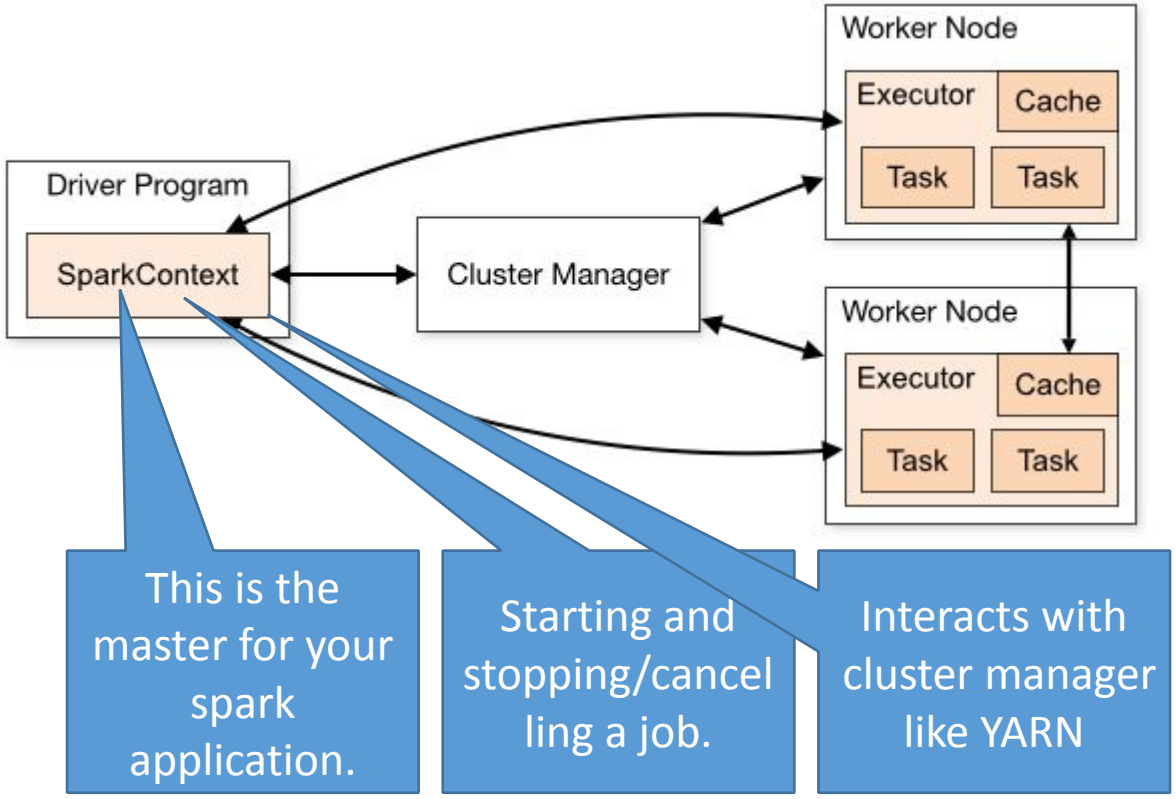
- Pyspark interactive terminal

```
prafullata@prafullata-VirtualBox:~$ pyspark
Python 3.10.12 (main, Aug 15 2025, 14:32:43) [GCC 11.4.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
25/10/08 21:51:00 WARN Utils: Your hostname, prafullata-VirtualBox resolves to a loopbac
0:7910:bd13:88b7:e0d6 instead (on interface enp0s3)
25/10/08 21:51:00 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another address
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLev
25/10/08 21:51:14 WARN NativeCodeLoader: Unable to load native-hadoop library for your p
 where applicable
Welcome to

      ____              __
     / __/__  ___ _____/ /__
    _\ \/ _ \/ _ `/ __/  '_/
   /__ / .__/\_,_/_/ /_/\_\   version 3.4.1
      /_/

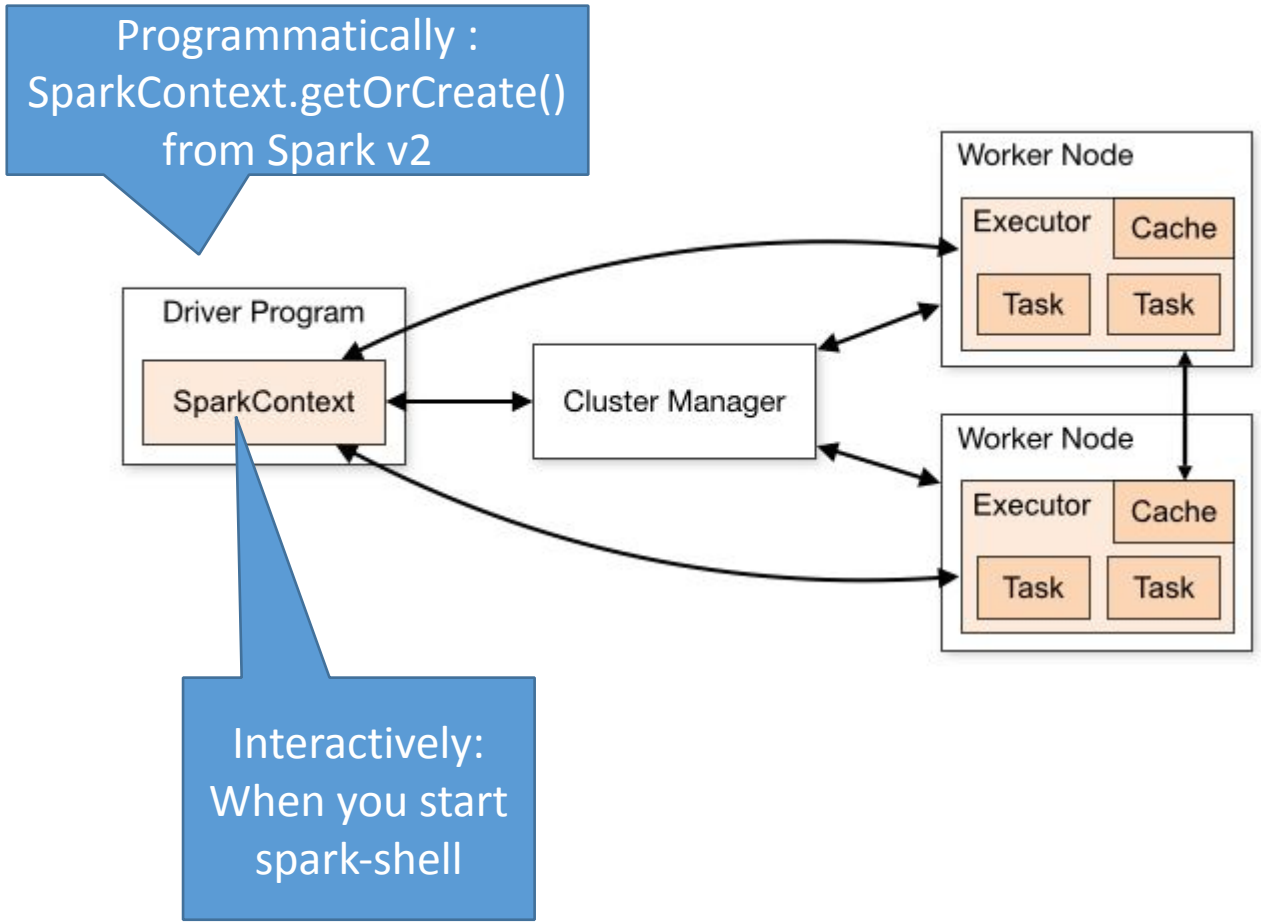
Using Python version 3.10.12 (main, Aug 15 2025 14:32:43)
Spark context Web UI available at http://[fd00:0:0:0:7910:bd13:88b7:e0d6]:4040
Spark context available as 'sc' (master = local[*], app id = local-1759940480991).
SparkSession available as 'spark'.
>>>
```

Recall: The Spark Context



https://spark.apache.org/docs/latest/cluster-overview.html

When is Spark Context created?



There is one SparkContext per JVM

https://spark.apache.org/docs/latest/cluster-overview.html

Pyspark architecture

Pyspark architecture

- So if the spark context is maintained per JVM
- How does pyspark take care of the SparkContext?
  - Who creates and maintains this?

PES UNIVERSITY
CELEBRATING 50 YEARS

py4j

- Acts like a bridge between python and java
- Allows python interpreter to access Java objects instantiated within the JVM
- Can invoke methods on the Java objects as if they were python methods

https://www.py4j.org/

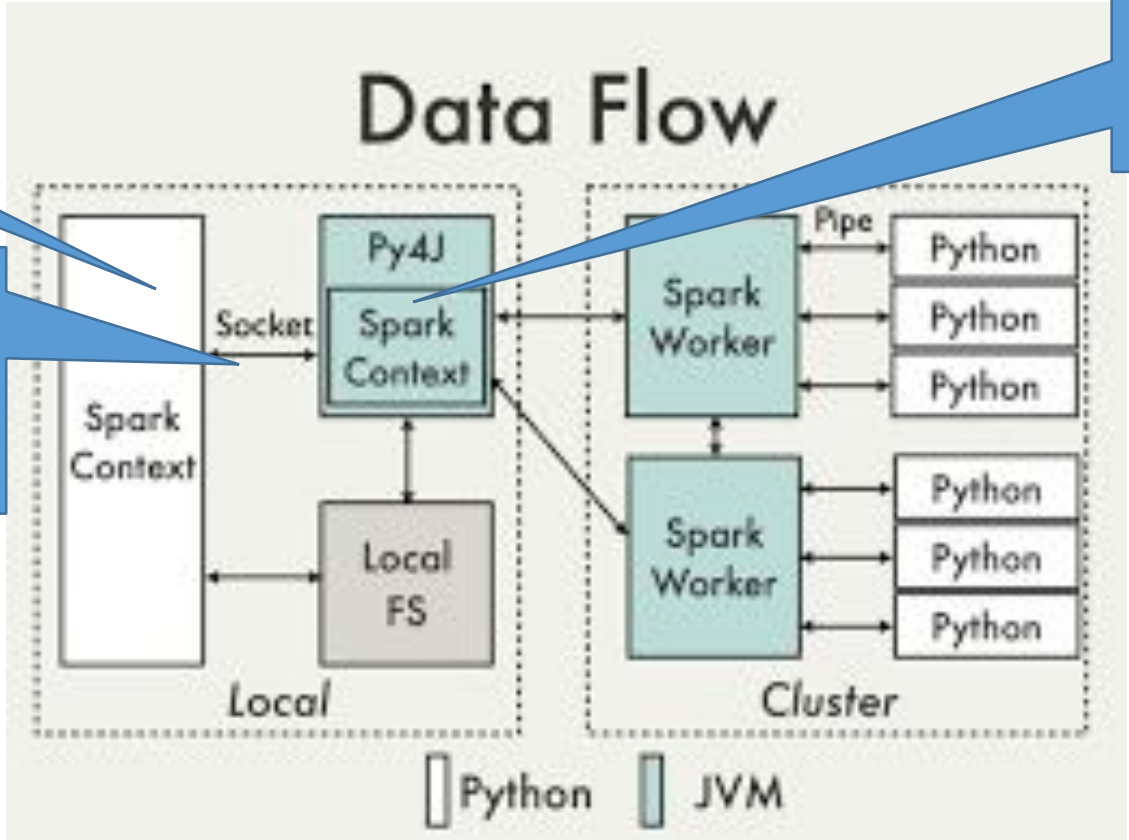
BIG DATA

Pyspark architecture

Python driver

Data has to be serialized using cPickle serializer

Java SparkContext used to drive the application



https://cwiki.apache.org/confluence/display/SPARK/PySpark+Internals

Lifecycle of a Spark Program

- Create some input RDDs from external data or parallelize a collection in your driver program. (Resilient Distributed Dataset (RDD) – is an immutable object. More details in the next upcoming classes.)
- Lazily transform them to define new RDDs using transformations like filter() or map()
- Ask Spark to cache() any intermediate RDDs that will need to be reused.
- Launch **actions** such as count() and collect() to kick off a parallel computation, which is then optimized and executed by Spark.
- Till **actions** are not seen, DAG is built by the master. No actual work will be done.

BIG DATA

PySpark Programming Model

PES UNIVERSITY
CELEBRATING 50 YEARS

PySparkShell - Details for ×

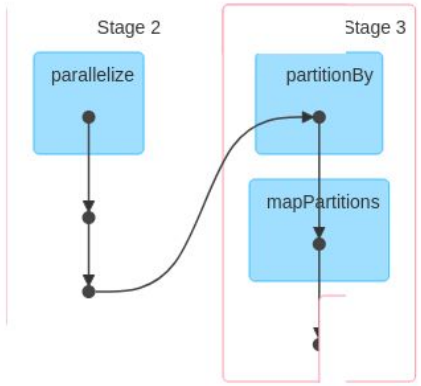
http://localhost:4040/jobs/job/?id=1

Spark 3.4.1 | Jobs | Stages | Storage | Environment | Executors

Details for Job 1

Status: SUCCEEDED
Submitted: 2025/10/08 22:08:09
Duration: 3 s
Completed Stages: 2

▸ Event Timeline
▾ DAG Visualization




```
prafullata@prafullata-VirtualBox: ~
PythonRDD[5] at RDD at PythonRDD.scala:53
>>> r = sc.parallelize([("a", 1), ("b", 1), ("a", 1)])
>>> y = r.reduceByKey(lambda a, b: a + b)
>>> y.collect()
[('b', 1), ('a', 2)]

>>> z=r.reduceByKey(lambda a,b:a+b)
>>> z.count()
[Stage 2:>                                                          (0
[Stage 2:==============>                                          (1
[Stage 2:==============================>                          (2
[Stage 2:============================================>            (3
[Stage 3:>
[Stage 3:==============================>                          (2

2
>>>
```

▾ Completed Stages (2)

Page: 1 — 1 Pages. Jump to 1

Stage Id ▾	Description	Submitted	Duration	Tasks: Succeeded/Total	Input	Output	Shuffle Read
3	count at <stdin>:1 +details	2025/10/08 22:08:11	1 s	4/4			214.0 B
2	reduceByKey at <stdin>:1 +details	2025/10/08 22:08:09	2 s	4/4			

Page: 1 — 1 Pages. Jump to 1

PySpark Programming Model

SparkContext

- Main entry point for Spark functionality.
- A SparkContext represents the connection to a Spark cluster, and it can be used to create RDD and broadcast variables on that cluster.
- When you create a new SparkContext, at least the master and app name should be set, either through the named parameters or through conf.

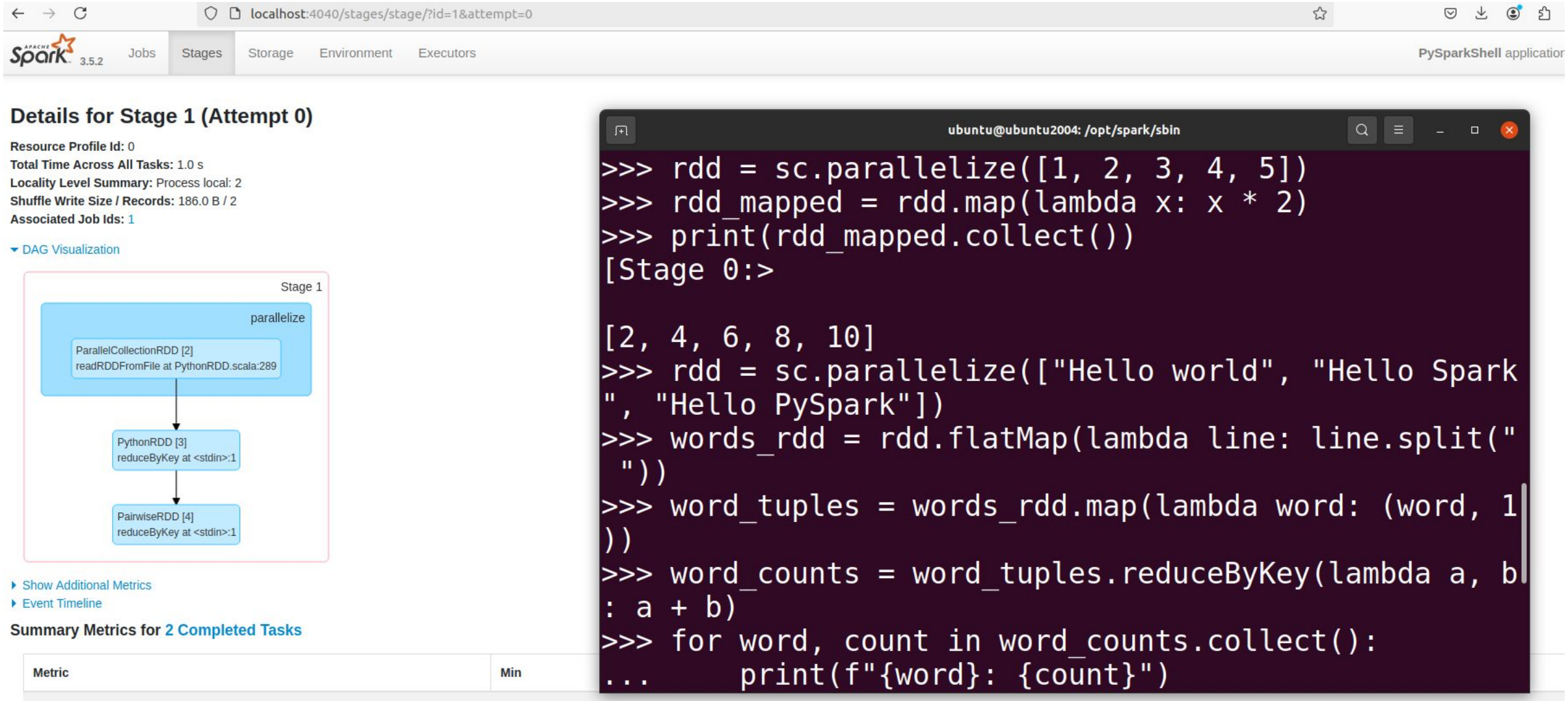
Create RDD

Two ways we can create the RDDs:

- loading an external dataset
- distributing a set of collection of objects.

BIG DATA

PySpark Programming Model: word count from terminal




```
ubuntu@ubuntu2004: /opt/spark/sbin
>>> rdd = sc.parallelize([1, 2, 3, 4, 5])
>>> rdd_mapped = rdd.map(lambda x: x * 2)
>>> print(rdd_mapped.collect())
[Stage 0:>

[2, 4, 6, 8, 10]
>>> rdd = sc.parallelize(["Hello world", "Hello Spark
", "Hello PySpark"])
>>> words_rdd = rdd.flatMap(lambda line: line.split("
 "))
>>> word_tuples = words_rdd.map(lambda word: (word, 1
))
>>> word_counts = word_tuples.reduceByKey(lambda a, b
: a + b)
>>> for word, count in word_counts.collect():
...     print(f"{word}: {count}")
```

PySpark Programming Model: word count from terminal

```
>>> for word, count in word_counts.collect():
...     print(f"{word}: {count}")
...
[Stage 1:==============================>

Hello: 3
world: 1
Spark: 1
PySpark: 1
>>>
```

```
>>> rdd = sc.parallelize(["Hello world", "Hello Spark", "Hello PySpark"])
```



This was the input

PySpark Programming Model

PES UNIVERSITY
CELEBRATING 50 YEARS

Using parallelize()

```
from pyspark.sql import SparkSession

spark = SparkSession \

.builder \

.appName("Python Spark create RDD example") \

.config("spark.some.config.option", "some-value") \

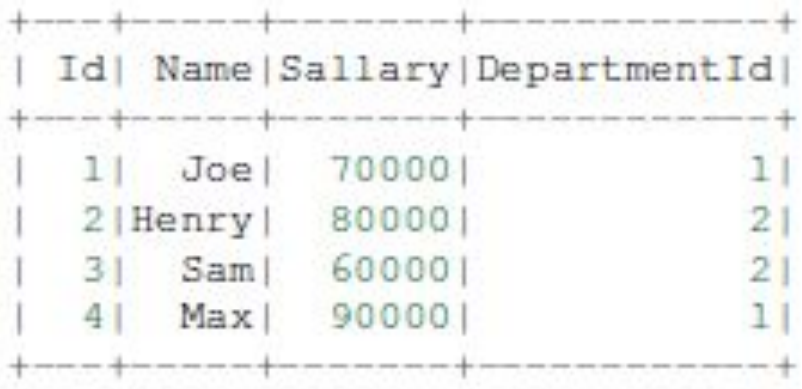
.getOrCreate()

myData = spark.sparkContext.parallelize([(1,2), (3,4), (5,6), (7,8), (9,10)])

myData.collect()
```

Using createDataFrame()

```
from pyspark.sql import SparkSession
spark = SparkSession \
.builder \
.appName("Python Spark create RDD example") \
.config("spark.some.config.option", "some-value") \
.getOrCreate()
Employee = spark.createDataFrame([
('1', 'Joe', '70000', '1'),
('2', 'Henry', '80000', '2'),
('3', 'Sam', '60000', '2'),
('4', 'Max', '90000', '1')],
['Id', 'Name', 'Sallary','DepartmentId']
)
```



Id	Name	Sallary	DepartmentId
1	Joe	70000	1
2	Henry	80000	2
3	Sam	60000	2
4	Max	90000	1

Simplifying tasks for programmers - DataFrames

Need for dataframes

- Data RDDs are completely opaque to Spark
  - Meaning Spark cannot parse these values
- Is there some way to make Spark understand the format, so that we can do processing more easily
  - Like sql type queries
- Consider data like on the right that we need to run a query on?

USN	Name	Marks
45	Vkoli	11
10	Stendul	43
195	Abachpan	28

BIG DATA

What is a dataframe

- Introduced in 2015
- Inspired by Dataframes in R and Pandas in python
- Distributed collection of data into named columns
- An abstraction built over RDD that allows
  - Schema to be defined on a RDD
- Also has an optimizer built in for queries

USN	Name	Marks
45	Vkoli	11
10	Stendul	43
195	Abachpan	28

BIG DATA

Creating dataframes

```
from pyspark.sql import SQLContext
sqlContext = SQLContext(sc)

df = spark.read.json("file:///home/prafullata/students.json")

# let us display the contents
df.show()
## USN  name  marks
## 045  Vkoli   11
## 010  Stendul  43
## 195  Abachpan  28

Df = rdd.toDF(“USN”, “name”)
```

Dataframes can be created from existing RDDs, HIVE tables other Data sources.

This example is creating from a JSON file

```
>>> df = sqlContext.read.option("multiLine", "true").json("/home/chetana/students.json")
>>> df.show()
+-----+--------+---+
|Marks|    Name|USN|
+-----+--------+---+
|   11|   Vkoli| 45|
|   43| Stendul| 10|
|   28|Abachpan|195|
+-----+--------+---+
```

Alternatively, from an existing RDD by naming the columns

BIG DATA

Using a dataframe

```
# Print the schema in a tree format
df.printSchema()
## root
## |-- usn: long (nullable = true)
## |-- name: string (nullable = true)
## |-- marks: long (nullable = true)

# Select only the "name" column
df.select("name").show()
## name
## VKoli
## STendul
## ABachpan

# Select everybody, but increment the marks by 1
df.select("name", df.marks + 1).show()
## name    (marks + 1)
## Vkoli       12
## STendul     44
## ABachpan    29
```

```
>>> df.printSchema()
root
 |-- Marks: long (nullable = true)
 |-- Name: string (nullable = true)
 |-- USN: long (nullable = true)

>>> df.select("name").show()
+--------+
|    name|
+--------+
|   Vkoli|
| Stendul|
|Abachpan|
+--------+
```

```
>>> df.select("name",df.Marks+1).show()
+--------+-----------+
|    name|(Marks + 1)|
+--------+-----------+
|   Vkoli|         12|
| Stendul|         44|
|Abachpan|         29|
+--------+-----------+

>>>
```

Exercise

- Consider a case where you have data in a CSV file that consists of <pan number, date, tax_paid> and you wanted to find out the total tax paid by each individual pan holder
  - How will you do it in Spark?
  - How will you do it with Spark Data frames

PES UNIVERSITY
CELEBRATING 50 YEARS

Task Assignment

- Consider a case where you have data in a CSV file that consists of <pan number, date, tax_paid> and you wanted to find out the total tax paid by each individual pan holder
  - How will you do it in Spark?
  - How will you do it with Spark Data frames

Using Dataframes

```
Df = rdd.toDF("pan number",
"date", "taxpaid")
Df.select("pan number", "tax
paid").groupBy("pan
number").sum()
```

Note that this is done using the name of the column rather than by splitting the data which we would do if used Spark.

PySpark Demo



THANK YOU

Prafullata Kiran Auradkar

Department of Computer Science and Engineering

prafullatak@pes.edu



BIG DATA

Spark : Architecture

Prafullata Kiran Auradkar

Department of Computer Science and Engineering

prafullatak@pes.edu

Acknowledgements:

Significant information in the slide deck presented through the Unit 1 of the course have been created by **Dr. K V Subramaniam's** and would like to acknowledge and thank him for the same. There have been some information which I might have leveraged from **Dr. H L Phalachandra's** slide contents too. I may have supplemented the same with contents from books and other sources from Internet and would like to sincerely thank, acknowledge and reiterate that the credit/rights for the same remain with the original authors/publishers only. These are intended for classroom presentation only.

Overview of lecture – Spark Introduction

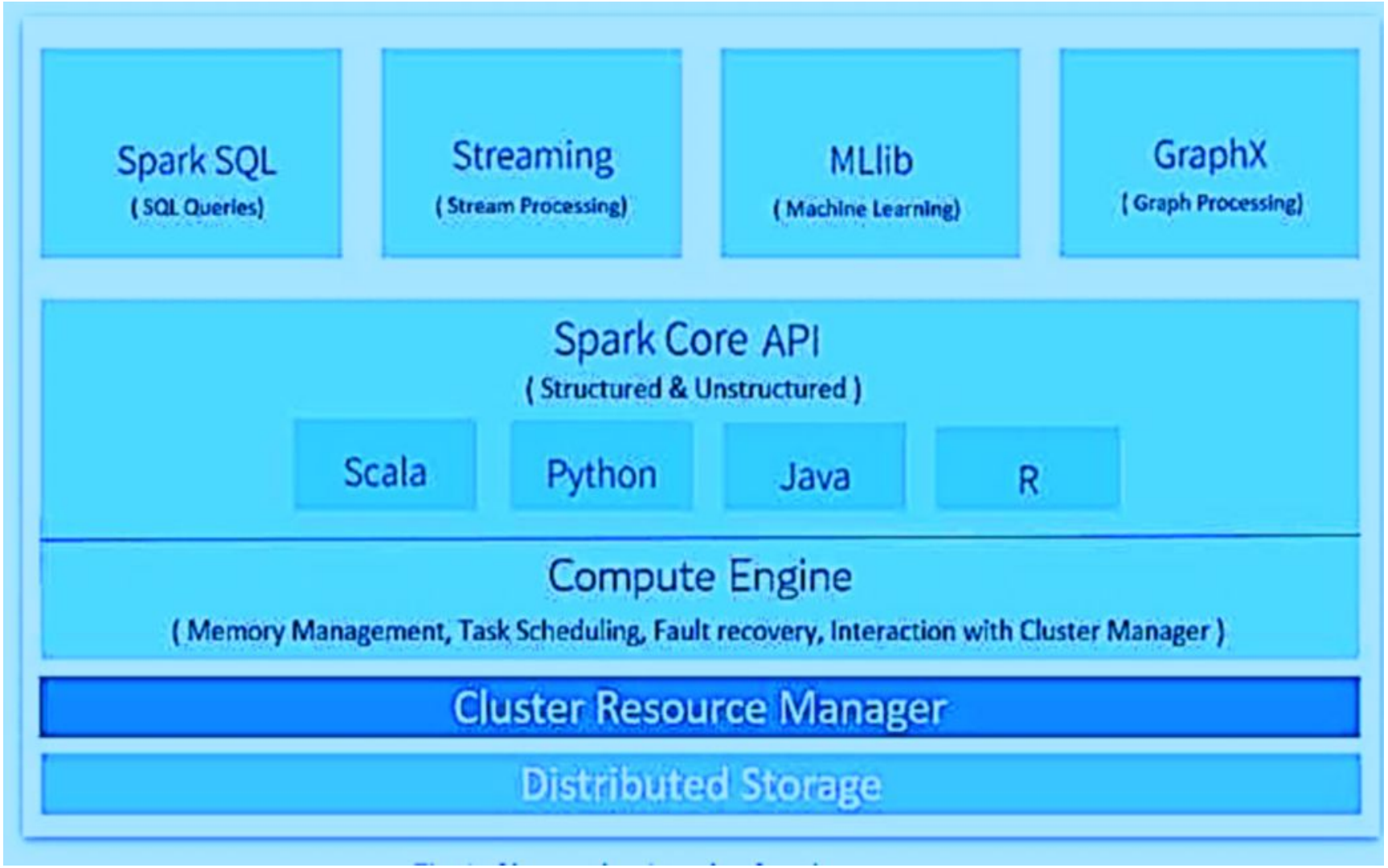
PES UNIVERSITY
CELEBRATING 50 YEARS

- Spark Ecosystem
- Spark Data processing
- Spark Architecture and working details
- Lazy Execution
- Spark Scheduling

Spark Ecosystem

BIG DATA

Spark Ecosystem



Storage and Cluster Manager

- Apache Spark is a distributed processing engine.
- it doesn't come with an inbuilt cluster resource manager and a distributed storage system.
- But, it allows us to use any compatible cluster manager and storage solution.
- Apache YARN, Mesos, and even Kubernetes can be used as a cluster manager
- HDFS, Amazon S3, Google Cloud storage, Cassandra File system etc., can be used for the storage system

PES UNIVERSITY
CELEBRATING 50 YEARS

Spark Core

- Apache Spark core contains two main components.
- Spark Compute engine
- Spark Core APIs
- The compute engine provides basic functionalities like memory management, task scheduling, fault recovery and most importantly interacting with the cluster manager and storage system.
- Spark compute engine executes and manages the Spark jobs

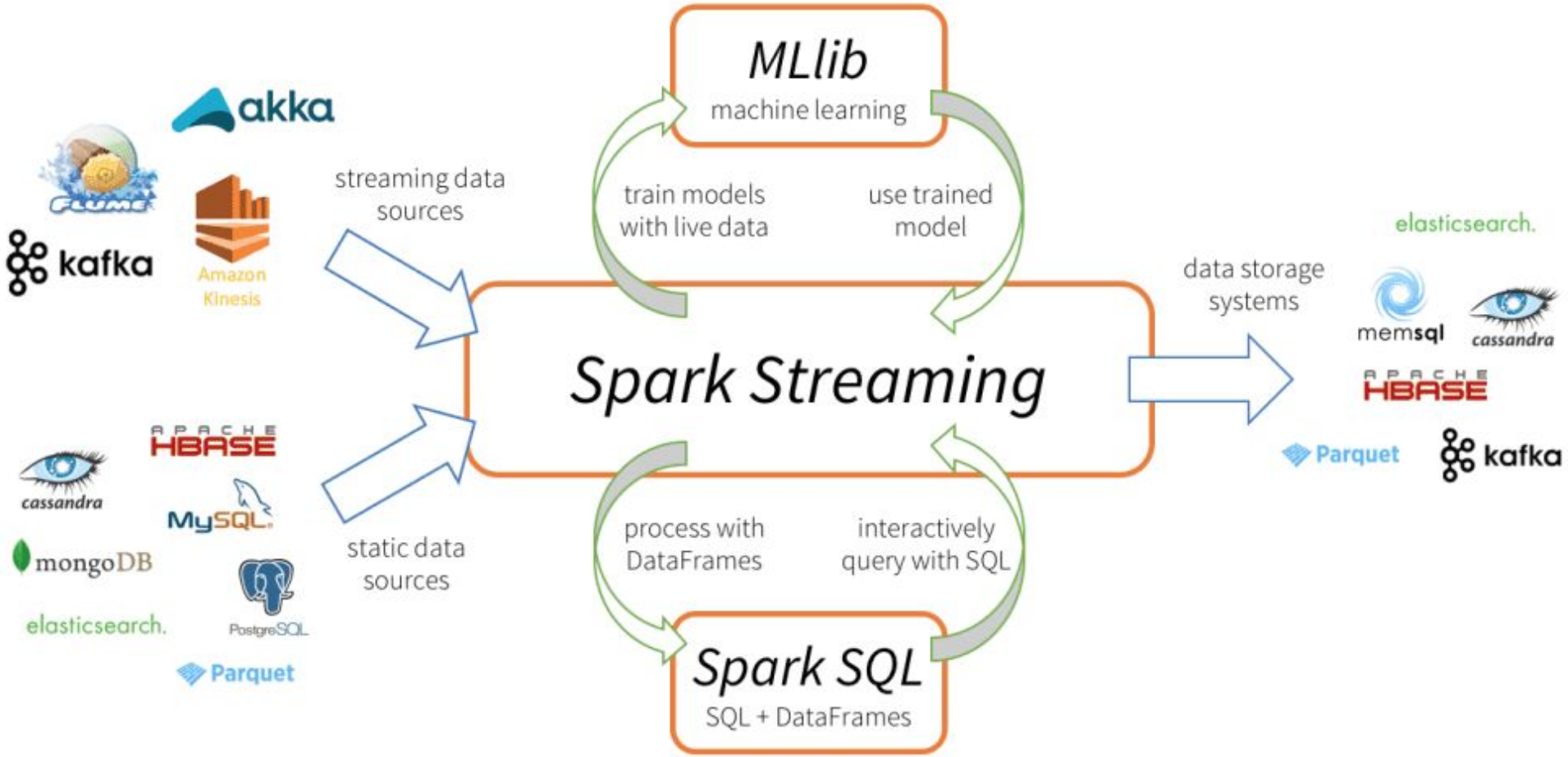
- The second part of Spark Core is core API. Spark core consists of two types of APIs.

  1.Structured API

  2.Unstructured API

- The Structured APIs consists of data frames and data sets.
- They are designed and optimized to work with structured data.
- The Unstructured APIs are the lower level APIs including RDDs, Accumulators and Broadcast variables. These core APIs are available in Scala, Python, Java, and R.

Spark Ecosystem-Libraries and DSL



PES UNIVERSITY
CELEBRATING 50 YEARS

Libraries and DSL

1. Spark SQL
   - For working with structured data
   - View datasets as relational tables
   - Define a schema of columns for a dataset
   - Perform SQL queries
   - Supports many sources of data
   - Hive tables, Parquet and JSON

2. Spark Streaming



- Data analysis of streaming data
- e.g. log files generated by production web servers
- Aimed at high-throughput and fault-tolerant stream processing
- Dstream: stream of datasets that contain data from certain intervals
- Usually applied to time series data; intervals are time windows (1 second; 1 minute etc)
- Joins on unlimited streams are impossible so Spark Streaming joins within a time window, or a static dataset with a stream

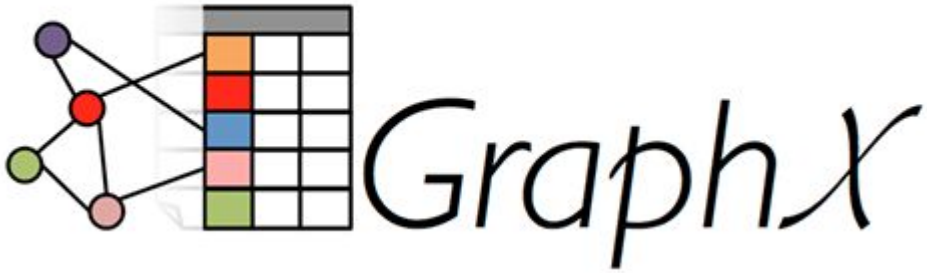
PES UNIVERSITY
CELEBRATING 50 YEARS

3.MLlib

- MLlib is a machine learning library that contains delivers high-quality algorithms.
- Basic statistics
- Classification (Naïve Bayes, decision tress, LR)
- Clustering (k-means, Gaussian mixture, …)
- All the methods are designed to scale out across a cluster.

GraphX - It comes with a library of typical graph algorithms.

- Graph Processing Library
- Defines a graph abstraction
- Directed multi-graph
- Properties attached to each edge and vertex
- RDDs for edges and vertices
- Provides various operators for manipulating graphs (e.g. subgraph and mapVertices)



Spark Data processing

BIG DATA

Spark data processing

Spark allows four different styles of data analysis and processing.

1) Batch:

This mode is used for manipulating large datasets, typically performing large map-reduce jobs

2) Streaming:

This mode is used to process incoming information in near real time

3) Iterative:

This mode is for machine learning algorithms such as a gradient descent where the data is accessed repetitively in order to reach convergence

4) Interactive:

This mode is used for data exploration as large chunks of data are in memory and due to the very quick response time of Spark

Spark Architecture and working details

Spark Architecture

Spark Applications consist of a *driver* process and a set of *executor* processes.

Driver process is responsible for

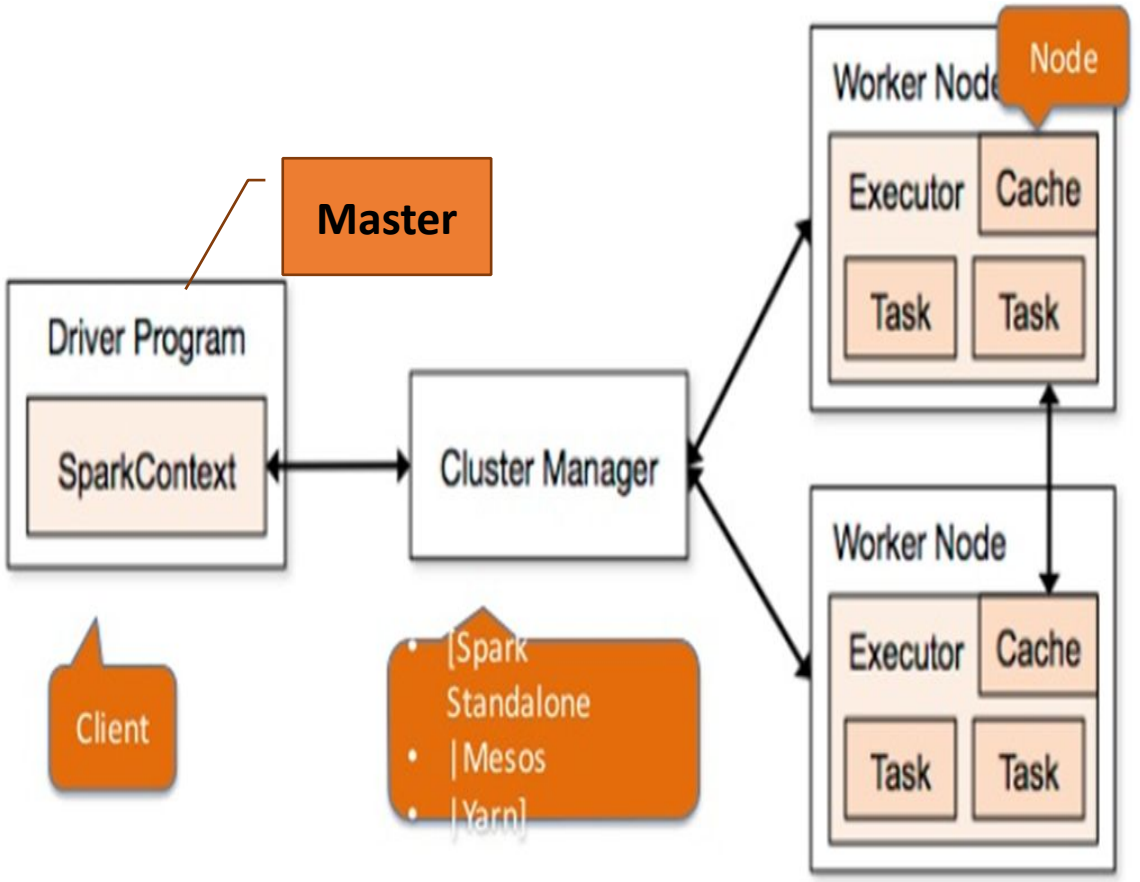
- maintaining information about the Spark Application;
- responding to a user's program or input;
- analyzing, distributing, and scheduling work across.

The **executors** are responsible for actually carrying out the work. It is responsible for

- executing code assigned to it by the driver
- reporting the state of the computation on that executor back to the driver node.

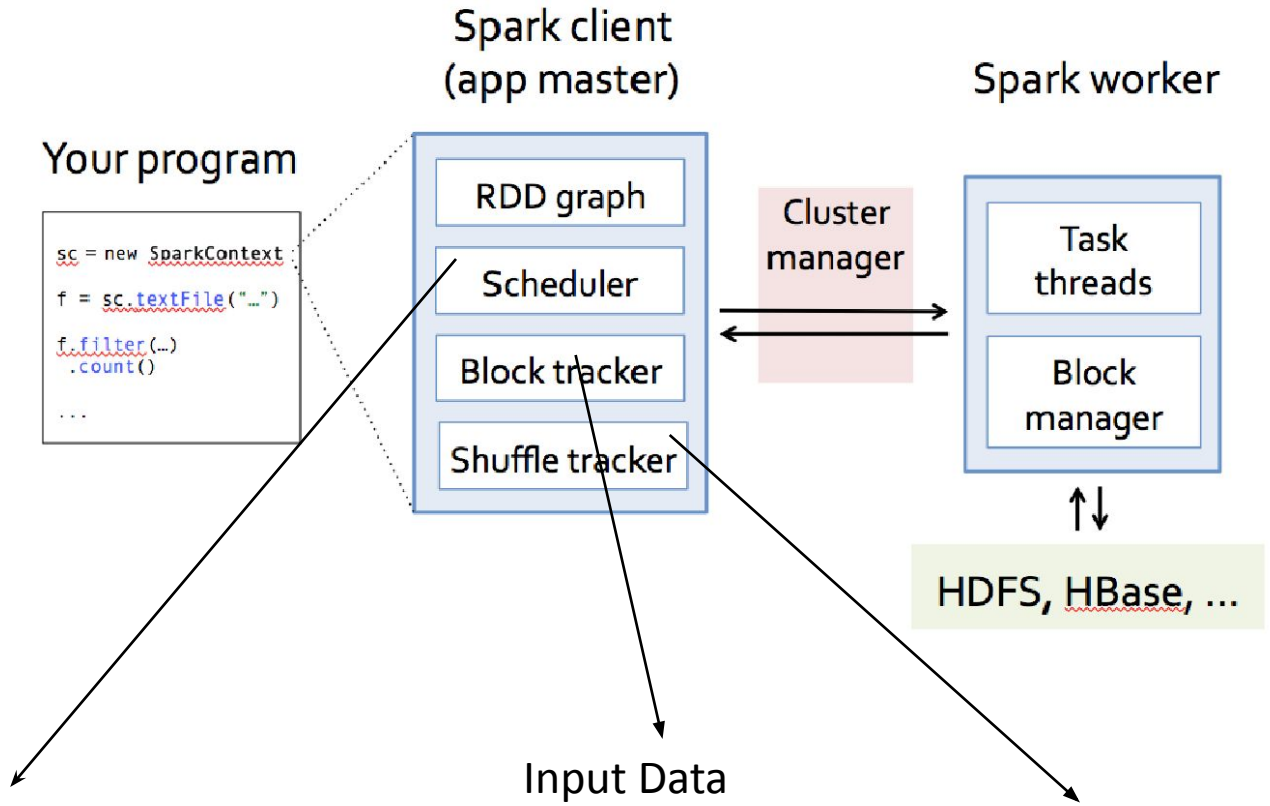
cluster manager controls physical machines and allocates resources to Spark Applications.

This can be one of three core cluster managers: Spark's standalone cluster manager, YARN, or Mesos.



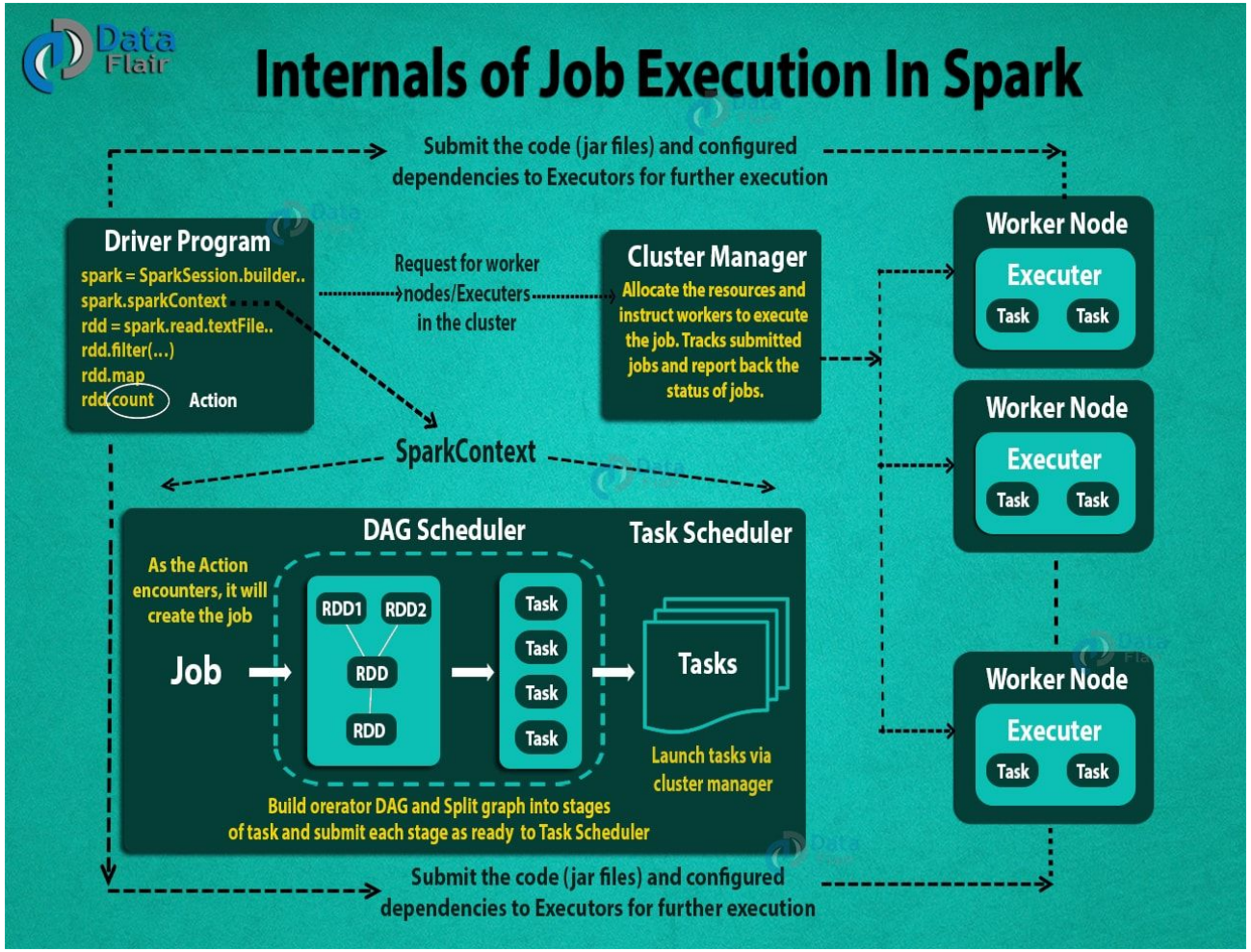
https://spark.apache.org/docs/latest/cluster-overview.html

Spark Architecture



BIG DATA

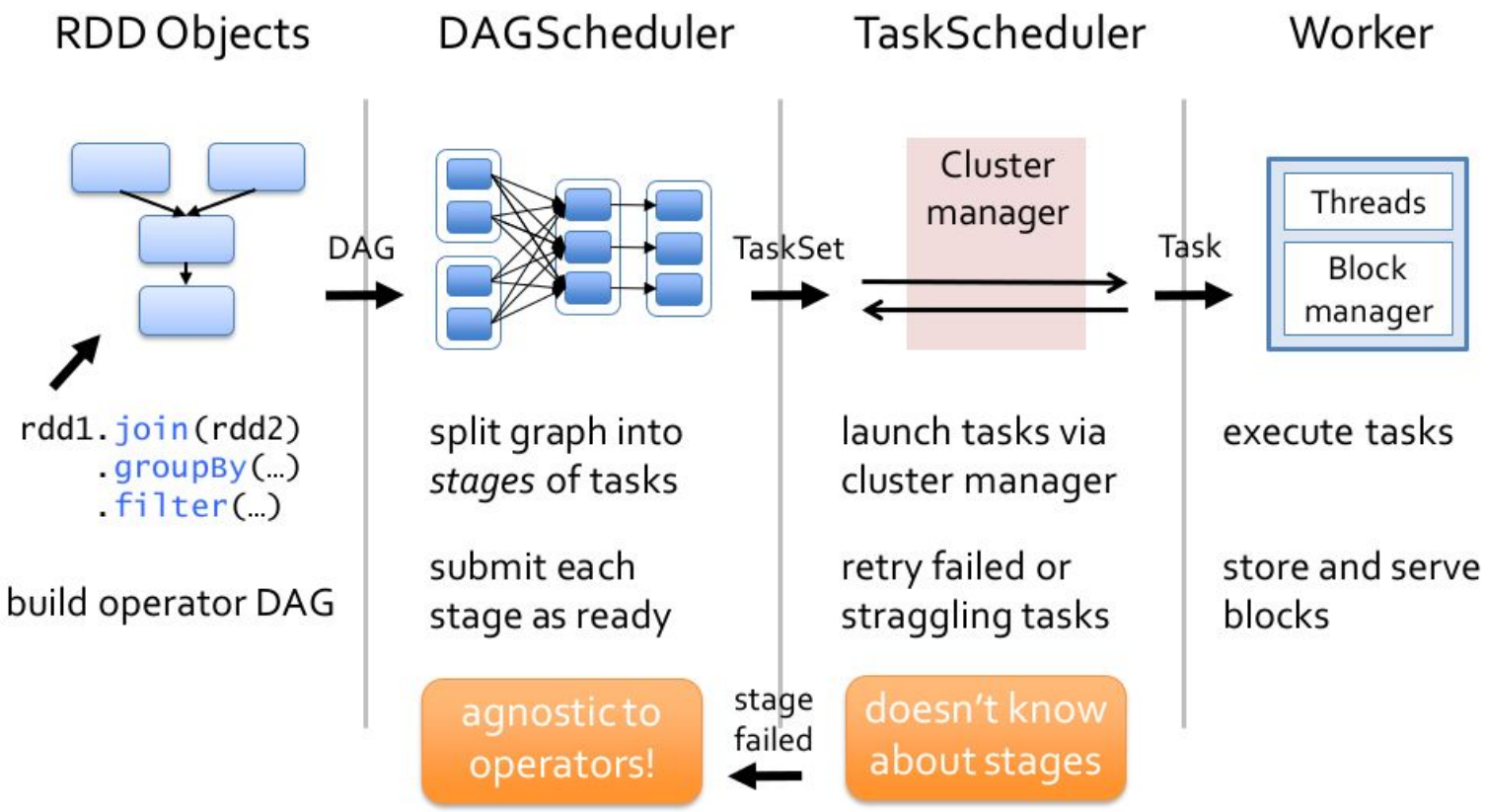
Spark Working details



https://data-flair.training/blogs/how-apache-spark-works/

BIG DATA

Spark Working details



https://hxquangnhat.com/2015/04/03/arch-spark-job-submission-breakdown/

Lazy Execution in Spark

BIG DATA

Lazy Execution

- In Hadoop, when we submit a job the master starts executing it
- In Spark, when does the master start executing the job?
  - **Spark** uses a technique called **Lazy execution**





Lazy Execution

- Remember that we defined Spark operations into *transformations* and *actions*
- The spark driver does not execute anything till it encounters an *action*
- *Transformations* are only noted for purpose of lineage.

Advantages of Lazy Evaluation in Spark Transformation

a. Increases Manageability

By lazy evaluation, users can organize their Apache Spark program into smaller operations. It reduces the number of passes on data by grouping operations.

b. Saves Computation and increases Speed

Spark Lazy Evaluation plays a key role in saving calculation overhead. Since only necessary values get compute. It saves the trip between driver and cluster, thus speeds up the process.

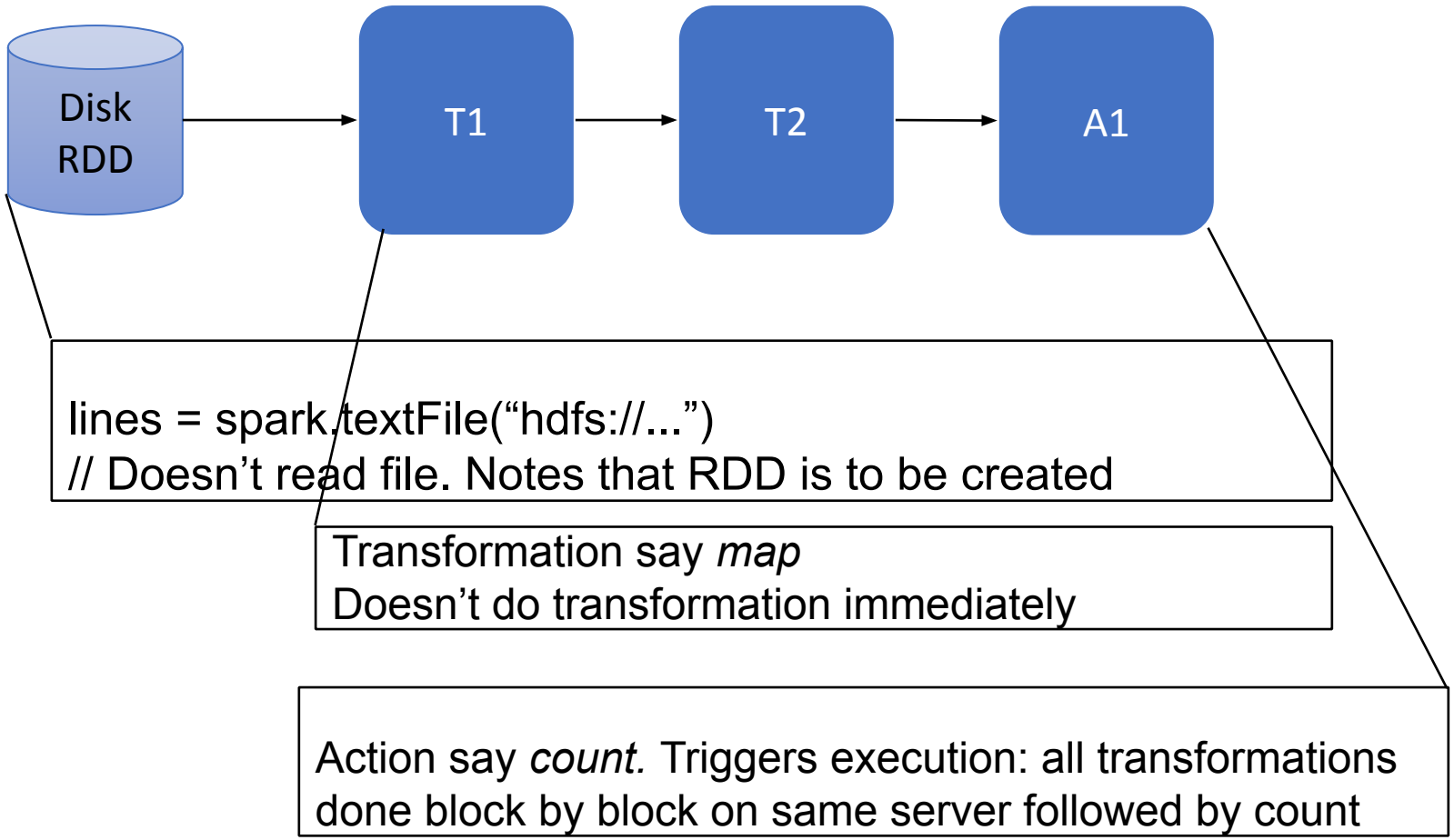
c. Reduces Complexities

The two main complexities of any operation are time and space complexity. Using Apache Spark lazy evaluation we can overcome both. Since we do not execute every operation, Hence, the time gets saved. It let us work with an infinite data structure. The action is triggered only when the data is required, it reduces overhead.

d. Optimization

It provides optimization by reducing the number of queries. Learn more about Apache Spark Optimization.

Lazy Execution



Lazy evaluation delays execution of transformations until an action is invoked. This avoids unnecessary computation and intermediate materialization and allows Spark to optimize the execution plan.

Spark Working with Log Mining Example

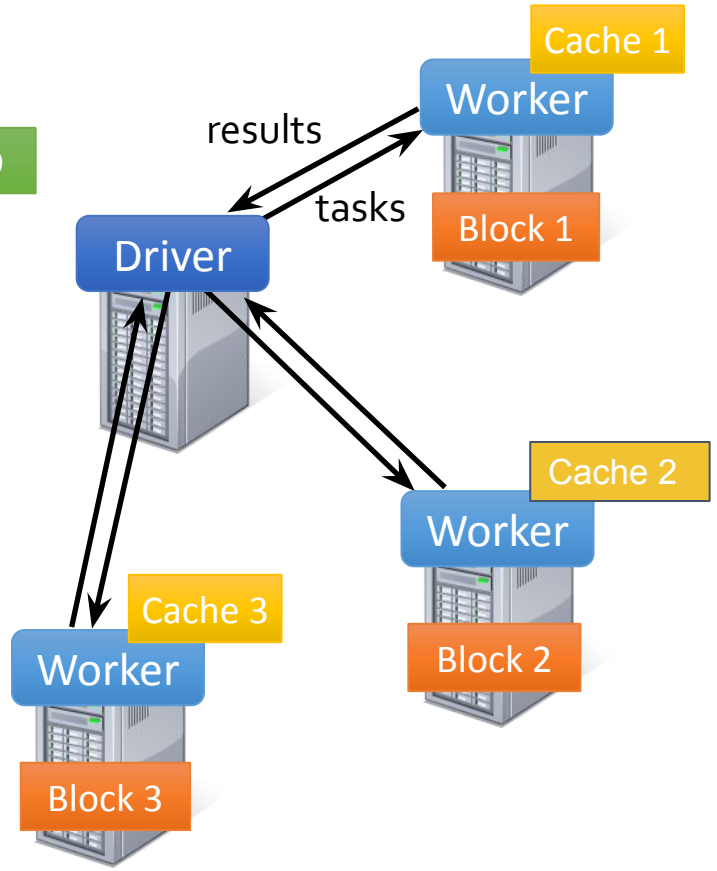
```
lines = spark.textFile("hdfs://...")
errors = lines.filter(startsWithERROR())
messages = errors.map(split("\t"),2)
cachedMsgs = messages.cache()

cachedMsgs.filter(containsfoo()).count()
cachedMsgs.filter(containsbar()).count()
```

Base RDD

Transformed RDD

Action



Spark Scheduling

BIG DATA

Page Rank example in Spark

```
lines = textfile (“urls.txt”))

links = lines.map (lambda urls:
urls.split()).groupByKey().cache()

ranks = links.map(lambda url_neighbors:
(url_neighbors[0], 1.0))

 for iteration in range(MAXITER)):

        contribs = links.join(ranks).flatMap(
lambda url_neighbors_rank: computeContribs
(url_neighbors_rank)

        ranks =
contribs.reduceByKey(add).mapValues(lambda
rank: rank * 0.85 + 0.15)
```

```
def computeContribs (url_neighbors_rank):
"""Calculates URL contributions to the rank of other URLs.
"""
num_neighbors = len (url_neighbors_rank) – 2

rank = url_neighbors_rank [len (url_neighbors_rank) – 1]

for i in range (1, num_neighbors):
      yield (url_neighbors_rank[i], rank / num_neighbors)
```

1. Start each page with a rank of 1
2. On each iteration, update each page’s rank to

$$\Sigma_{i \in neighbors} rank_i / |neighbors_i|$$

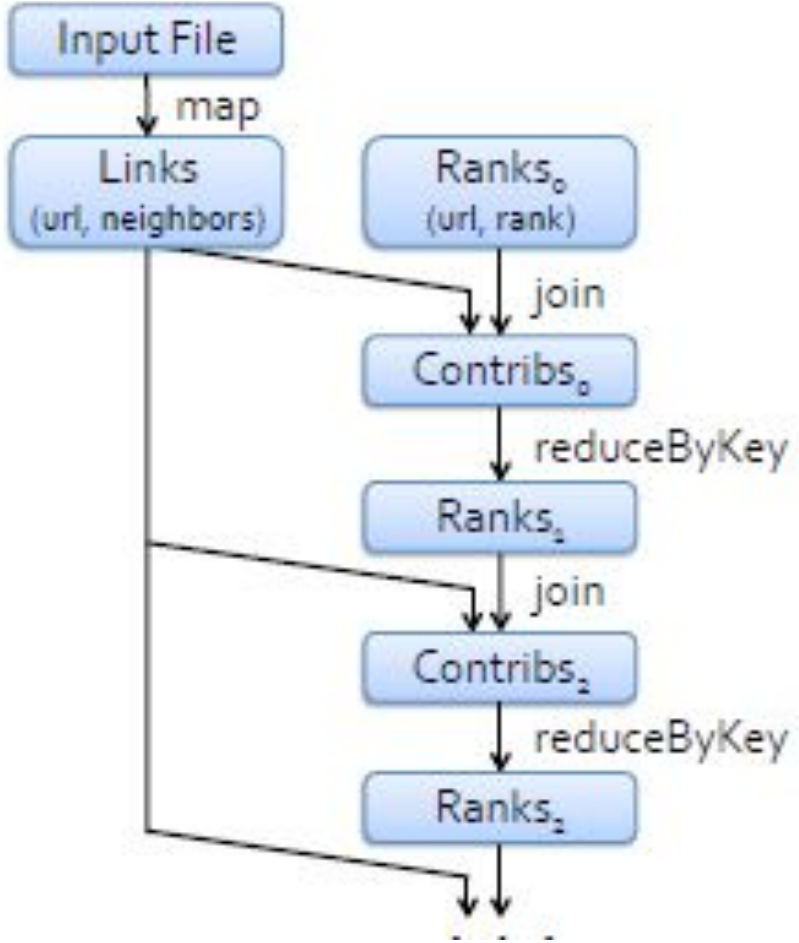
DAG representation

- The Spark Driver will first convert this program into a DAG representation

- What does the DAG representation contain?

    - Each RDD is a node in the graph and
    - all transformations/actions on the RDD as edges

DAG representation of Page Rank




```
lines = textfile ("urls.txt"))

links = lines.map (lambda urls:
urls.split()).groupByKey().cache()

ranks = links.map(lambda url_neighbors:
(url_neighbors[0], 1.0))

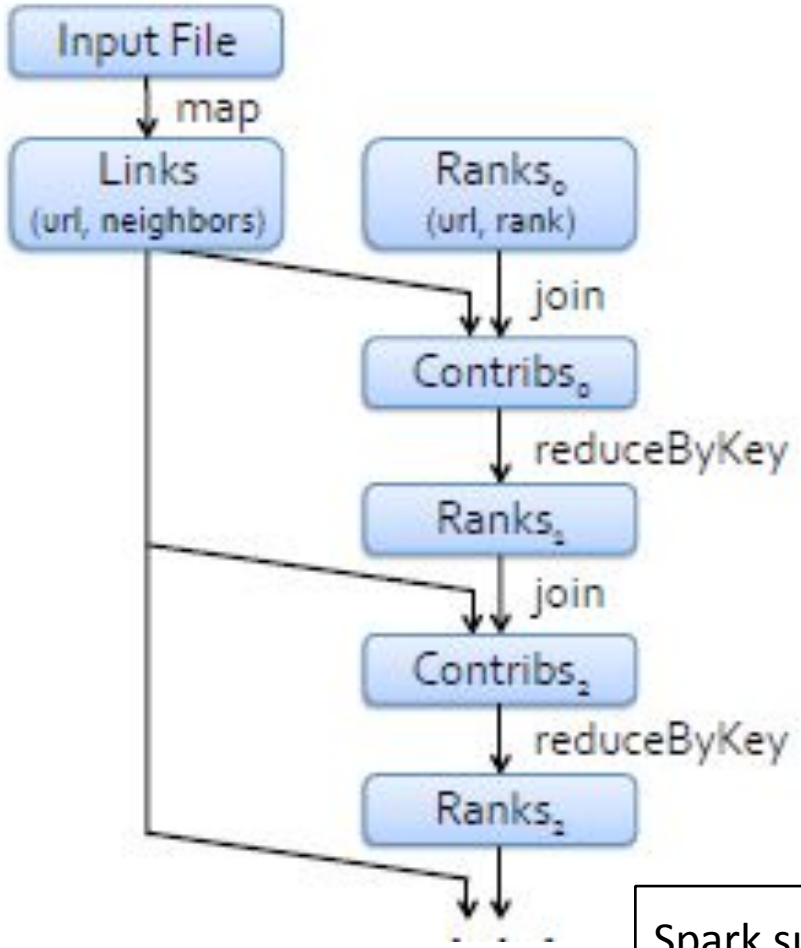
 for iteration in range(MAXITER)):

      contribs = links.join(ranks).flatMap(
lambda url_neighbors_rank:
computeContribs
(url_neighbors_rank)

      ranks =
contribs.reduceByKey(add).mapValues(la
mbda rank: rank * 0.85 + 0.15)
```

Ranks and Links are spread across multiple nodes. How does Spark ensure join works properly? Hint: Think about how join works.

DAG representation of Page Rank



links and ranks are repeatedly joined

Each join requires a full shuffle over the network

» Hash both onto same nodes

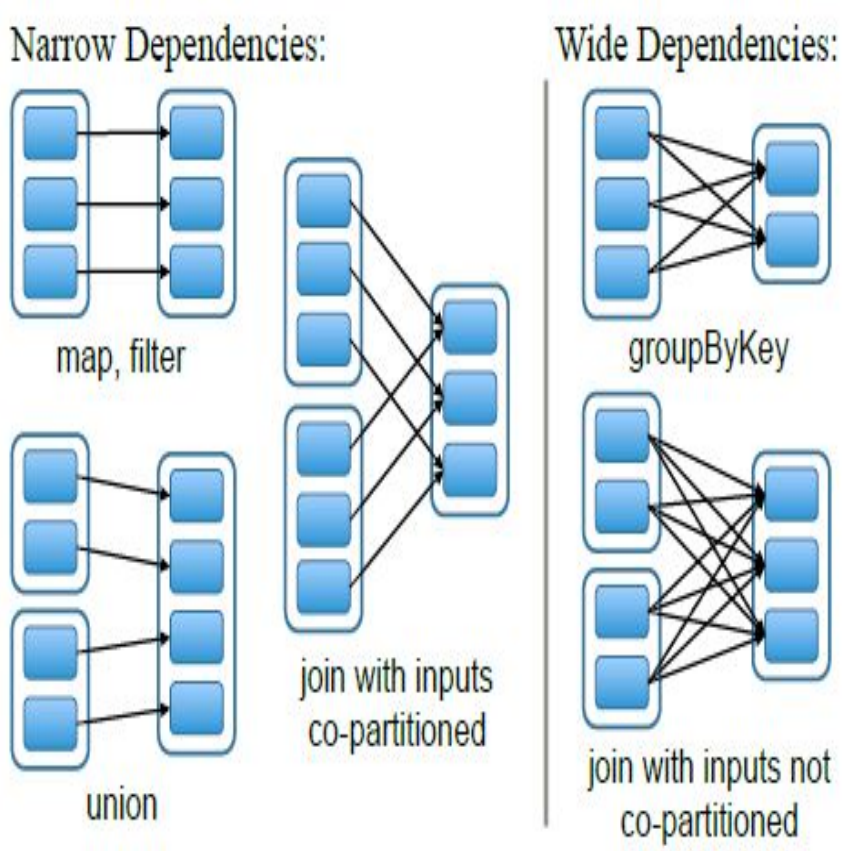


Spark supports two types of partitioning – hash and range

- **narrow dependencies**
  - **where each partition of** the parent RDD is used by at most one partition of the child RDD
  - Does not need a shuffle; pipeline operations
  - Shuffle: movement of data from one node to another

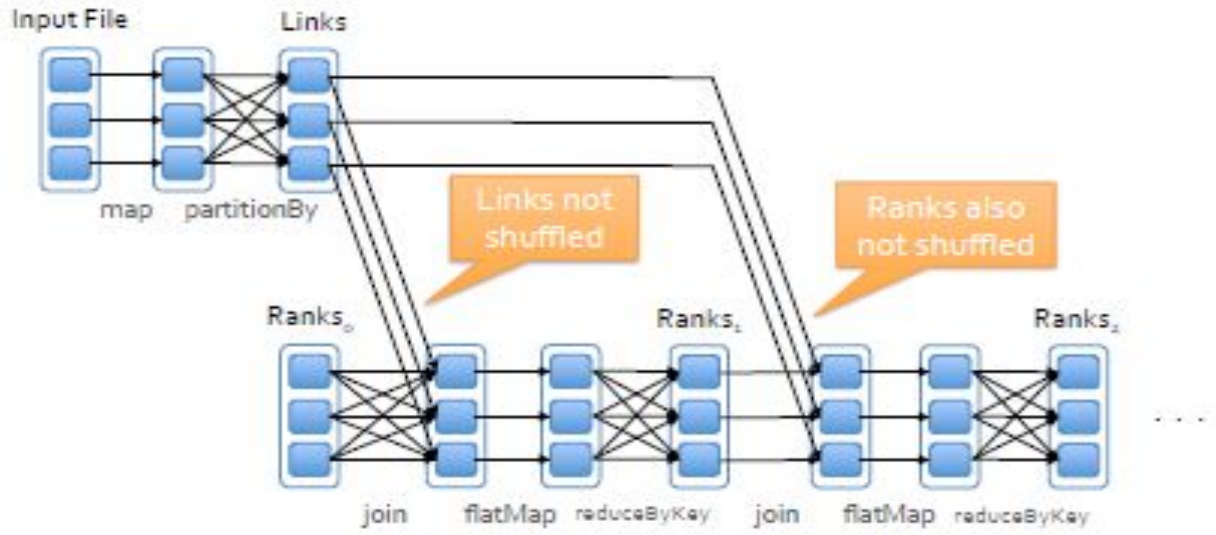
- **wide dependencies**
  - where multiple child partitions may depend on it.
  - May need a shuffle

- Copartition - technique to make sure that both inputs to a join are partitioned using same function



Lineage and Optimizing Placement

- links & ranks repeatedly joined
- Can copartition them (e.g.hash both on URL) to avoid shuffles
- Spark supports two types of partitioning: hash and range




```
lines = textfile ("urls.txt"))

links = lines.map (lambda urls:
urls.split()).groupByKey().cache()

ranks = links.map(lambda url_neighbors:
(url_neighbors[0], 1.0))

 for iteration in range(MAXITER)):

        contribs = links.join(ranks).flatMap( lambda
url_neighbors_rank: computeContribs
(url_neighbors_rank)

        ranks =
contribs.reduceByKey(add).mapValues(lambda
rank: rank * 0.85 + 0.15)
```

Narrow and Wide Dependencies

Narrow Dependencies

- Map
- FlatMap
- MapPartitions
- Filter
- Sample
- Union

Wide Dependencies

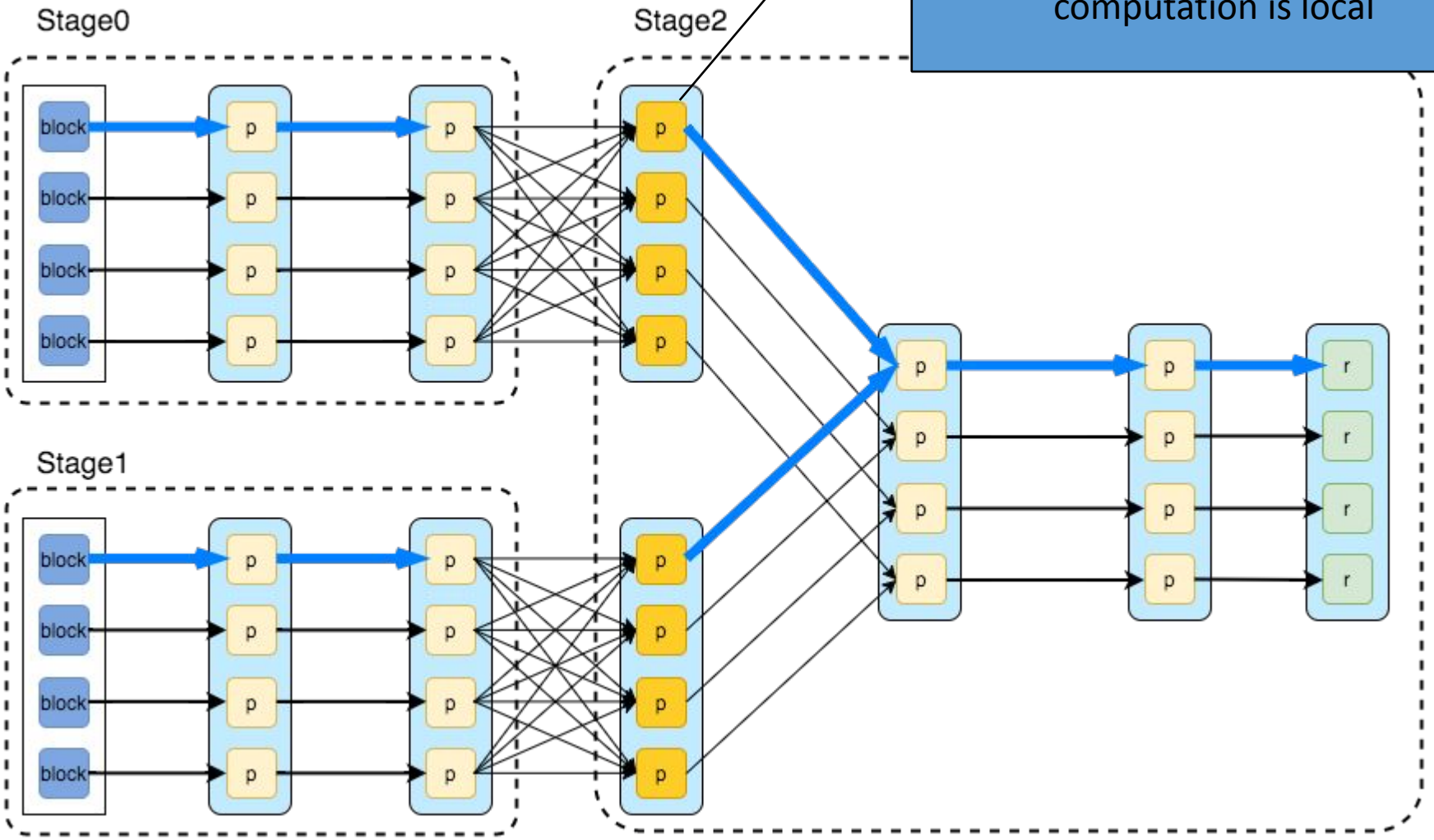
- Intersection
- Distinct
- ReduceByKey
- GroupByKey
- Join
- Cartesian
- Repartition
- Coalesce

PES UNIVERSITY
CELEBRATING 50 YEARS

BIG DATA

Scheduling

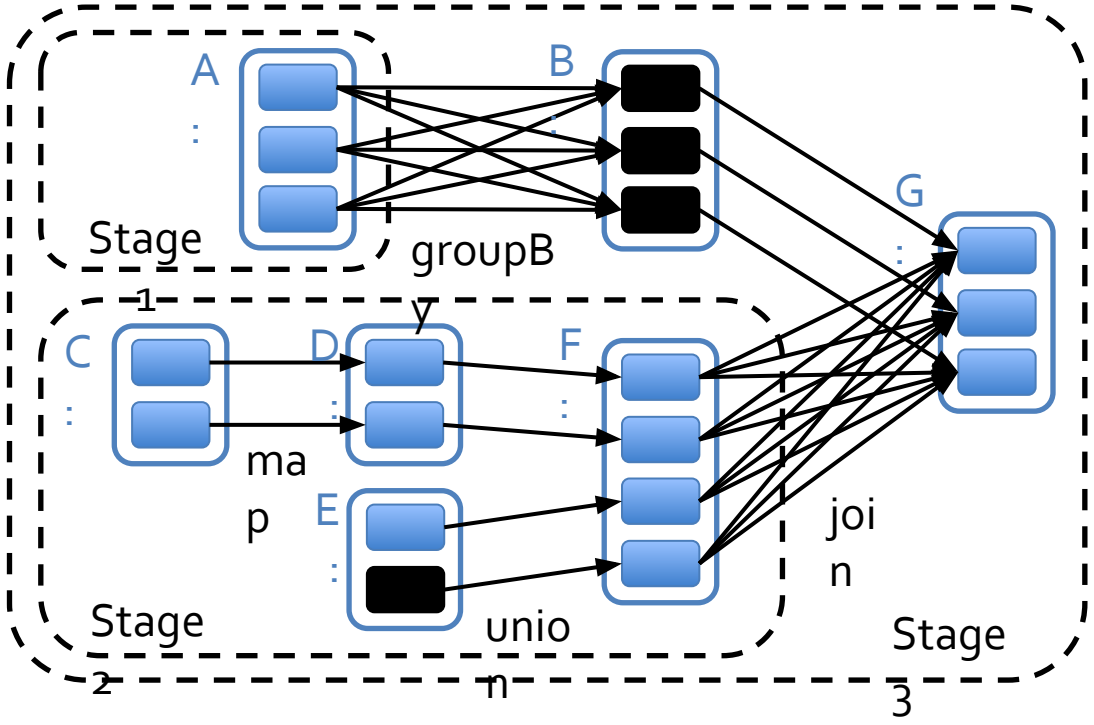
Breakup DAG into stages at shuffle boundaries. Within stage computation is local



BIG DATA

Task Assignment

- scheduler assigns tasks to machines based on data locality using delay scheduling
  - if a task needs to process a partition that is available in
  - memory on a node, then send it to that node
  - otherwise, a task processes a partition for which the containing RDD provides preferred locations (e.g., an HDFS file), then send it to those





THANK YOU

Prafullata Kiran Auradkar
Department of CSE
prafullatak@pes.edu



BIG DATA

RDD

Prafullata Kiran Auradkar

Department of Computer Science and Engineering

prafullatak@pes.edu

Acknowledgements:

Significant information in the slide deck presented through the Unit 1 of the course have been created by **Dr. K V Subramaniam's** and would like to acknowledge and thank him for the same. There have been some information which I might have leveraged from **Dr. H L Phalachandra's** slide contents too. I may have supplemented the same with contents from books and other sources from Internet and would like to sincerely thank, acknowledge and reiterate that the credit/rights for the same remain with the original authors/publishers only. These are intended for classroom presentation only.

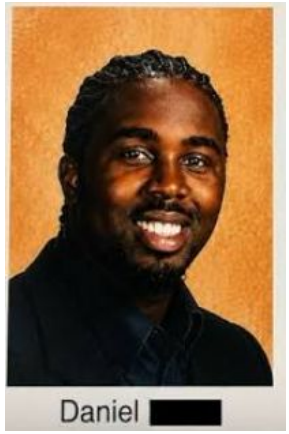
RDDs - details

References

Learning Spark Lightning-Fast Data Analysis- Holden Karau, Andy Konwinski, Patrick Wendell & Matei Zaharia
https://www.usenix.org/system/files/conference/nsdi12/nsdi12-final138.pdf

What is an RDD

- RDD is partitioned, locality aware, distributed collections
  - RDDs are immutable. (Why it's necessary?)
- RDDs are data structures that either
  - Point to the source (HDFS)
  - Apply some transformations to the parent RDDs to generate new data elements
- Computations on RDDs
  - Lazily evaluated lineage DAGs composed of chained RDDs



RDD 1

Transformations



RDD 2

Why the RDD abstraction?

- Support operations other than map and reduce
- Support in memory computation
- Arbitrary composition of such operators
- Simplify scheduling

How to capture dependencies generically?

Representing RDDs

- Set of partitions ("splits")
  - Much like Hadoop. Each RDD associated with a input partitions
- List of dependencies on parent RDDs
  - Not there in Hadoop. This is new
- Function to compute a partition given parents
  - User defined code. (similar to map()/reduce() in Hadoop
- Optional preferred locations
  - For data locality
- Optional partitioning information (partitioner)
  - Advanced – for shuffle (see later)

BIG DATA

RDDs Interface

Operation	Meaning
partitions()	Return a list of Partition objects
preferredLocations(*p*)	List nodes where partition *p* can be accessed faster due to data locality
dependencies()	Return a list of dependencies
iterator(*p, parentIters*)	Compute the elements of partition *p* given iterators for its parent partitions
partitioner()	Return metadata specifying whether the RDD is hash/range partitioned

https://medium.com/@shagun/resilient-distributed-datasets-97c28c3a9411

Examples of RDDs

- Hadoop RDD
  - Partitions – one per block
  - Dependencies – none
  - Compute (partition) – read corresponding block
  - Preferred locations – HDFS block location
  - Partitioner - none
- Filtered RDD (as in sample application)
  - Partitions – same as parent
  - Dependencies – 1-1 with parent
  - Compute – compute parent and filter it.
  - Preferred locations – ask parent (none)
  - Partitioner - none

PES UNIVERSITY
CELEBRATING 50 YEARS

Exercise

Based on the sample of the Filter RDD, can you work out what will be the partitions, compute, dependencies, preferred locations and partitioner for a joinRDD

- Filtered RDD (as in sample application)
  - Partitions – same as parent
  - Dependencies – 1-1 with parent
  - Compute – compute parent and filter it.
  - Preferred locations – ask parent (none)
  - Partitioner - none

PES UNIVERSITY
CELEBRATING 50 YEARS

Examples of RDDs

- Joined RDD
  - RDDPartitions – one per reduce task
  - Dependencies – shuffle on each parent
  - Compute (partition) – read and join shuffled data
  - Preferred locations – none
  - Partitioner – HashPartioner (num tasks)

Adding fault tolerance – The RDD

BIG DATA

Handling fault tolerance

PES UNIVERSITY
CELEBRATING 50 YEARS

Consider the following code:

```
messages = textFile(...).filter(startsWithERROR()) .map(split("\t")(2))
```



Step1: Read in the file to an in memory RDD

Step2: remove all lines that don't contain the term ERROR

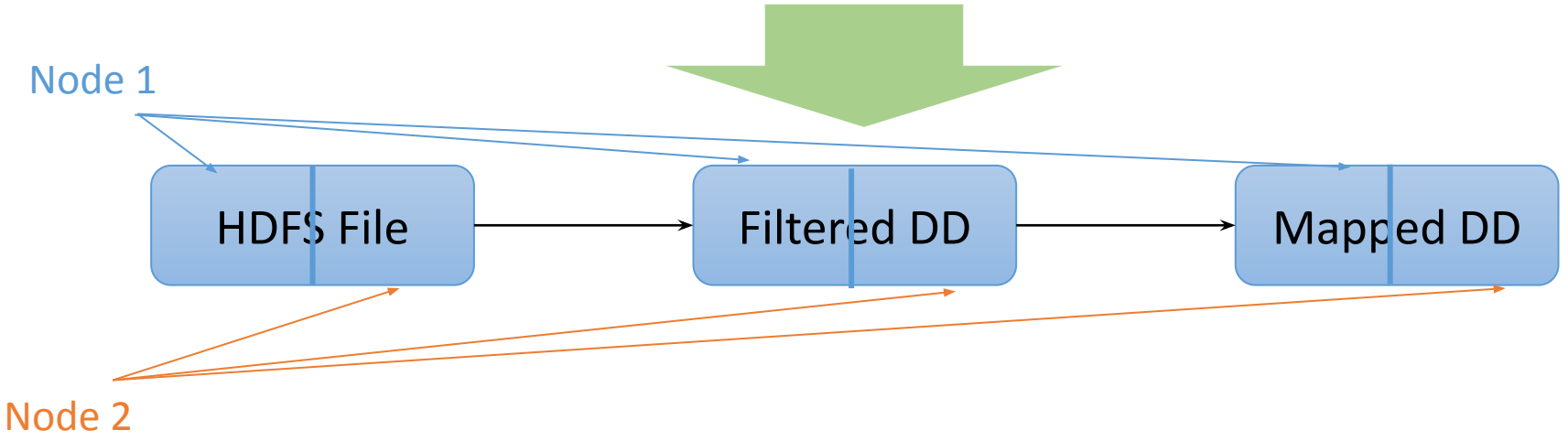
Step3: split the line

map(*func*)	Return a new distributed dataset formed by passing each element of the source through a function *func*.
filter(*func*)	Return a new dataset formed by selecting those elements of the source on which *func* returns true.

Handling fault tolerance

Ex:

```
messages = textFile(...).filter(startsWithERROR())
                        .map(split("\t")(2))
```



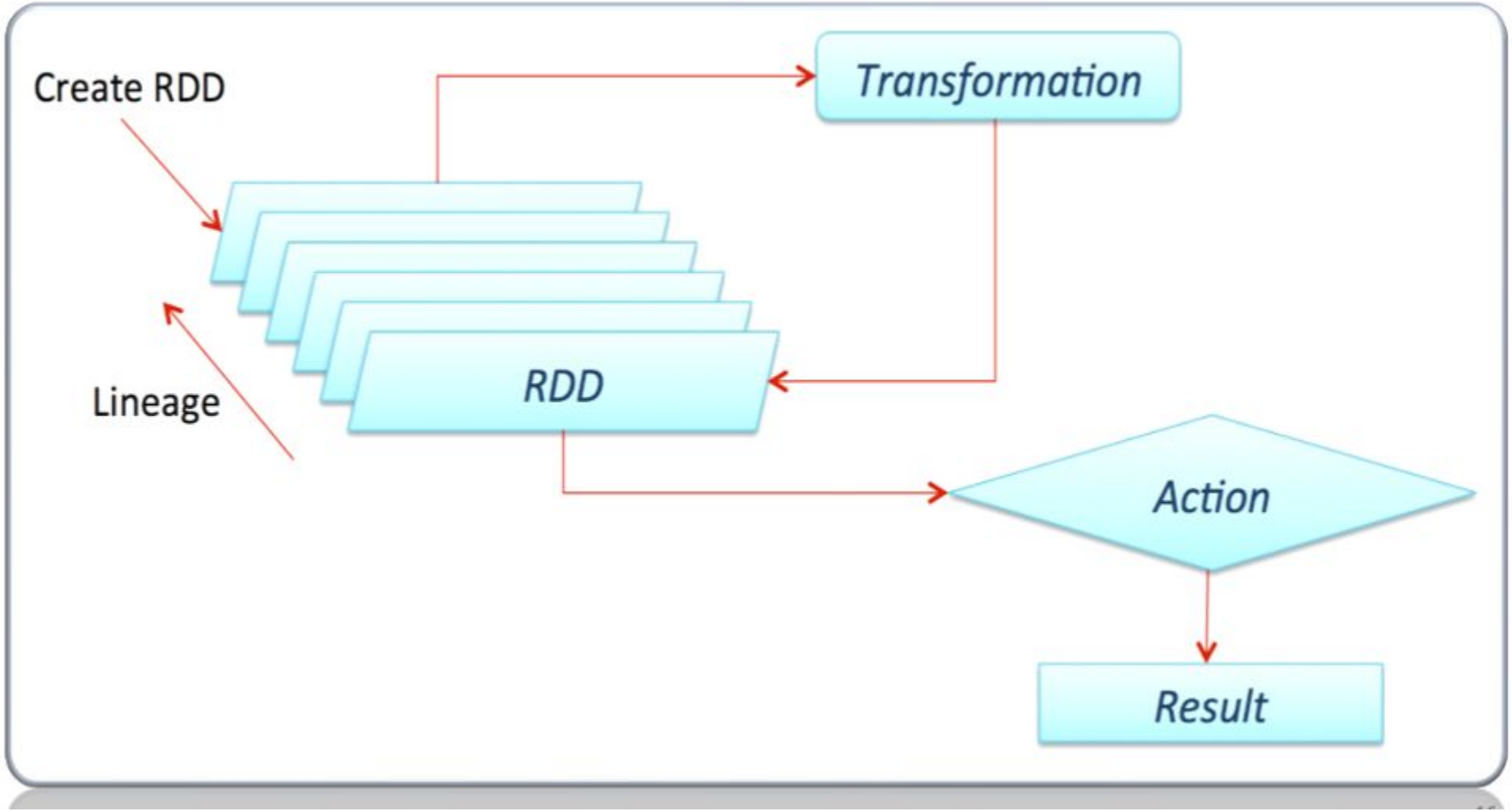
RDD Operations : Transformations and Actions

BIG DATA

Types of Operations

- Operations are of two types
  - ***Transformations***
  - ***Actions***

Iterative Operations on Spark RDD



Transformation

- Are operations that create a *new dataset* from an existing dataset
- For example:
  - *map()* is a transformation
    - Each line on input RDD is passed through the *map()* function
    - result of *map()* function applied on each value is stored in the output RDD.
- Note it is similar to the Map of map-reduce, but is more generic.

Transformations

Sr.	Transformation and Meaning
1	**map(func)** Returns a new distributed dataset, formed by passing each element of the source through a function func.
2	**filter(func)** Returns a new dataset formed by selecting those elements of the source on which func returns true.
3	**flatMap(func)** Similar to map, but each input item can be mapped to 0 or more output items (so func should return a Seq rather than a single item).
4	**mapPartitions(func)** Similar to map, but runs separately on each partition (block) of the RDD, so func must be of type Iterator<T> => Iterator<U> when running on an RDD of type T.



Transformations

Sr.	Transformation and Meaning
5	**mapPartitionsWithIndex(func)** Similar to map Partitions, but also provides func with an integer value representing the index of the partition, so func must be of type (Int, Iterator<T>) => Iterator<U> when running on an RDD of type T.
6	**sample(withReplacement, fraction, seed)** Sample a fraction of the data, with or without replacement, using a given random number generator seed.
7	**union(otherDataset)** Returns a new dataset that contains the union of the elements in the so dataset and the argument.

Transformations

Sr.	Transformation and Meaning
8	**intersection(otherDataset)** Returns a new RDD that contains the intersection of elements in the source dataset and the argument.
9	**distinct([numTasks]))** Returns a new dataset that contains the distinct elements of the source dataset.
10	**groupByKey([numTasks])** When called on a dataset of (K, V) pairs, returns a dataset of (K, Iterable<V>) pairs. Note: If you are grouping in order to perform an aggregation (such as a sum or average) over each key, using reduceByKey or aggregateByKey will yield much better performance.

How groupByKey and reduceByKey are different?

Transformations

Sr.	Transformation and Meaning
11	**reduceByKey(func, [numTasks])** When called on a dataset of (K, V) pairs, returns a dataset of (K, V) pairs where the values for each key are aggregated using the given reduce function func, which must be of type (V, V) => V.
12	**aggregateByKey(zeroValue)(seqOp, combOp, [numTasks])** When called on a dataset of (K, V) pairs, returns a dataset of (K, U) pairs where the values for each key are aggregated using the given combine functions and a neutral "zero" value. Allows an aggregated value type that is different from the input value type, while avoiding unnecessary allocations.
13	**sortByKey([ascending], [numTasks])** When called on a dataset of (K, V) pairs where K implements Ordered, returns a dataset of (K, V) pairs sorted by keys in ascending or descending order, as specified in the Boolean ascending argument.

Transformations

Sr.	Transformation and Meaning
17	**pipe(command, [envVars])** Pipe each partition of the RDD through a shell command, e.g. a Perl or bash script. RDD elements are written to the process's stdin and lines output to its stdout are returned as an RDD of strings.
18	**coalesce(numPartitions)** Decrease the number of partitions in the RDD to numPartitions. Useful for running operations more efficiently after filtering down a large dataset.
19	**repartition(numPartitions)** Reshuffle the data in the RDD randomly to create either more or fewer partitions and balance it across them. This always shuffles all data over the network.
20	**epartitionAndSortWithinPartitions(partitioner)** Repartition the RDD according to the given partitioner and, within each resulting partition, sort records by their keys. This is more efficient than calling repartition and then sorting within each partition because it can push the sorting down into the shuffle machinery.

Actions

- Are operations that return a value

- For example:
  - Reduce() is an action

  - Aggregates all elements of a RDD to produce a result.

Actions

Sr.	Actions and Meaning
5	**take(n)** Returns an array with the first n elements of the dataset.
6	**takeSample (withReplacement,num, [seed])** Returns an array with a random sample of num elements of the dataset, with or without replacement, optionally pre-specifying a random number generator seed.
7	**takeOrdered(n, [ordering])** Returns the first n elements of the RDD using either their natural order or a custom comparator.
8	**saveAsTextFile(path)** Writes the elements of the dataset as a text file (or set of text files) in a given directory in the local filesystem, HDFS or any other Hadoop-supported file system. Spark calls toString on each element to convert it to a line of text in the file.

Actions

Sr.	Actions and Meaning
1	**reduce(func)** Aggregate the elements of the dataset using a function func (which takes two arguments and returns one). The function should be commutative and associative so that it can be computed correctly in parallel.
2	**collect()** Returns all the elements of the dataset as an array at the driver program. This is usually useful after a filter or other operation that returns a sufficiently small subset of the data.
3	**count()** Returns the number of elements in the dataset.
4	**first()** Returns the first element of the dataset (similar to take (1)).

Actions

Sr.	Actions and Meaning
9	**saveAsSequenceFile(path) (Java and Scala)** Writes the elements of the dataset as a Hadoop SequenceFile in a given path in the local filesystem, HDFS or any other Hadoop-supported file system. This is available on RDDs of key-value pairs that implement Hadoop's Writable interface. In Scala, it is also available on types that are implicitly convertible to Writable (Spark includes conversions for basic types like Int, Double, String, etc).
10	**saveAsObjectFile(path) (Java and Scala)** Writes the elements of the dataset in a simple format using Java serialization, which can then be loaded using SparkContext.objectFile().
11	**countByKey()** Only available on RDDs of type (K, V). Returns a hashmap of (K, Int) pairs with the count of each key.
12	**foreach(func)** Runs a function func on each element of the dataset. This is usually, done for side effects such as updating an Accumulator or interacting with external storage systems.

RDD Operations : Working with key-value pairs

Key Value Pairs

- Consider our earlier operation of map/reduce using Spark
- Worked on datasets with only single values
- Let's consider how to represent <*key, value*> pairs
- Spark provides
  - Separate RDDs called pair RDDs for this
  - Separate operations to function on Pair RDDs

BIG DATA

Pair RDD Transformations

Table 4-1. Transformations on one pair RDD (example: {(1, 2), (3, 4), (3, 6)})

Function name	Purpose	Example	Result
`reduceByKey(func)`	Combine values with the same key.	`rdd.reduceByKey( (x, y) => x + y)`	`{(1, 2), (3, 10)}`
`groupByKey()`	Group values with the same key.	`rdd.groupByKey()`	`{(1, [2]), (3, [4, 6])}`
`combineByKey(createCombiner, mergeValue, mergeCombiners, partitioner)`	Combine values with the same key using a different result type.	See Examples 4-12 through 4-14.	

BIG DATA

Pair RDD Transformations

`mapValues(func)`	Apply a function to each value of a pair RDD without changing the key.	`rdd.mapValues(x => x+1)`	`{(1, 3), (3, 5), (3, 7)}`
`flatMapValues(func)`	Apply a function that returns an iterator to each value of a pair RDD, and for each element returned, produce a key/value entry with the old key. Often used for tokenization.	`rdd.flatMapValues(x => (x to 5)`	`{(1, 2), (1, 3), (1, 4), (1, 5), (3, 4), (3, 5)}`
`keys()`	Return an RDD of just the keys.	`rdd.keys()`	`{1, 3, 3}`
`values()`	Return an RDD of just the values.	`rdd.values()`	`{2, 4, 6}`

Pair RDD Actions

countByKey(k, V) -> returns a HashMap of (k, Int) key value pairs with count of each key

Persistence

- Spark can persist (or cache) a dataset in memory across operations
- Each node stores in memory any slices of it that it computes and reuses them in other actions on that dataset – often making future actions more than 10x faster
- The cache is fault-tolerant:
- if any partition of an RDD is lost, it will automatically be recomputed using the transformations that originally created it

Persistence

transformation	description
MEMORY_ONLY	Store RDD as deserialized Java objects in the JVM. If the RDD does not fit in memory, some partitions will not be cached and will be recomputed on the fly each time they're needed. This is the default level.
MEMORY_AND_DISK	Store RDD as deserialized Java objects in the JVM. If the RDD does not fit in memory, store the partitions that don't fit on disk, and read them from there when they're needed.
MEMORY_ONLY_SER	Store RDD as serialized Java objects (one byte array per partition). This is generally more space-efficient than deserialized objects, especially when using a fast serializer, but more CPU-intensive to read.
MEMORY_AND_DISK_SER	Similar to MEMORY_ONLY_SER, but spill partitions that don't fit in memory to disk instead of recomputing them on the fly each time they're needed.
DISK_ONLY	Store the RDD partitions only on disk.
MEMORY_ONLY_2, MEMORY_AND_DISK_2, etc	Same as the levels above, but replicate each partition on two cluster nodes.

Persistence

PES UNIVERSITY
CELEBRATING 50 YEARS

Scala:

```
val f = sc.textFile("README.md")
val w = f.flatMap(l => l.split(" ")).map(word => (word, 1)).cache()
w.reduceByKey(_ + _).collect.foreach(println)
```

Python:

```
from operator import add
f = sc.textFile("README.md")
w = f.flatMap(lambda x: x.split(' ')).map(lambda x: (x, 1)).cache()
w.reduceByKey(add).collect()
```

Task Assignment

DryadLINQ, FlumeJava

- Similar “distributed collection” API, but cannot reuse datasets efficiently *across* queries

- Relational databases
  - Lineage/provenance, logical logging, materialized views

GraphLab, Piccolo, BigTable, RAMCloud

- Fine-grained writes similar to distributed shared memory

- Iterative MapReduce (e.g. Twister, HaLoop)
  - Implicit data sharing for a fixed computation pattern

- Caching systems (e.g. Nectar)
  - Store data in files, no explicit control over what is cached

BIG DATA

Shared Variables

- Spark contains two different types of shared variables – one is broadcast variables and second is accumulators.
- Broadcast variables – used to efficiently, distribute large values.
- Accumulators – used to aggregate the information of particular collection.

Shared Variables

Broadcast Variables

- Broadcast variables allow the programmer to keep a read-only variable cached on each machine rather than shipping a copy of it with tasks. They can be used, for example, to give every node, a copy of a large input dataset, in an efficient manner. Spark also attempts to distribute broadcast variables using efficient broadcast algorithms to reduce communication cost.

- Spark actions are executed through a set of stages, separated by distributed “shuffle” operations. Spark automatically broadcasts the common data needed by tasks within each stage.

- The data broadcasted this way is cached in serialized form and is deserialized before running each task. This means that explicitly creating broadcast variables, is only useful when tasks across multiple stages need the same data or when caching the data in deserialized form is important.

Shared Variables

Accumulators

- Spark Accumulators are shared variables which are only “added” through an associative and commutative operation and are used to perform counters (Similar to Map-reduce counters) or sum operations

- Spark by default supports to create an accumulators of any numeric type and provide a capability to add custom accumulator types



THANK YOU

Prafullata Kiran Auradkar
Department of CSE
prafullatak@pes.edu



BIG DATA

RDD Operations : Transformations and Actions

Prafullata Kiran Auradkar

Department of Computer Science and Engineering

prafullatak@pes.edu

Acknowledgements:

Significant information in the slide deck presented through the Unit 1 of the course have been created by **Dr. K V Subramaniam's** and would like to acknowledge and thank him for the same. There have been some information which I might have leveraged from **Dr. H L Phalachandra's** slide contents too. I may have supplemented the same with contents from books and other sources from Internet and would like to sincerely thank, acknowledge and reiterate that the credit/rights for the same remain with the original authors/publishers only. These are intended for classroom presentation only.

RDD Operations : Transformations and Actions

Types of Operations

Spark Operations = 
TRANSFORMATIONS

+


ACTIONS

BIG DATA

Types of Operations

 = easy  = medium

Essential Core & Intermediate Spark Operations



General

- map
- filter
- flatMap
- mapPartitions
- mapPartitionsWithIndex
- groupBy
- sortBy

Math / Statistical

- sample
- randomSplit

Set Theory / Relational

- union
- intersection
- subtract
- distinct
- cartesian
- zip

Data Structure / I/O

- keyBy
- zipWithIndex
- zipWithUniqueID
- zipPartitions
- coalesce
- repartition
- repartitionAndSortWithinPartitions
- pipe



- reduce
- collect
- aggregate
- fold
- first
- take
- forEach
- top
- treeAggregate
- treeReduce
- forEachPartition
- collectAsMap

- count
- takeSample
- max
- min
- sum
- histogram
- mean
- variance
- stdev
- sampleVariance
- countApprox
- countApproxDistinct

- takeOrdered

- saveAsTextFile
- saveAsSequenceFile
- saveAsObjectFile
- saveAsHadoopDataset
- saveAsHadoopFile
- saveAsNewAPIHadoopDataset
- saveAsNewAPIHadoopFile

Transformation (Recap)

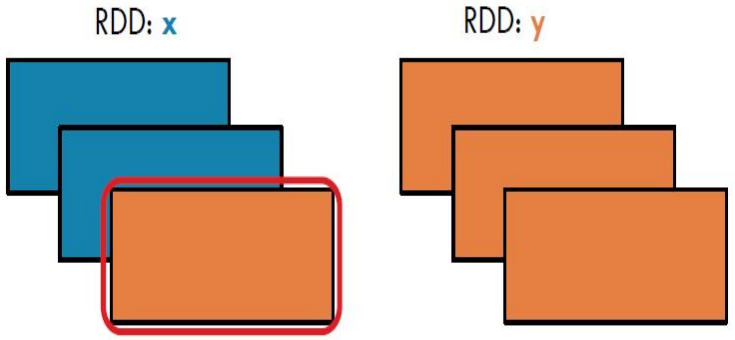
- Are operations that create a *new dataset* from an existing dataset
- For example:
  - *map()* is a transformation
    - Each line on input RDD is passed through the *map()* function
    - result of *map()* function applied on each value is stored in the output RDD.
- Note it is similar to the Map of map-reduce, but is more generic.

map(func)

The map function iterates over every line in RDD and split into new RDD.

Using map() transformation we take in any function, and that function is applied to every element of RDD.

In the map, we have the flexibility that the input and the return type of RDD may differ from each other. For example, we can have input RDD type as String, after applying the map() function the return RDD can be Boolean.




```
x = sc.parallelize(["b", "a", "c"])
y = x.map(lambda z: (z, 1))
print(x.collect())
print(y.collect())
```

A lambda function is a small anonymous function. It can take any number of arguments, but can only have one expression.

collect() is an action used to retrieve all the elements of the dataset

```
x: ['b', 'a', 'c']

y: [('b', 1), ('a', 1), ('c', 1)]
```

PES UNIVERSITY
CELEBRATING 50 YEARS

flatMap()

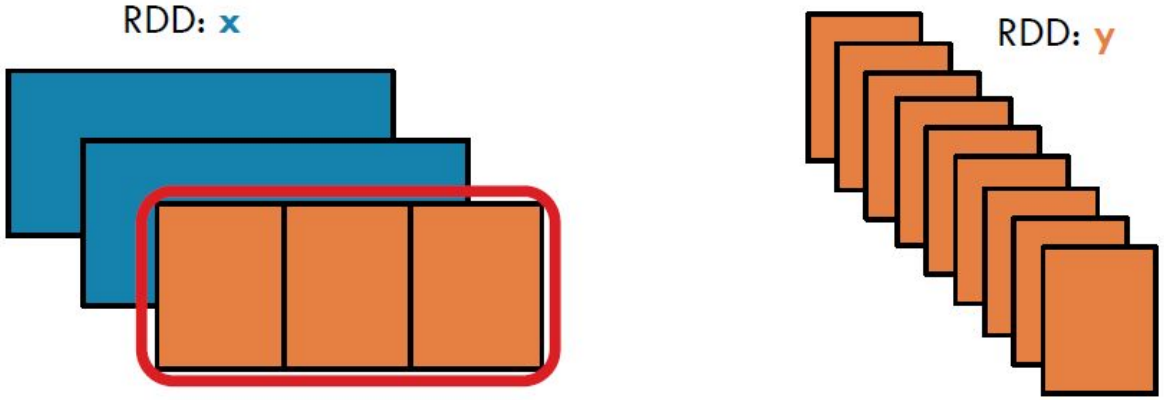
With the help of flatMap() function, to each input element, we have many elements in an output RDD. The most simple use of flatMap() is to split each input string into words.

Map and flatMap are similar in the way that they take a line from input RDD and apply a function on that line.

The key difference between map() and flatMap() is map() returns only one element, while flatMap() can return a list of elements.

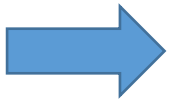
BIG DATA

Transformations - flatMap



Return a new RDD by first applying a function to all elements of this RDD, and then flattening the results

```
x = sc.parallelize([1,2,3])
y = x.flatMap(lambda x: (x, x*100, 42))
print(x.collect())
print(y.collect())
```



```
x: [1, 2, 3]

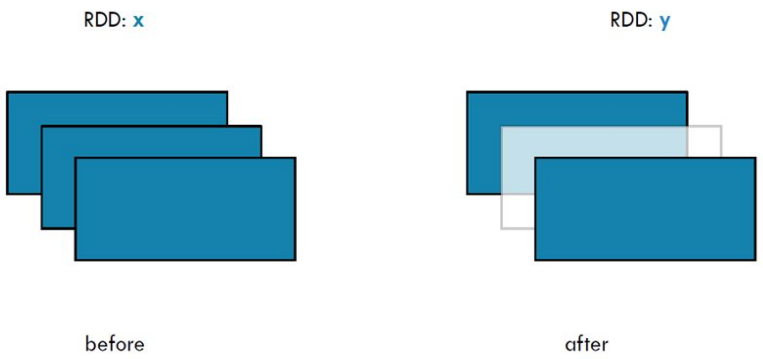
y: [1, 100, 42, 2, 200, 42, 3, 300, 42]
```

Transformations - filter

filter(func)

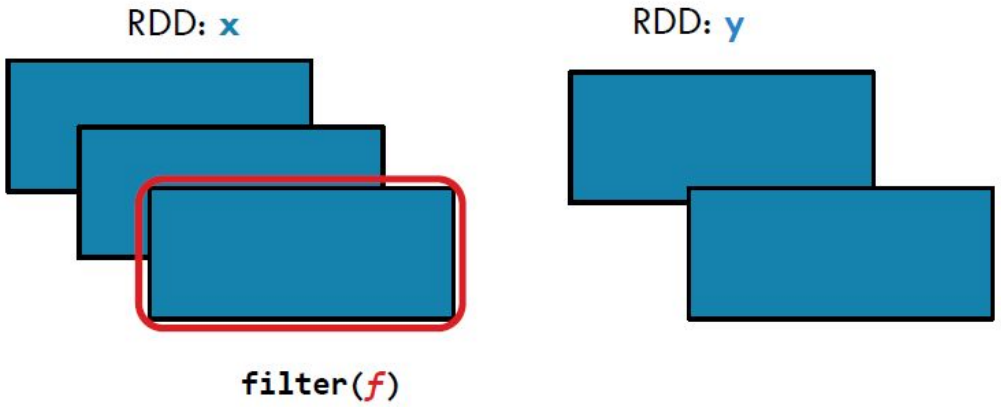
Spark RDD filter() function returns a new RDD, containing only the elements that meet a predicate. It is a narrow operation because it does not shuffle data from one partition to many partitions.

For example, Suppose RDD contains first five natural numbers (1, 2, 3, 4, and 5) and the predicate is check for an even number. The resulting RDD after the filter will contain only the even numbers i.e., 2 and 4.



BIG DATA

Transformations - filter



Return a new RDD containing only the elements that satisfy a predicate

```
x = sc.parallelize([1,2,3])
y = x.filter(lambda x: x%2 == 1) #keep odd values
print(x.collect())
print(y.collect())
```



```
x: [1, 2, 3]

y: [1, 3]
```

Transformations - groupBy

- Group the data in the original RDD. Create pairs where the key is the output of a user function, and the value is all items for which the function yields this key.

```
x = sc.parallelize(['John', 'Fred', 'Anna', 'James'])
y = x.groupBy(lambda w: w[0])
print ([(k, list(v)) for (k, v) in y.collect()])
```

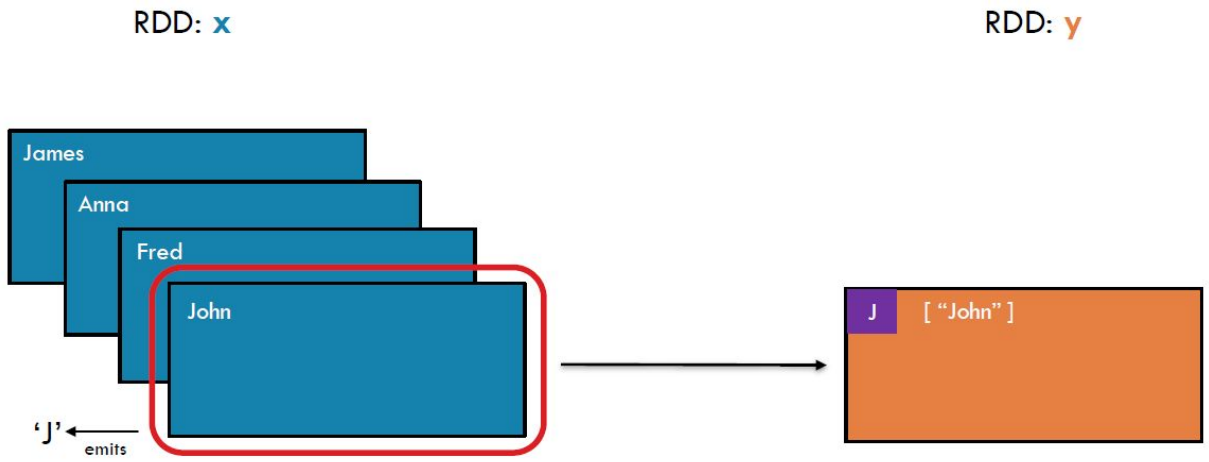


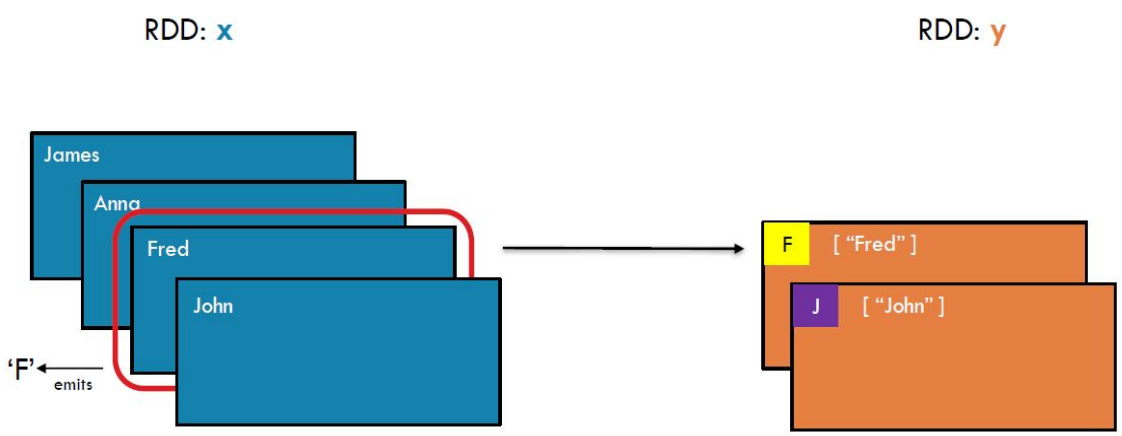
```
x: ['John', 'Fred', 'Anna', 'James']

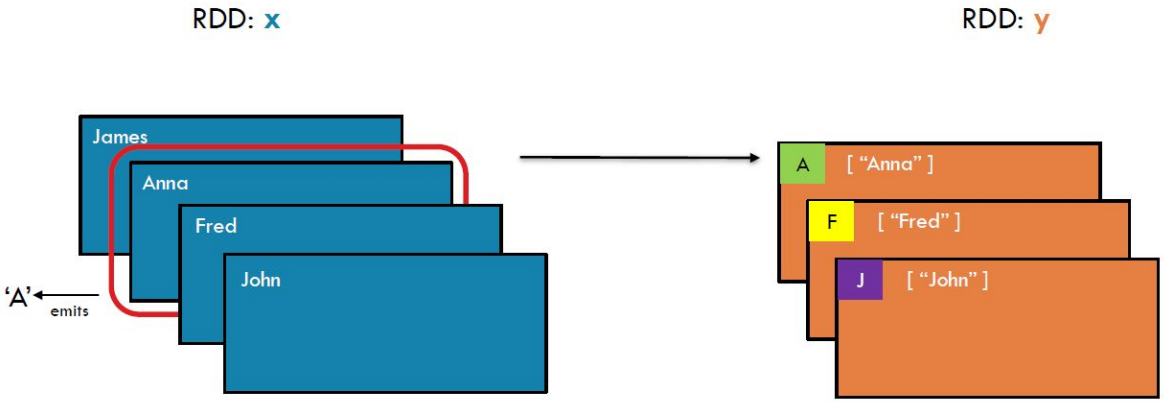
y: [('A',['Anna']),('J',['John','James']),('F',['Fred'])]
```

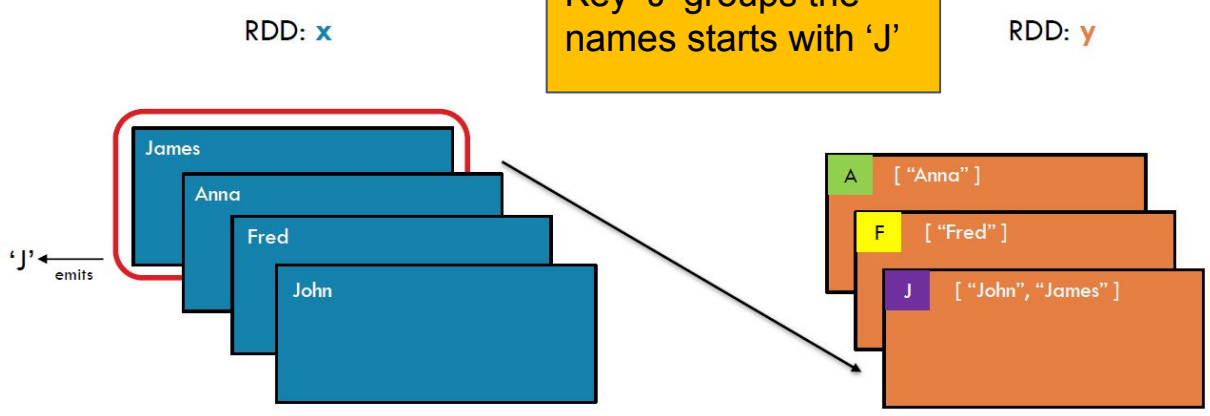
BIG DATA

Transformations – groupBy





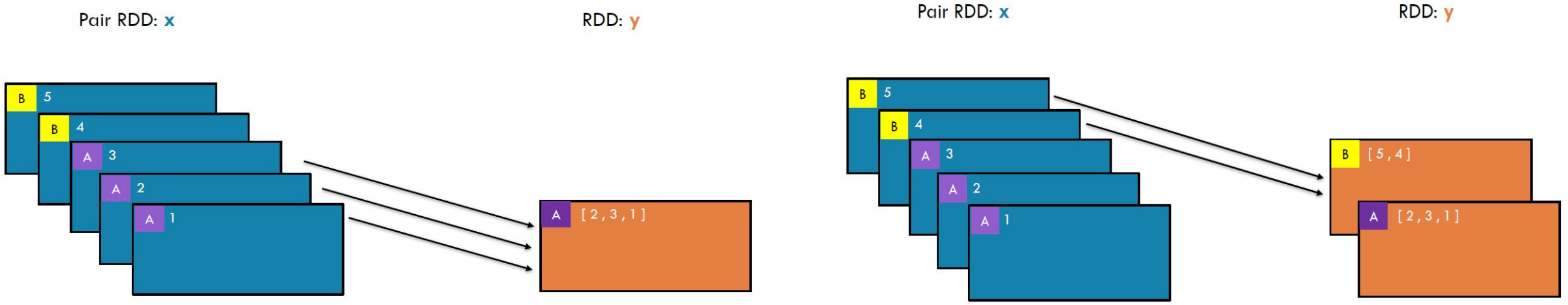




BIG DATA

Transformations - groupByKey

- Group the values for each key in the original RDD. Create a new pair where the original key corresponds to this collected group of values.




```
x = sc.parallelize([('B',5),('B',4),('A',3),('A',2),('A',1)])
y = x.groupByKey()
print(x.collect())
print(list((j[0], list(j[1])) for j in y.collect()))
```

```
x: [('B', 5),('B', 4),('A', 3),('A', 2),('A', 1)]

y: [('A', [2, 3, 1]),('B',[5, 4])]
```

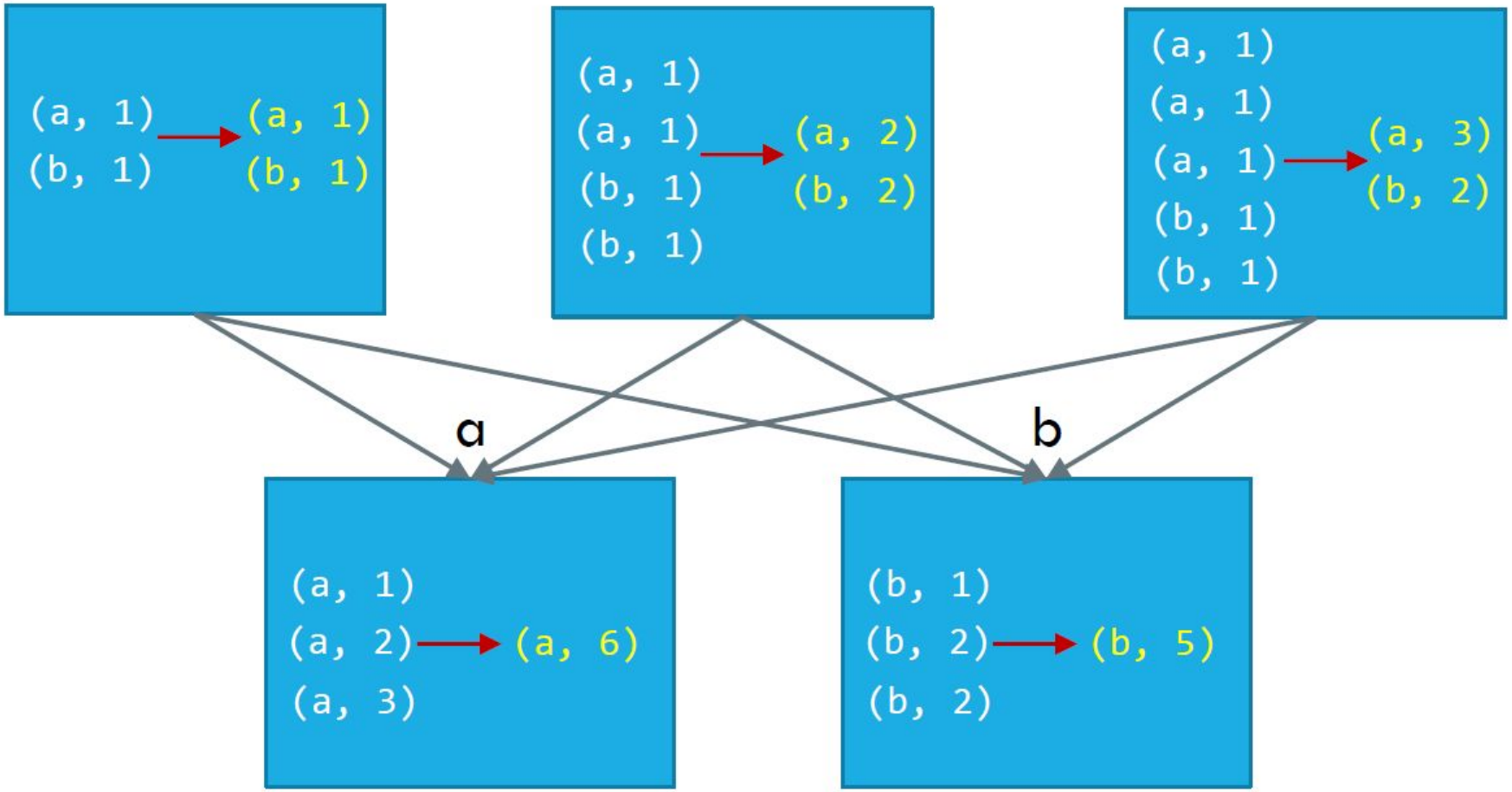
reduceByKey VS groupByKey

PES UNIVERSITY
CELEBRATING 50 YEARS

```
val words = Array("one", "two", "two", "three", "three", "three")
val wordPairsRDD = sc.parallelize(words).map(word => (word, 1))
val wordCountsWithReduce = wordPairsRDD.reduceByKey(_ + _).collect()
val wordCountsWithGroup = wordPairsRDD.groupByKey().map(t =>(t._1,t._2.sum)).collect()
```

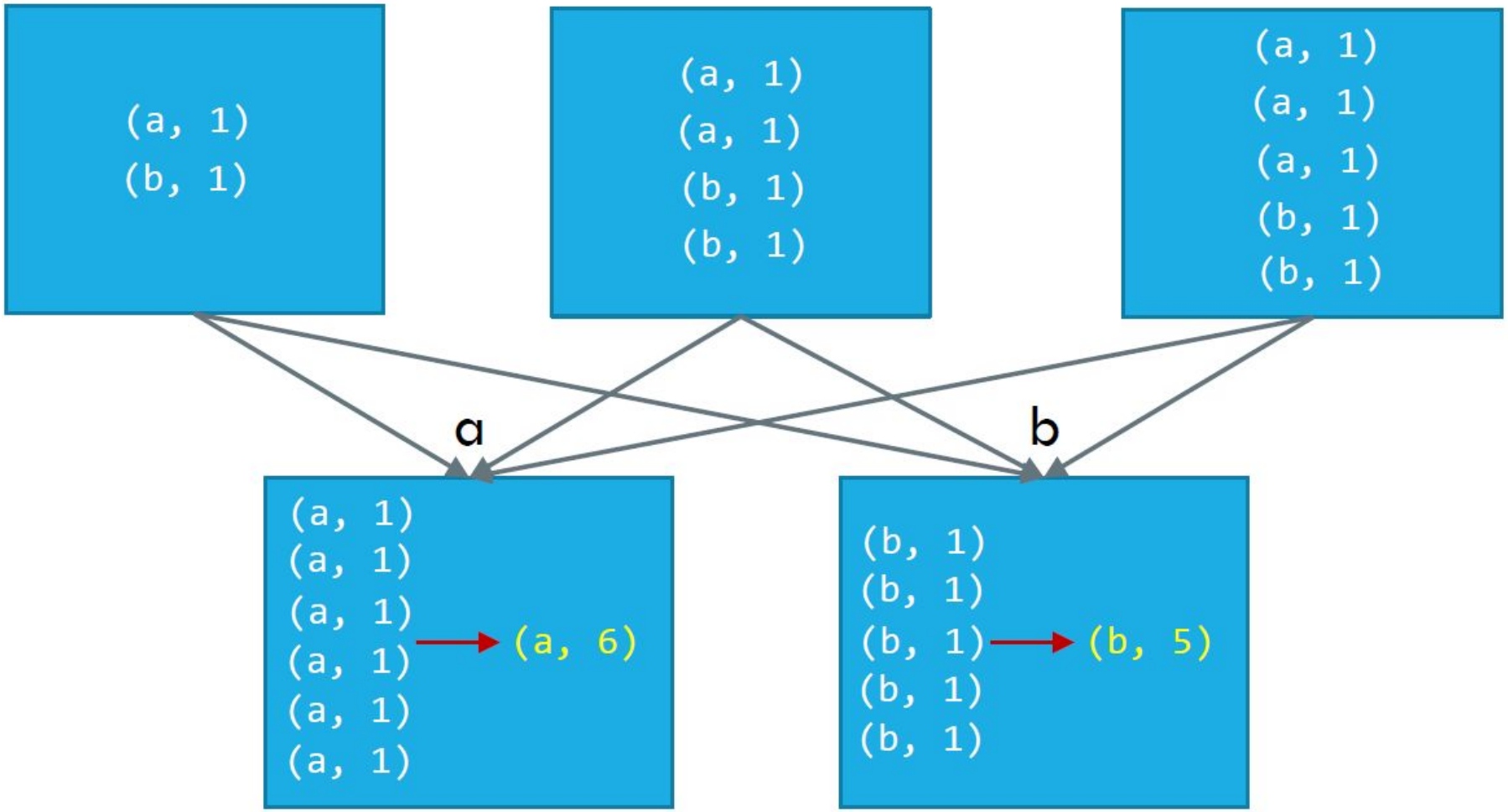
BIG DATA

reduceByKey



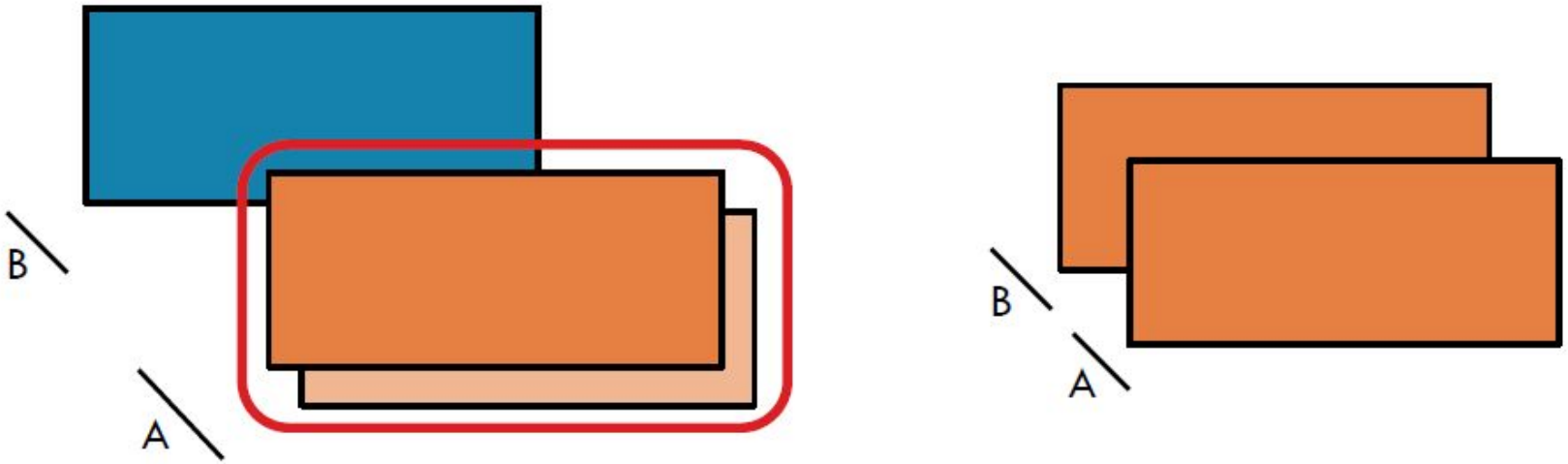
BIG DATA

groupByKey



BIG DATA

Transformations - Map Partitions



Return a new RDD by applying a function to each partition of this RDD

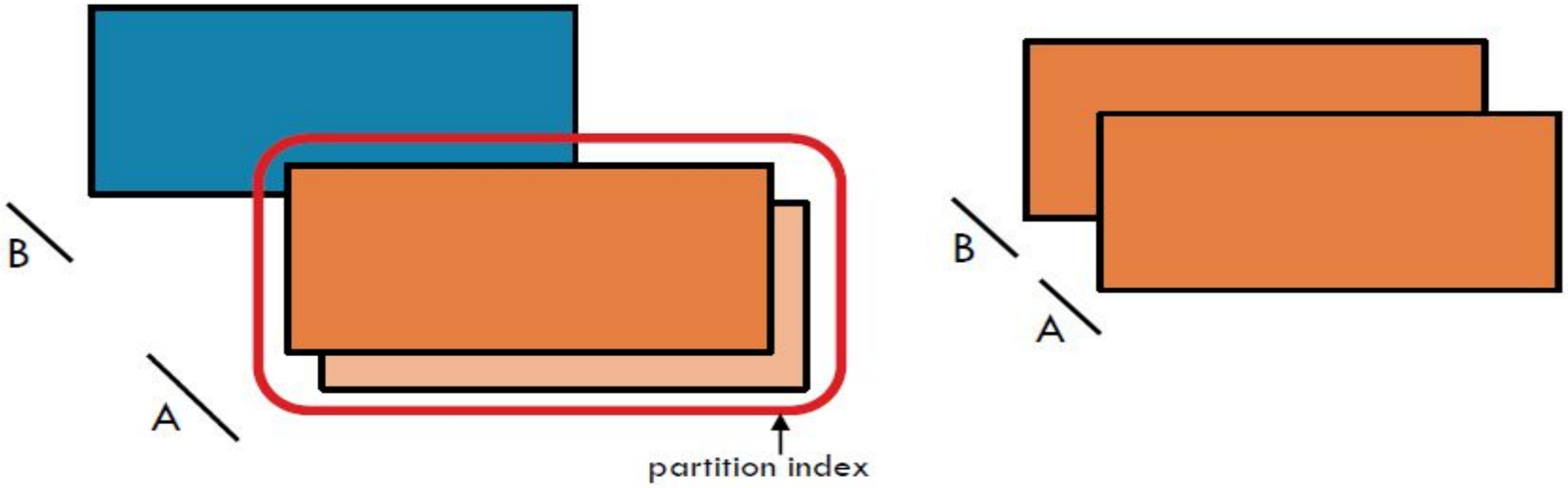
```
x = sc.parallelize([1,2,3], 2)
def f(iterator): yield sum(iterator); yield 42
y = x.mapPartitions(f)
# glom() flattens elements on the same partition
print(x.glom().collect())
print(y.glom().collect())
```

```
x: [[1], [2, 3]]

y: [[1, 42], [5, 42]]
```

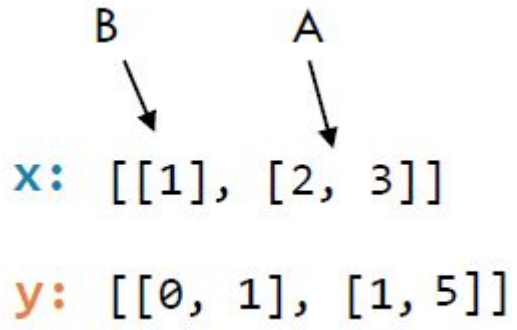
BIG DATA

Transformations - Map Partitions with Index



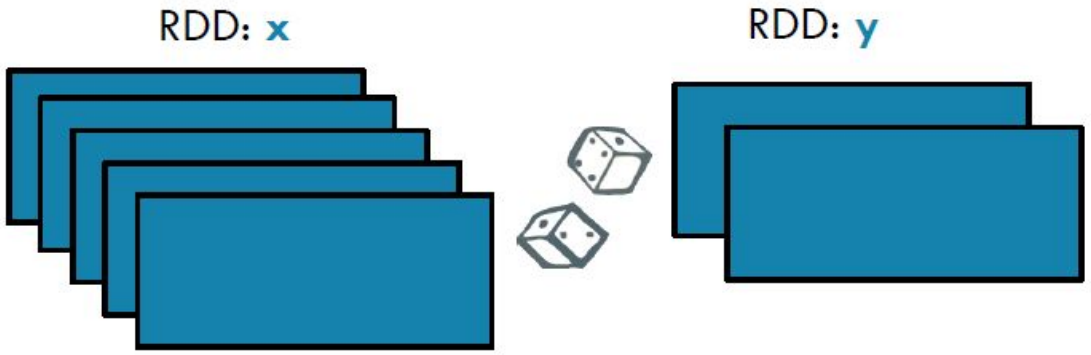
Return a new RDD by applying a function to each partition of this RDD, while tracking the index of the original partition

```
x = sc.parallelize([1,2,3], 2)
def f(partitionIndex, iterator): yield
(partitionIndex, sum(iterator))
y = x.mapPartitionsWithIndex(f)
# glom() flattens elements on the same partition
print(x.glom().collect())
print(y.glom().collect())
```



BIG DATA

Transformations - Sample



Return a new RDD containing a statistical sample of the original RDD

```
sample(withReplacement, fraction, seed=None)
```

```
x = sc.parallelize([1, 2, 3, 4, 5])
y = x.sample(False, 0.4, 42)
print(x.collect())
print(y.collect())
```

```
>>> x = sc.parallelize([1, 2, 3, 4, 5])
>>>
>>> print("Without replacement:", x.sample(False, 2.0).collect())
Without replacement: [1, 2, 3, 4, 5]
>>> print("With replacement:", x.sample(True, 2.0).collect())
With replacement: [1, 3, 4, 5, 5]
>>>
```

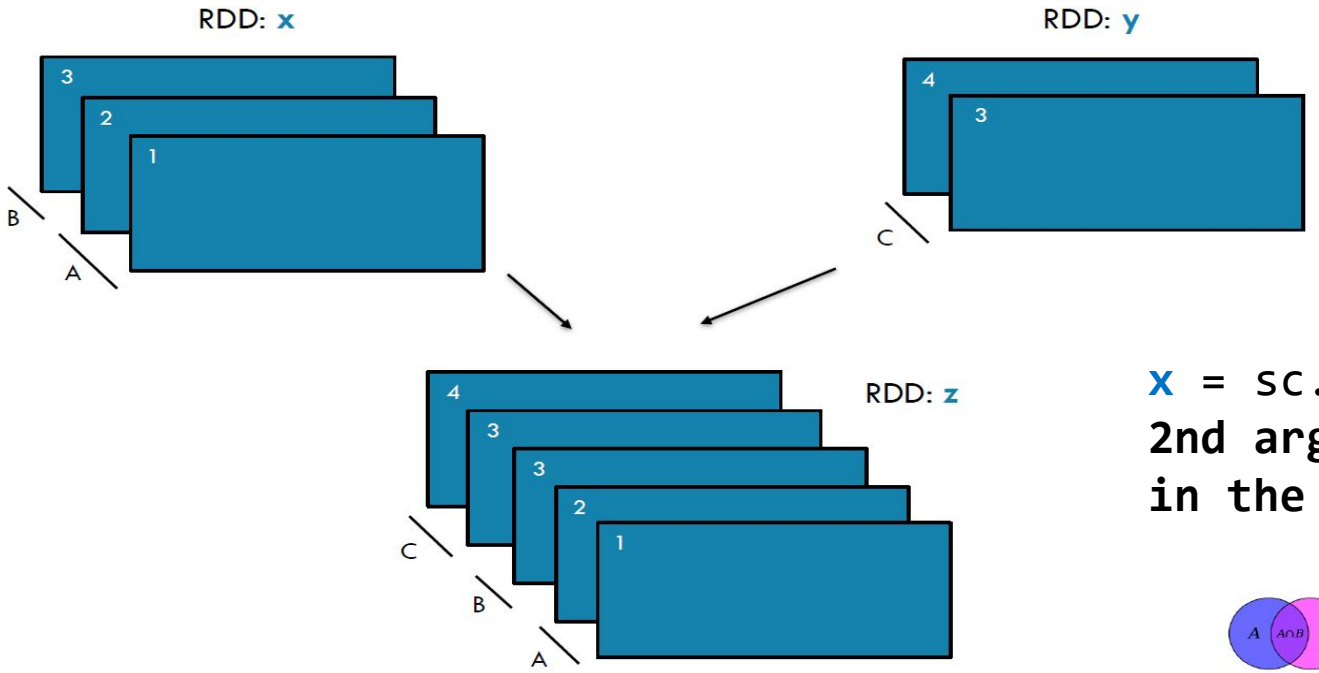


```
x: [1, 2, 3, 4, 5]

y: [1, 3]
```

BIG DATA

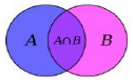
Transformations - Union



Return a new RDD containing all items from two original RDDs. **Duplicates are not culled.**

```
x = sc.parallelize([1,2,3], 2)
```

2nd argument indicates the number of partitions in the RDD.




```
x = sc.parallelize([1,2,3], 2)
y = sc.parallelize([3,4], 1)
z = x.union(y)
print(z.glom().collect())
```



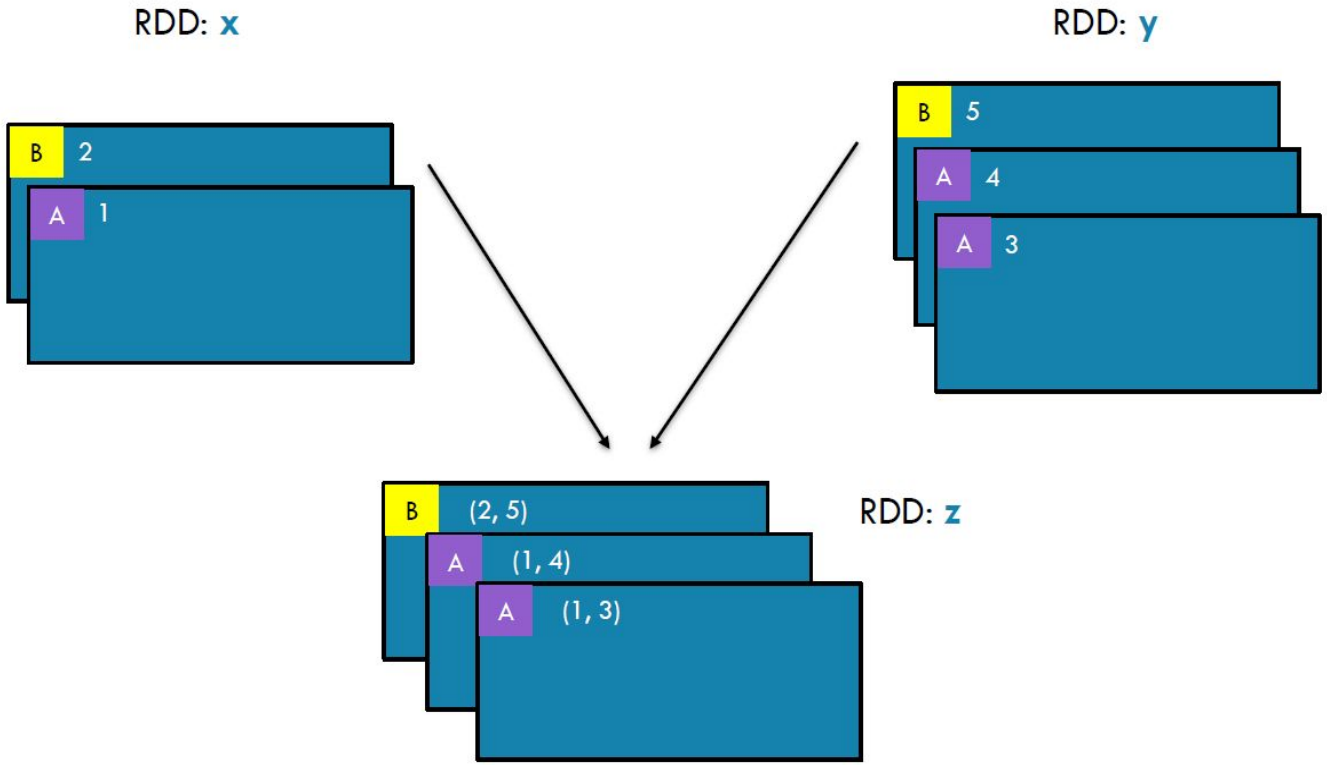
```
x: [1, 2, 3]

y: [3, 4]

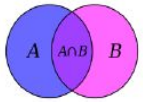
z: [[1], [2, 3], [3, 4]]
```

BIG DATA

Transformations - join



Return a new RDD containing all pairs of elements having the same key in the original RDDs




```
x = sc.parallelize([("a", 1), ("b", 2)])
y = sc.parallelize([("a", 3), ("a", 4), ("b", 5)])
z = x.join(y)
print(z.collect())
```



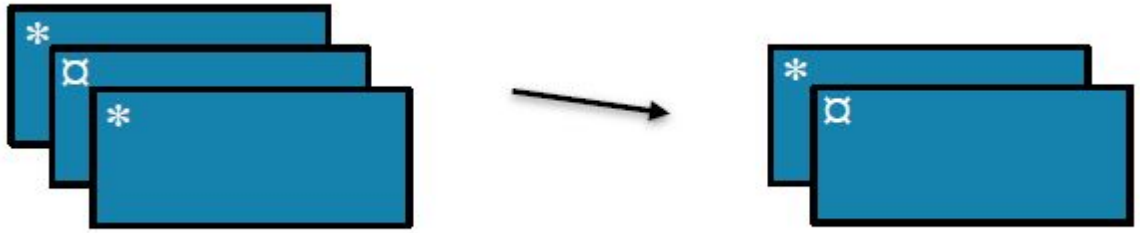
```
x: [("a", 1), ("b", 2)]

y: [("a", 3), ("a", 4), ("b", 5)]

z: [('a', (1, 3)), ('a', (1, 4)), ('b', (2, 5))]
```

BIG DATA

Transformations - distinct



Return a new RDD containing distinct items from the original RDD (omitting all duplicates)

```
x = sc.parallelize([1,2,3,3,4])
y = x.distinct()
print(y.collect())
```

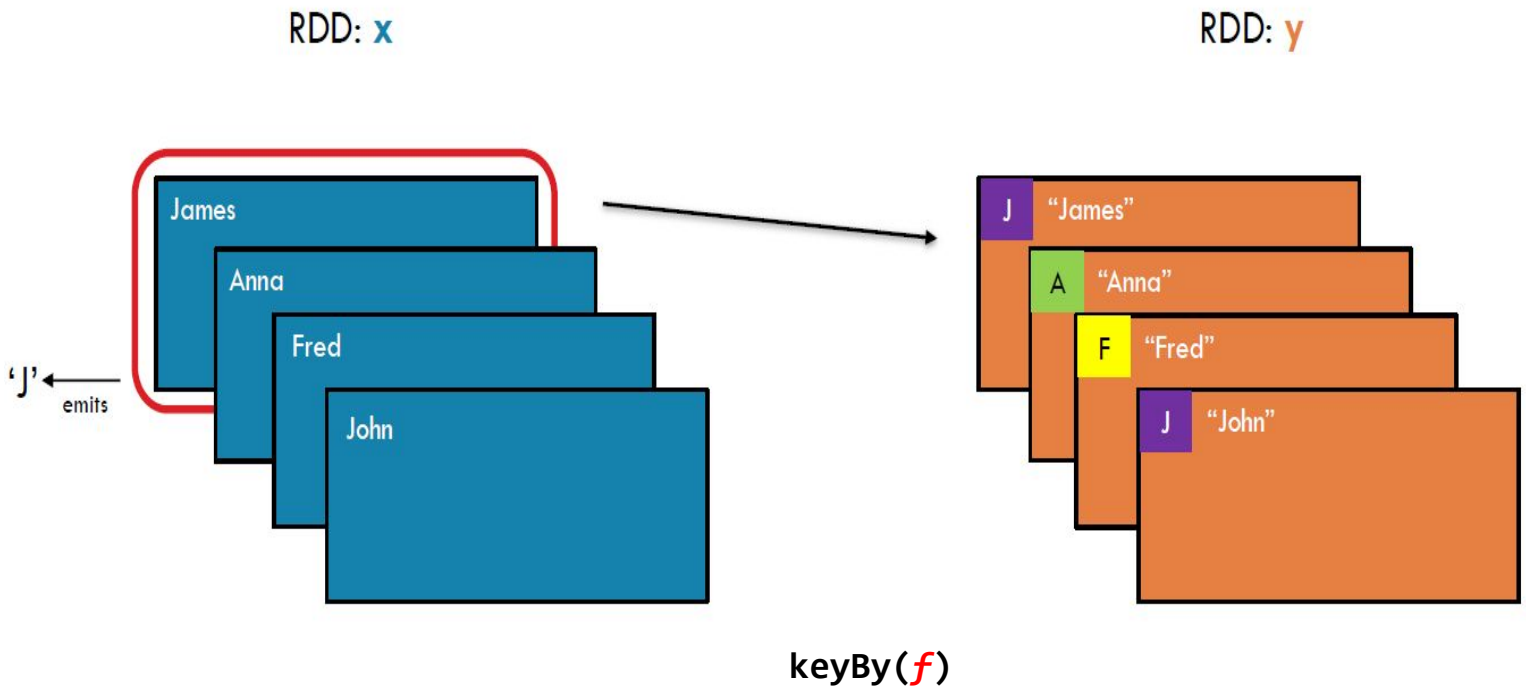


```
x: [1, 2, 3, 3, 4]

y: [1, 2, 3, 4]
```

BIG DATA

Transformations - keyBy



Create a Pair RDD, forming one pair for each item in the original RDD. The pair's key is calculated from the value via a user-supplied function.

```
x = sc.parallelize(['John', 'Fred', 'Anna', 'James'])
y = x.keyBy(lambda w: w[0])
print y.collect()
```

```
x: ['John', 'Fred', 'Anna', 'James']

y: [('J','John'),('F','Fred'),('A','Anna'),('J','James')]
```

Actions (Recap)

- Are operations that return a value

- For example:
  - Reduce() is an action

  - Aggregates all elements of a RDD to produce a result.

Actions

take(n)

The action take(n) returns n number of elements from RDD. It tries to cut the number of partition it accesses, so it represents a biased collection. We cannot presume the order of the elements.

For example, consider RDD {1, 2, 2, 3, 4, 5, 5, 6} in this RDD “take (4)” will give result { 2, 2, 3, 4}

count()

Action count() returns the number of elements in RDD.

For example, RDD has values {1, 2, 2, 3, 4, 5, 5, 6} in this RDD “rdd.count()” will give the result 8.

PES UNIVERSITY
CELEBRATING 50 YEARS

countByValue()

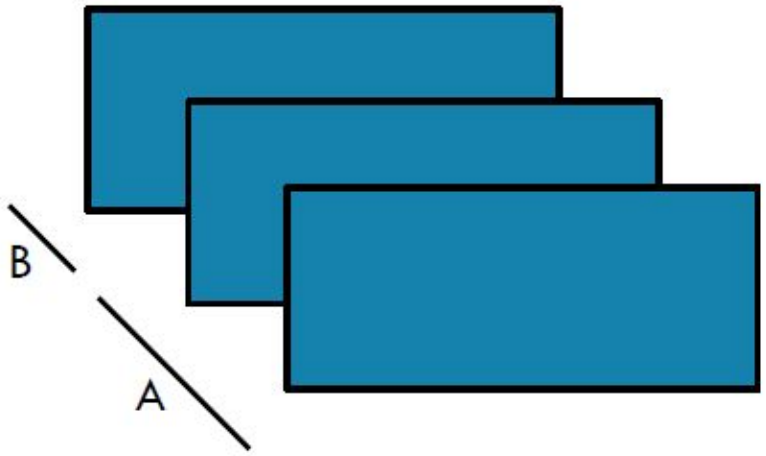
The countByValue() returns, many times each element occur in RDD.
For example, RDD has values {1, 2, 2, 3, 4, 5, 5, 6} in this RDD "rdd.countByValue()" will give the result {(1,1), (2,2), (3,1), (4,1), (5,2), (6,1)}

top()

If ordering is present in our RDD, then we can extract top elements from our RDD using top(). Action top() use default ordering of data.

BIG DATA

Actions - GetNum Partitions



getNumPartitions()

Return the number of partitions in RDD




```
x = sc.parallelize([1,2,3], 2)
y = x.getNumPartitions()
print(x.glom().collect())
print(y)
```

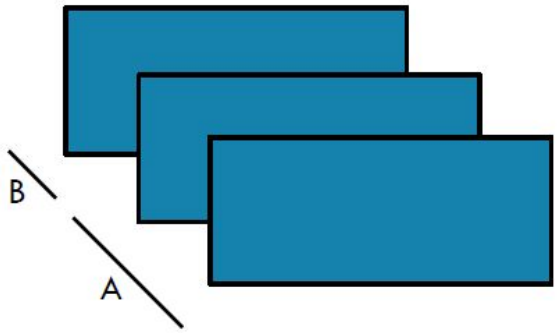
```
x: [[1], [2, 3]]

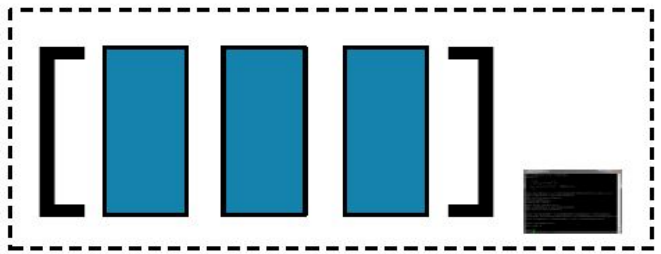
y: 2
```

Actions - collect

collect()

The action collect() is the common and simplest operation that returns our entire RDDs content to driver program. The application of collect() is unit testing where the entire RDD is expected to fit in memory. As a result, it makes easy to compare the result of RDD with the expected result. Action Collect() had a constraint that all the data should fit in the machine, and copies to the driver.





Return all items in the RDD to the driver in a single list

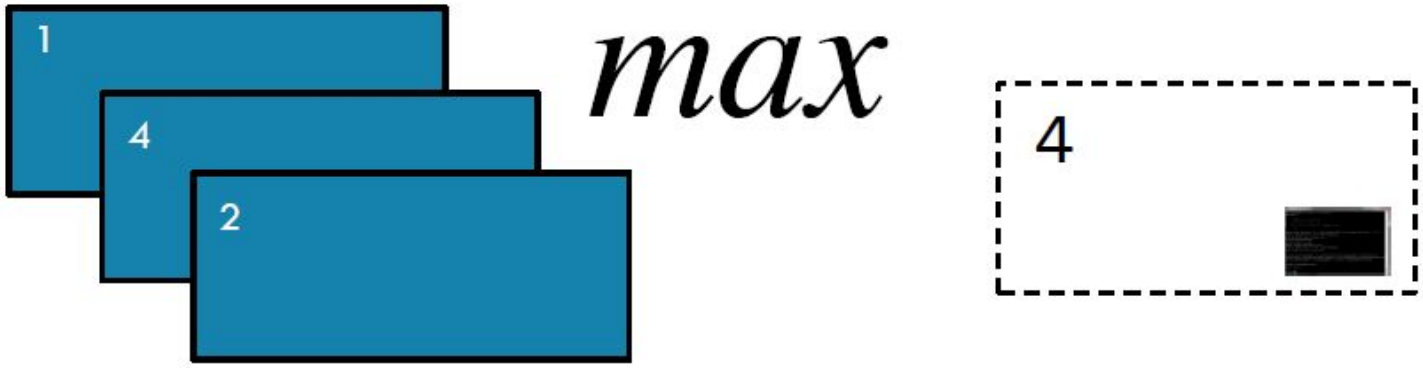
```
x = sc.parallelize([1,2,3], 2)
y = x.collect()
print(x.glom().collect())
print(y)
```



```
x: [[1], [2, 3]]
y: [1, 2, 3]
```

BIG DATA

Actions - max



`max()`

Return the maximum item in the RDD

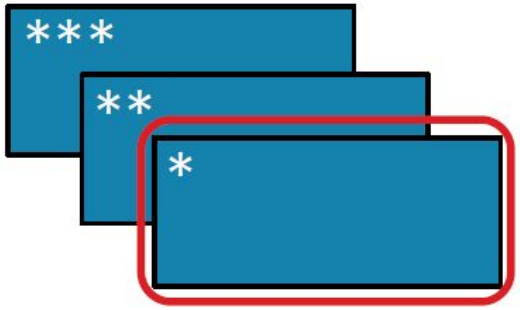
```
x = sc.parallelize([2,4,1])
y = x.max()
print(x.collect())
print(y)
```

```
x: [2, 4, 1]

y: 4
```

BIG DATA

Actions - Aggregate

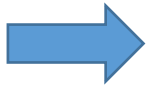




aggregate(identity, *seqOp*, *combOp*)

Aggregate all the elements of the RDD by:
- applying a user function to combine elements with user-supplied objects,
- then combining those user-defined results via a second user function,
- and finally returning a result to the driver.

```
seqOp = lambda data, item: (data[0] + [item], data[1] + item)
combOp = lambda d1, d2: (d1[0] + d2[0], d1[1] + d2[1])
x = sc.parallelize([1,2,3,4])
y = x.aggregate(([], 0), seqOp, combOp)
print(y)
```

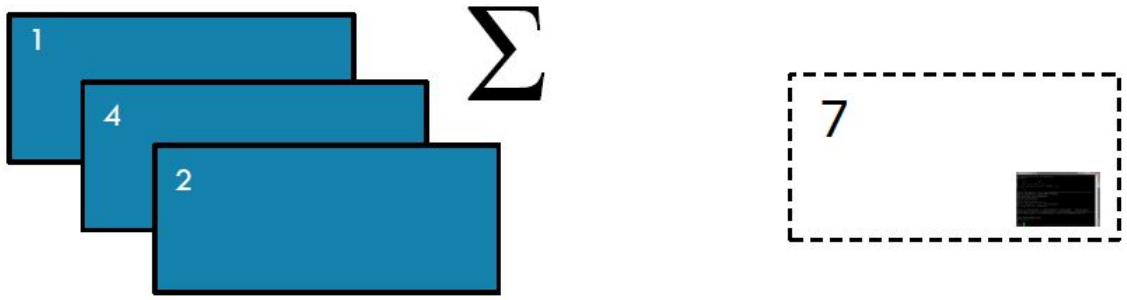


```
x: [1, 2, 3, 4]

y: ([1, 2, 3, 4], 10)
```

BIG DATA

Actions - sum



sum()

Return the sum of the items in the RDD

```
x = sc.parallelize([2,4,1])
y = x.sum()
print(x.collect())
print(y)
```

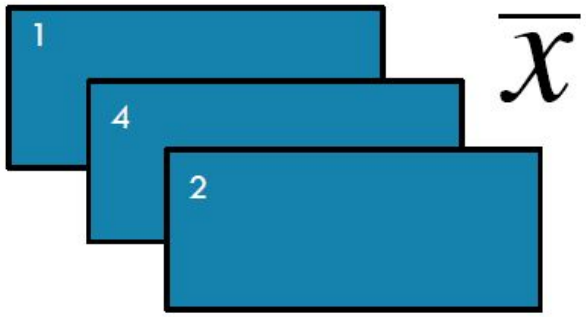


```
x: [2, 4, 1]

y: 7
```

BIG DATA

Actions - mean





mean()

Return the mean of the items in the RDD

```
x = sc.parallelize([2,4,1])
y = x.mean()
print(x.collect())
print(y)
```

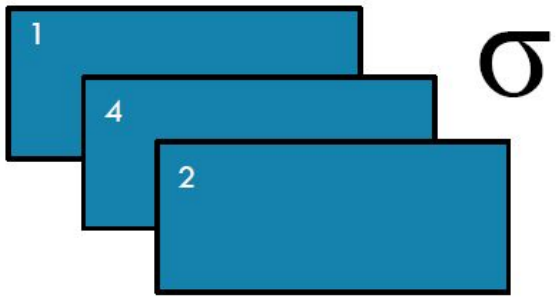


```
x: [2, 4, 1]

y: 2.3333333
```

BIG DATA

Actions - stddev





stdev()

Return the standard deviation of the items in the RDD

```
x = sc.parallelize([2,4,1])
y = x.stdev()
print(x.collect())
print(y)
```



```
x:  [2, 4, 1]

y:  1.2472191
```

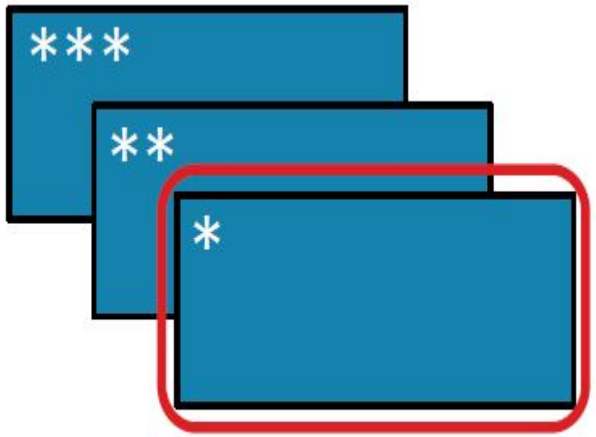
Actions - reduce

PES UNIVERSITY
CELEBRATING 50 YEARS

reduce()

The reduce() function takes the two elements as input from the RDD and then produces the output of the same type as that of the input elements. The simple forms of such function are an addition. We can add the elements of RDD, count the number of words. It accepts commutative and associative operations as an argument.

Actions - reduce





reduce(*f*)

Aggregate all the elements of the RDD by applying a user function pairwise to elements and partial results, and returns a result to the driver

```
x = sc.parallelize([1,2,3,4])
y = x.reduce(lambda a,b: a+b)
print(x.collect())
print(y)
```



```
x:  [1, 2, 3, 4]

y:  10
```

Actions - fold

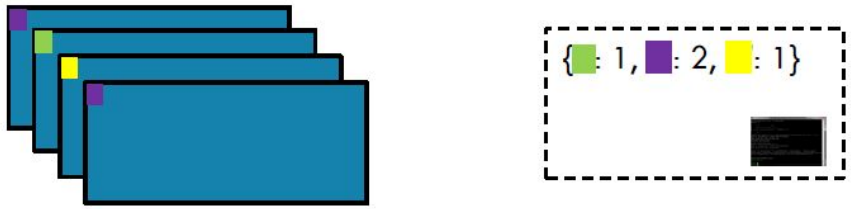
fold()

The signature of the fold() is like reduce(). Besides, it takes “zero value” as input, which is used for the initial call on each partition. But, the condition with zero value is that it should be the identity element of that operation. The key difference between fold() and reduce() is that, reduce() throws an exception for empty collection, but fold() is defined for empty collection.

For example, zero is an identity for addition; one is identity element for multiplication. The return type of fold() is same as that of the element of RDD we are operating on.
For example, rdd.fold(0)((x, y) => x + y).

BIG DATA

Actions - countByKey



countByKey()

Return a map of keys and counts of their occurrences in the RDD

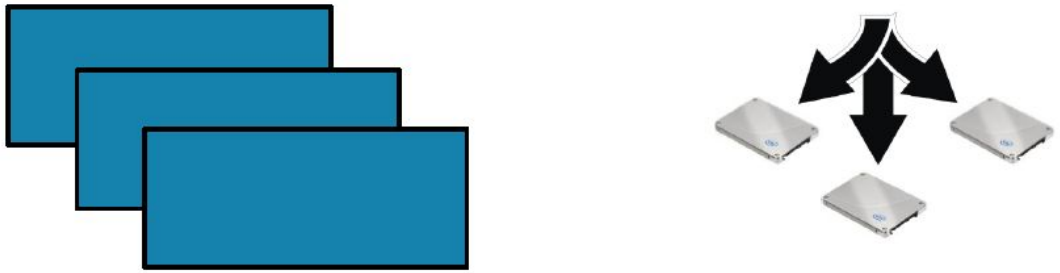
```
x = sc.parallelize([('J', 'James'), ('F','Fred'),
('A','Anna'), ('J','John')])
y = x.countByKey()
print(y)
```

```
x: [('J', 'James'), ('F','Fred'),
    ('A','Anna'), ('J','John')]

y: {'A': 1, 'J': 2, 'F': 1}
```

BIG DATA

Actions -saveAsTextFile



saveAsTextFile(*path, compressionCodecClass=None*)

Save the RDD to the filesystem indicated in the path

```
dbutils.fs.rm("/temp/demo", True)
x = sc.parallelize([2,4,1])
x.saveAsTextFile("/temp/demo")
y = sc.textFile("/temp/demo")
print(y.collect())
```



```
x: [2, 4, 1]

y: [u'2', u'4', u'1']
```



THANK YOU

Prafullata Kiran Auradkar
Department of CSE
prafullatak@pes.edu



BIG DATA

Spark SQL

Prafullata Kiran Auradkar

Department of Computer Science and Engineering

prafullatak@pes.edu

Acknowledgements:

Significant information in the slide deck presented through the Unit 1 of the course have been created by **Dr. K V Subramaniam's** and would like to acknowledge and thank him for the same. There have been some information which I might have leveraged from **Dr. H L Phalachandra's** slide contents too. I may have supplemented the same with contents from books and other sources from Internet and would like to sincerely thank, acknowledge and reiterate that the credit/rights for the same remain with the original authors/publishers only. These are intended for classroom presentation only.

Spark sql: Relational data processing in **spark**

M Armbrust, RS Xin, C Lian, Y Huai, D Liu… - Proceedings of the …, 2015 - dl.acm.org

… **Spark**, use **Spark SQL** within other programming languages. Performance-wise, we find that **Spark SQL** is competitive with **SQL**-… naive **Spark** code in computations expressible in **SQL**. …

☆ Save 99 Cite Cited by 2171 Related articles All 38 versions

[PDF] acm.org

✦ Quick read - berkeley.edu

As on 07/09/2026

Spark SQL

Armbrust, Michael, Reynold S. Xin, Cheng Lian, Yin Huai, Davies Liu, Joseph K. Bradley, Xiangrui Meng et al. "Spark sql: Relational data processing in spark." In *Proceedings of the 2015 ACM SIGMOD international conference on management of data*, pp. 1383-1394. 2015.

Structured Data Processing

A common big-data use case is processing structured or semi-structured data

- In Spark RDD, all functions and objects are black-boxes. Spark doesn't know the data's internal structure

- Any structure of the data has to be part of the functions which includes:

  ➔ Parsing: read raw records into fields

  ➔ Conversion: cast into types needed

  ➔ Processing: apply logic on structured result

BIG DATA

Structured data processing

- Pig/Pig Latin
  - Builds on Hadoop
  - Converts SQL-like programs to MapReduce

- Hive/HiveQL
  - Supports SQL-like queries

- Shark (Hive on Spark)
  - Translates HiveQL queries to RDD programs
  - Initial attempt to support SQL on Spark

Shark: SQL and rich analytics at scale [PDF] acm.org

RS Xin, J Rosen, M Zaharia, MJ Franklin... - Proceedings of the ..., 2013 - dl.acm.org

... In the remainder of this section, we cover the basics of **Spark** and the RDD programming model, and then we describe how **Shark** query plans are generated and executed. ...

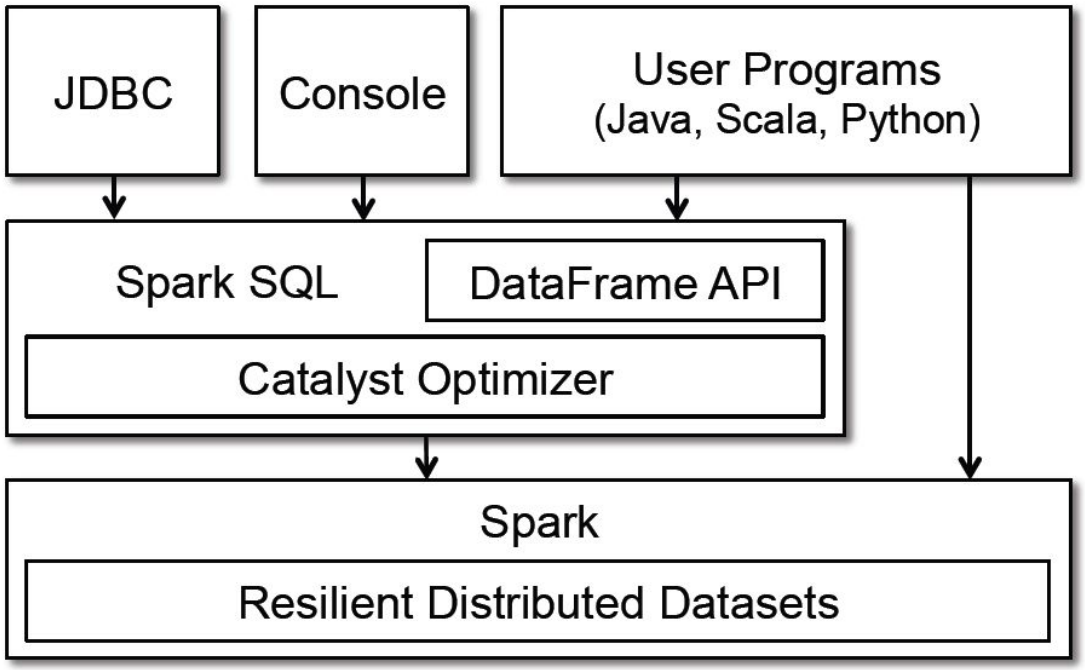
☆ Save 99 Cite Cited by 667 Related articles All 35 versions

Redesigned to consider Spark query model

- Supports all the popular relational operators
- Can be intermixed with RDD operations
- Uses the Dataframe API as an enhancement to the RDD API

Dataframe = RDD + schema

Programming interface



https://amplab.cs.berkeley.edu/wp-content/uploads/2015/03/SparkSQLSigmod2015.pdf

Data Model

- Nested data model
- Supports
  - Primitive SQL types: boolean, integer, double, decimal, string, date, timestamp
  - Complex types: structs, arrays, maps, and unions
  - User defined types.
- First class support for complex data types

Built-in operations in SparkSQL

PES UNIVERSITY
CELEBRATING 50 YEARS

- Filter (Selection)
- Select (Projection)
- Join
- Group By (Aggregation)
- Load/Store in various formats
- Cache
- Conversion between RDD (back and forth)

BIG DATA

DataFrame Operations

- DataFrames use the same operations as SQL (select, where, join, groupBy), written as method calls in code, not SQL text
- Each call like employees("gender") === "female" doesn't evaluate immediately, instead it builds a column expression
- Operators build up an abstract syntax tree (AST), which is then optimized by Catalyst.

```
employees
  .join(dept, employees("deptId") === dept("id"))
  .where(employees("gender") === "female")
  .groupBy(dept("id"), dept("name"))
  .agg(count("name"))
```

- Alternatively, register the DataFrame as a temp SQL table, then query it with real SQL strings

```
users.where(users("age") < 21)
     .registerTempTable("young")
ctx.sql("SELECT count(*), avg(age) FROM young")
```

Advantages over Relational Query Languages

- **Holistic optimization**: functions written across different languages (Python, Scala, SQL) can still be optimized together as one plan, not as separate opaque steps
- **Real control structures**: since DataFrame code is regular code, you can use if/for around your queries which is not possible inside a plain SQL string
- **Early error detection**: the logical plan is validated immediately, so schema mismatches surface at write-time, not after a job runs and fails

Querying Native Datasets

PES UNIVERSITY
CELEBRATING 50 YEARS

- **Spark infers column names/types straight from your own data classes**
  - Java/Scala: read via reflection (inspects the class definition)
  - Python: inferred by sampling actual rows, since Python has no static types
  - case class User(name: String, age: Int) → schema {name: String, age: Int} for free

- Your native objects are used directly — no conversion into a separate row format first

- Benefits:
  - Add relational operations (.where, .groupBy) onto RDDs of your own objects
  - Join native RDDs against external structured data (CSV, Parquet, JDBC)

enables columnar storage, with frequently-used ("hot") columns cached in memory

BIG DATA

User-Defined Functions (UDFs)

- SparkSQL ships a limited built-in function set. UDFs let you plug in custom logic
- Defined and registered inline, directly in your code
- Contrast with Pig: Pig UDFs must be written as a Java package and loaded separately into the script
- Can operate on a single value or aggregate across an entire table
- Once registered, callable from both DataFrame code and SQL strings.

```
val model: LogisticRegressionModel = ...

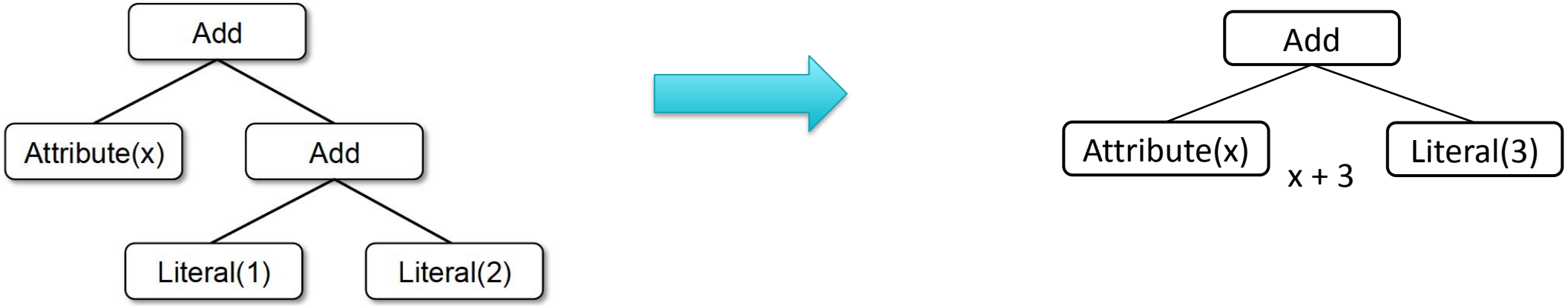
ctx.udf.register("predict",
  (x: Float, y: Float) => model.predict(Vector(x, y)))

ctx.sql("SELECT predict(age, weight) FROM users")
```

Catalyst

Catalyst is Spark SQL's query optimizer. It represents queries as trees, and applies transformations to rewrite them into faster equivalent plans

```
tree.transform {
  case Add(Literal(c1), Literal(c2)) => Literal(c1+c2)
}
```



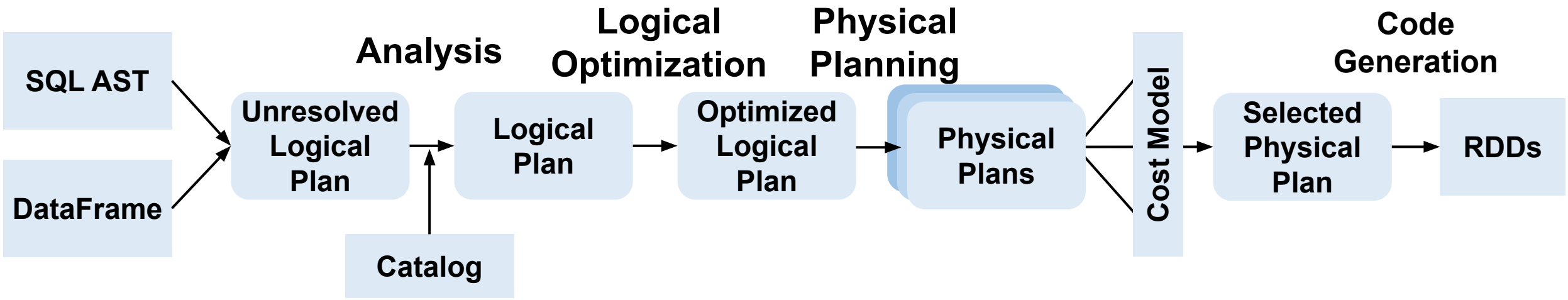
x + (1 + 2)

x + 3

Catalyst Rules

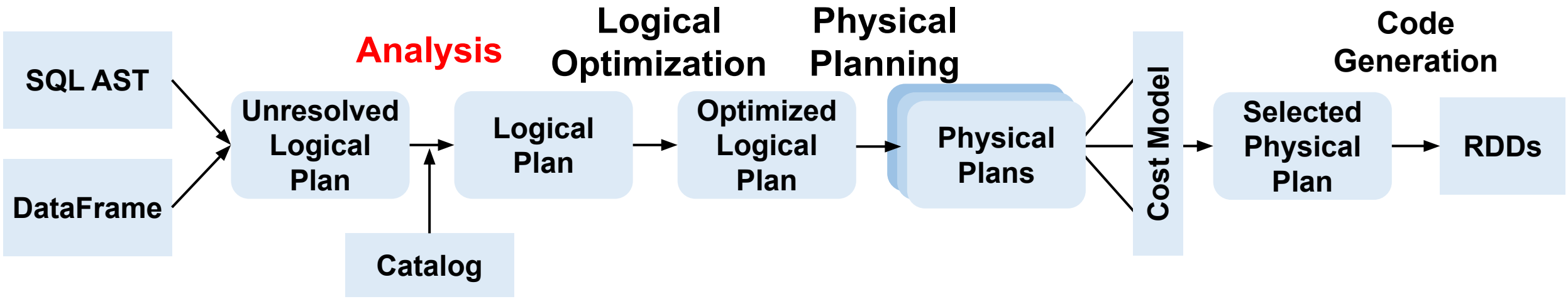
- Pattern-matching functions that rewrite subtrees into specific structures.
- A rule is a partial function: it only fires on patterns it matches, and leaves everything else untouched. Hence, adding a new operator type never requires editing existing rules
- Multiple patterns in the same transform call.
- May take multiple batches to reach a fixed point.
- transform can contain arbitrary Scala code.

Plan Optimization & Execution

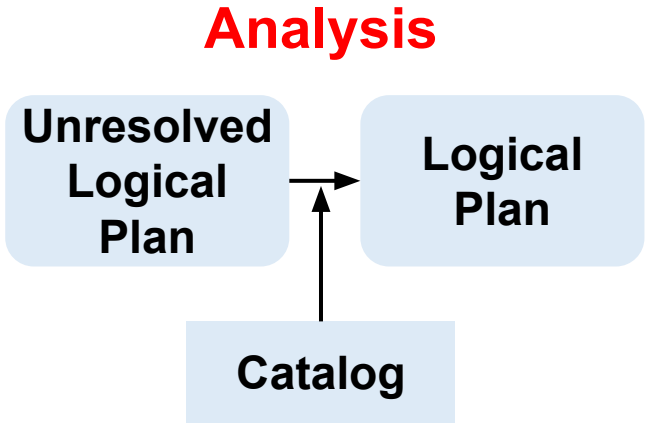


Data Frames and SQL share the same optimization/execution pipeline

Plan Optimization & Execution



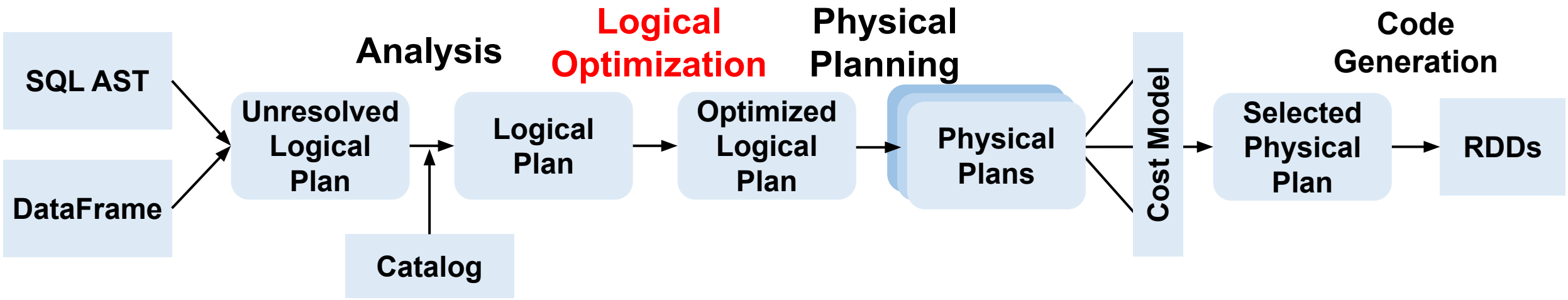
Plan Optimization & Execution - Analysis



SELECT *col* FROM *sales*

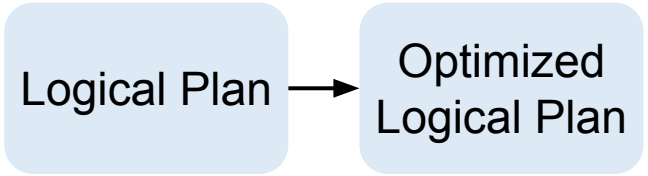
- An attribute is unresolved if its type is not known or it's not matched to an input table.
- To resolve attributes:
  - Look up relations by name from the catalog.
  - Map named attributes to the input provided given operator's children.
  - UID for references to the same value
  - Propagate and coerce types through expressions (e.g. 1 + col)

Plan Optimization & Execution



Plan Optimization & Execution – Logical Optimization

Logical Optimization

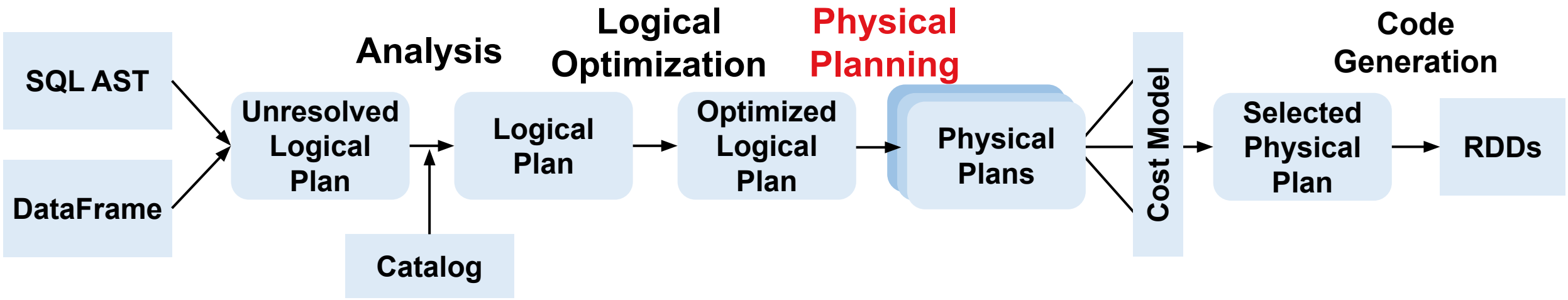


- Applies standard rule-based optimization (constant folding, predicate-pushdown, projection pruning, null propagation, boolean expression simplification, etc)
- 800LOC implement this stage

```
object DecimalAggregates extends Rule[LogicalPlan] {
  /** Maximum number of decimal digits in a Long */
  val MAX_LONG_DIGITS = 18

  def apply(plan: LogicalPlan): LogicalPlan = {
    plan transformAllExpressions {
      case Sum(e @ DecimalType.Expression(prec, scale))
              if prec + 10 <= MAX_LONG_DIGITS =>
        MakeDecimal(Sum(LongValue(e)), prec + 10, scale)
    }
}
```

Plan Optimization & Execution



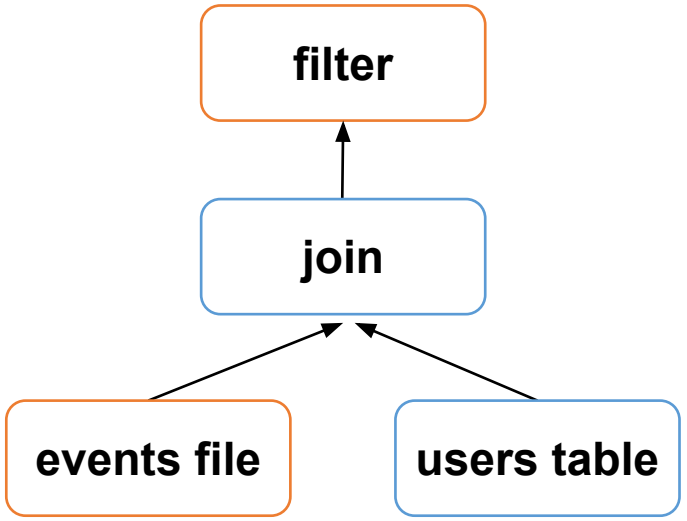
- e.g. Pipeline projections and filters into a single map

Plan Optimization & Execution – Logical Optimization

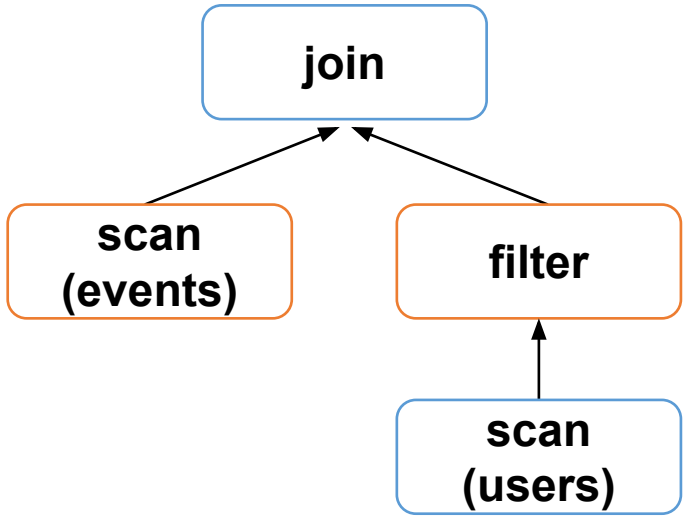
```
def add_demographics(events):
    u = sqlCtx.table("users")                    # Load partitioned Hive table
    events \
      .join(u, events.user_id == u.user_id) \    # Join on user_id
      .withColumn("city", zipToCity(df.zip))     # Run udf to add city column

events = add_demographics(sqlCtx.load("/data/events", "parquet"))
training_data = events.where(events.city == "Melbourne").select(events.timestamp).collect()
```

Logical Plan

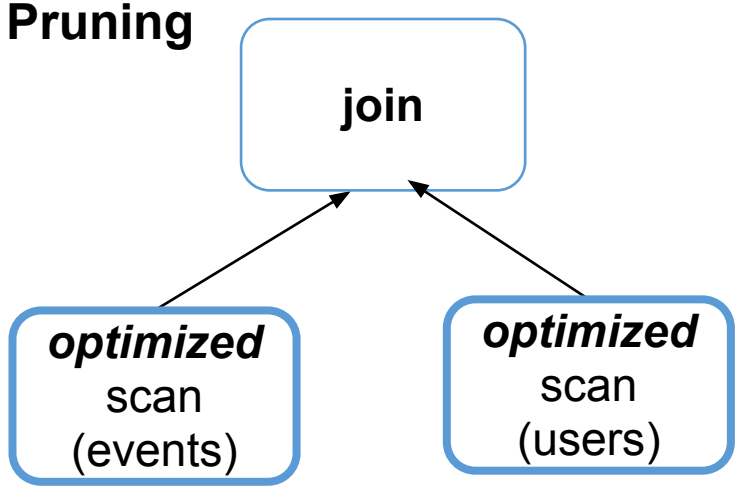


Physical Plan



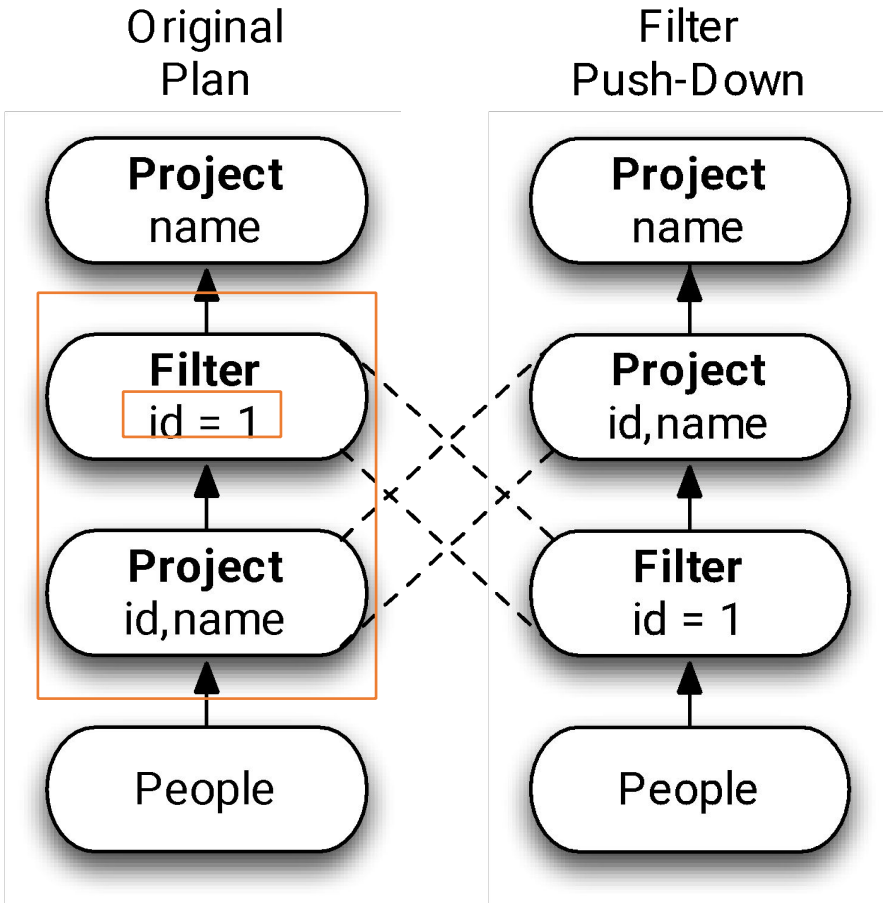
Physical Plan

with Predicate Pushdown and Column Pruning

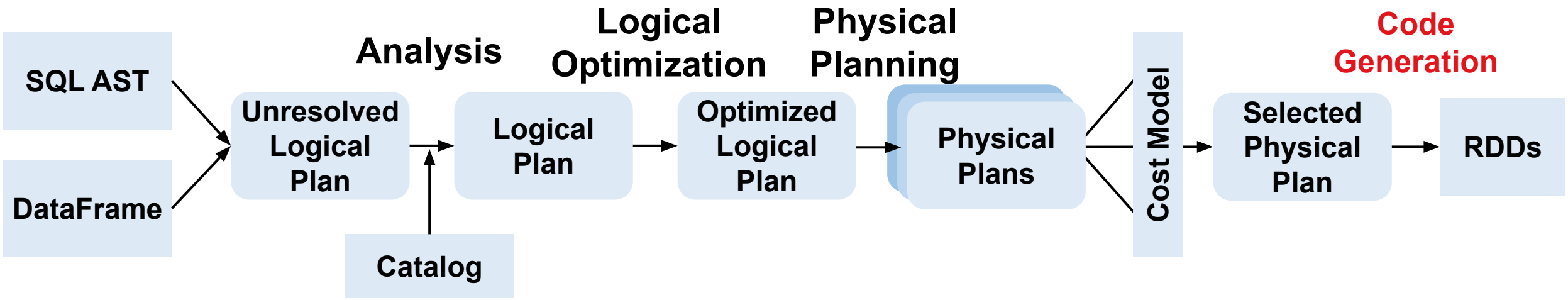


An Example Catalyst Transformation

- Consider an example of reading People and project columns such as id and name from people table:
- Find filters on top of projections.
- Check that the filter can be evaluated without the result of the project.
- If so, switch the operators.



Plan Optimization & Execution



Plan Optimization & Execution – Code Generation

```
def compile(node: Node): AST = node match {
  case Literal(value) => q"$value"
  case Attribute(name) => q"row.get($name)"
  case Add(left, right) =>
    q"${compile(left)} + ${compile(right)}"
}
```

- Relies on Scala's quasiquotes to simplify code generation: q"..."
- Catalyst transforms a SQL tree into an abstract syntax tree (AST) for Scala code to evaluate expression and generate code
- 700LOC to implement this stage

Extensions

Data Sources

- must implement a createRelation function that takes a set of key-value params and returns a BaseRelation object.
- E.g. CSV, Avro, Parquet, JDBC

User-Defined Types (UDTs)

- Map user-defined types to structures composed of Catalyst's built-in types.

```
class PointUDT extends UserDefinedType[Point] {
  def dataType = StructType(Seq( // Our native structure
    StructField("x", DoubleType),
    StructField("y", DoubleType)
  ))
  def serialize(p: Point) = Row(p.x, p.y)
  def deserialize(r: Row) =
    Point(r.getDouble(0), r.getDouble(1))
}
```

Advanced Analytics Features

PES UNIVERSITY
CELEBRATING 50 YEARS

Schema Inference for Semi structured Data JSON

- Automatically infers schema from a set of records, in one pass or sample
- Represents as a tree of STRUCT types, each of which may contain atoms, arrays, or other STRUCTs.
- Find the most appropriate type for a field based on all data observed in that column. Determine array element types in the same way.
- Merge schema of single records in one reduce operation.
- Same trick for Python typing

```
{
  "text": "This is a tweet about #Spark",
  "tags": ["#Spark"],
  "loc": {"lat": 45.1, "long": 90}
}

{
  "text": "This is another tweet",
  "tags": [],
  "loc": {"lat": 39, "long": 88.5}
}

{
  "text": "A #tweet without #location",
  "tags": ["#tweet", "#location"]
}
```

```
text STRING NOT NULL,
tags ARRAY<STRING NOT NULL> NOT NULL,
loc STRUCT<lat FLOAT NOT NULL, long FLOAT NOT NULL>
```

Demo Spark SQL

Demo



THANK YOU

Prafullata Kiran Auradkar
Department of CSE
prafullatak@pes.edu



BIG DATA

Kafka - Architecture and Usage

Prafullata Kiran Auradkar

Department of Computer Science and Engineering

prafullatak@pes.edu

Acknowledgements:

Significant information in the slide deck presented through the Unit 1 of the course have been created by **Dr. K V Subramaniam's** and would like to acknowledge and thank him for the same. There have been some information which I might have leveraged from **Dr. H L Phalachandra's** slide contents too. I may have supplemented the same with contents from books and other sources from Internet and would like to sincerely thank, acknowledge and reiterate that the credit/rights for the same remain with the original authors/publishers only. These are intended for classroom presentation only.

BIG DATA

Kafka Overview

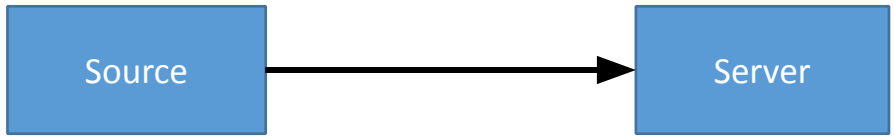
- Need for data pipelines
- Kafka architecture
- Kafka components – messaging model
- Communication and Routing
- Messages
- Scaling
- Fault Tolerance

https://www.confluent.io/resources/ebook/kafka-the-definitive-guide/

Kafka: Need for data pipelines

The need for processing events

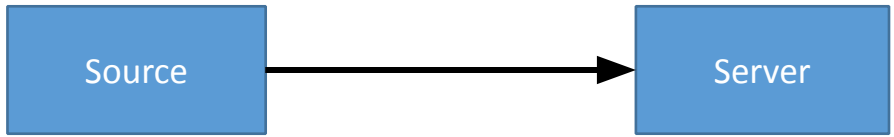
- Stream processing requires processing of events
- A university gate scans a student ID: “Student 23489 entered building"
- That’s an event, which happens at a specific time, one at a time
- Events are processed on the server, once it arrives



Data pipelines starts like this.

What are Data Pipelines

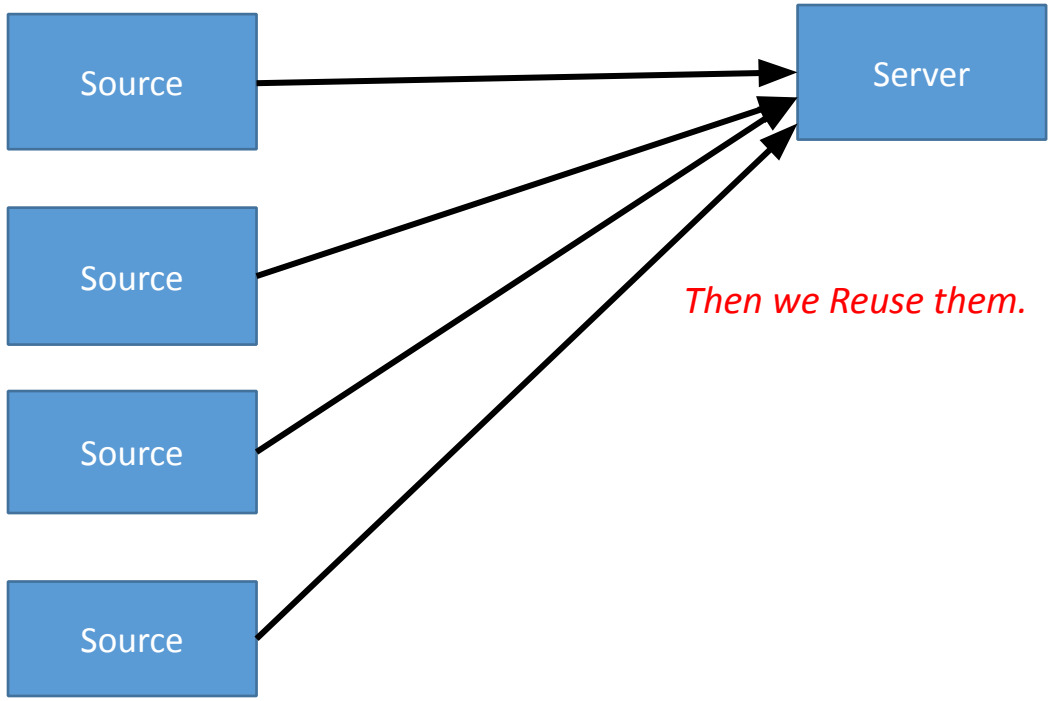
- Data sources are varied and streamed at varied rates
- They are very lightweight and not capable of compute
- Analysis is done on servers
- Data has to be transferred from sources to the compute servers
- Point to point connection is a simple way to connect data to servers



Data pipelines starts like this.

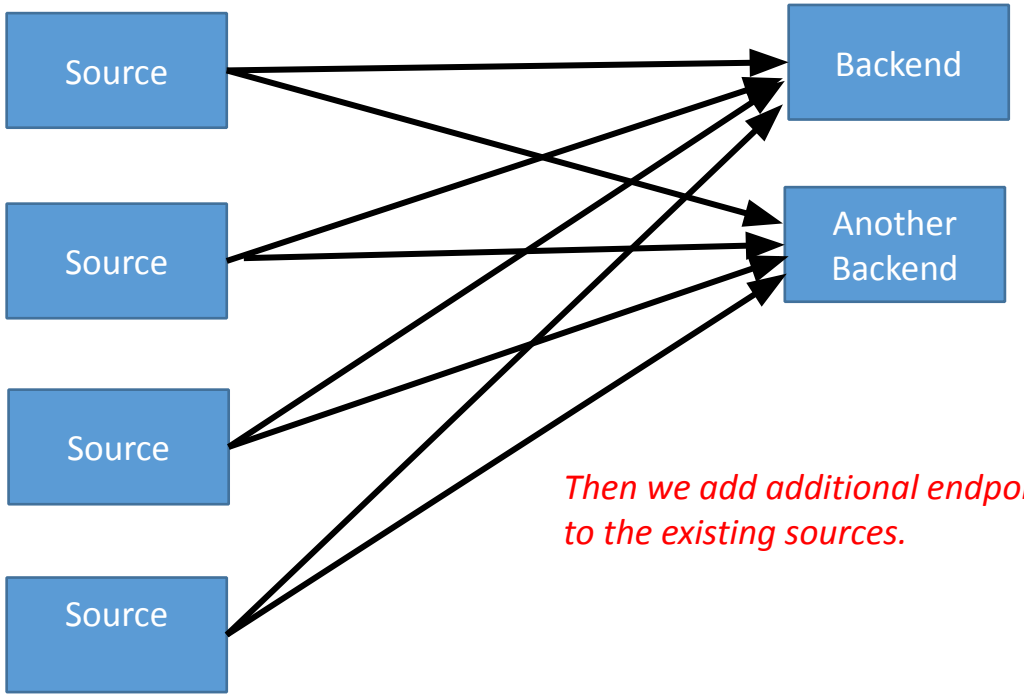
Handling multiple sources

- But we often have to read from multiple data sources
  - E.g: Multiple cameras connected to a central processing system
- We can connect multiple clients each over a pool of connections



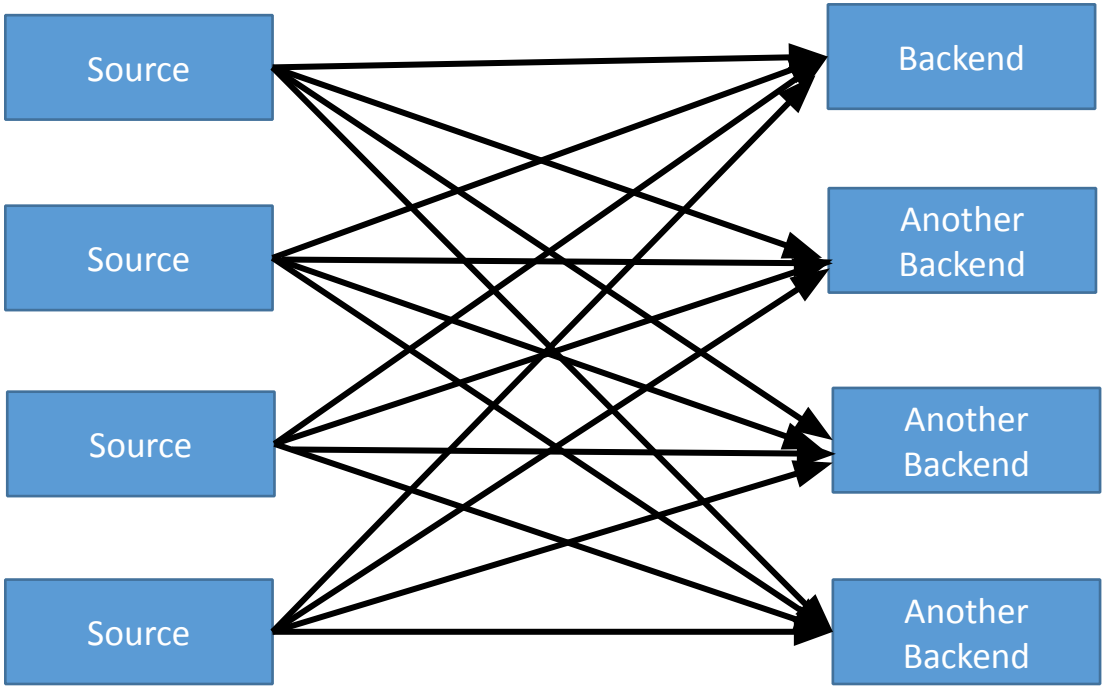
Need for multiple backends

- But we might have multiple servers
- Each process the same data in different ways
- Use case: Stock trading – may have different analysts wanting to analyze the behavior in different ways



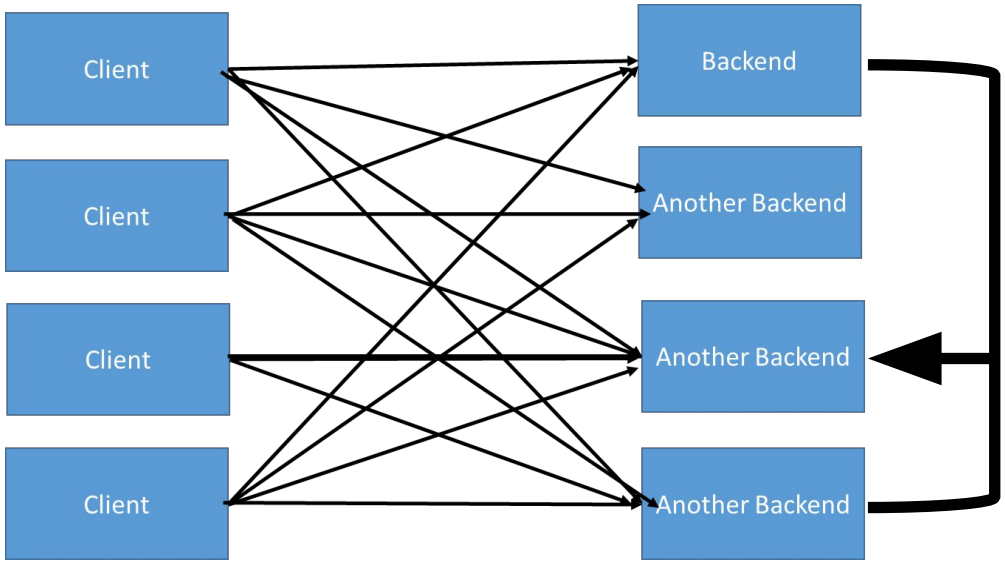
The maze of connections

- Then it starts to look like this.



BIG DATA

The maze of connections

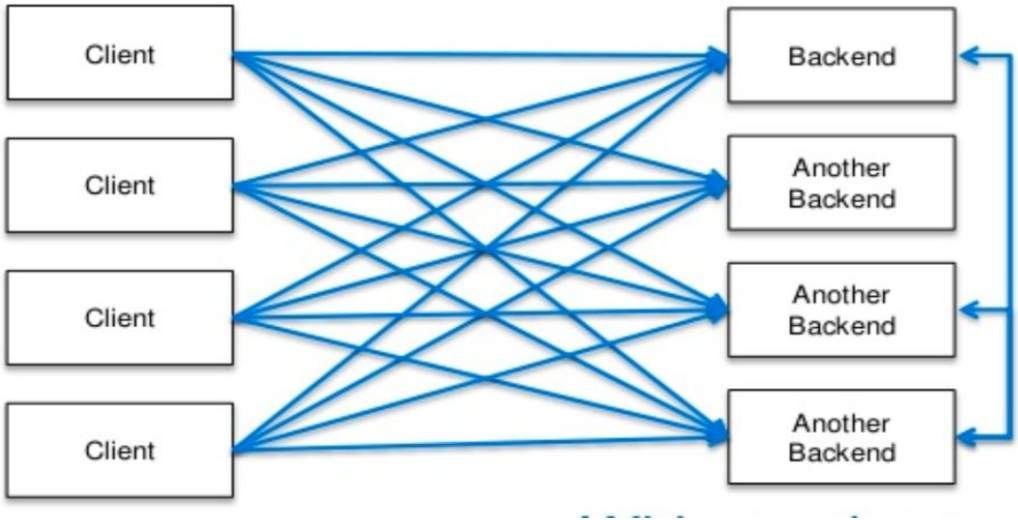


With may be some of these.

- N clients × M backends = N×M connections to build and maintain
- As Distributed systems grow, one failing connection or one new service touches everything

Exercise 1 (5 minutes)

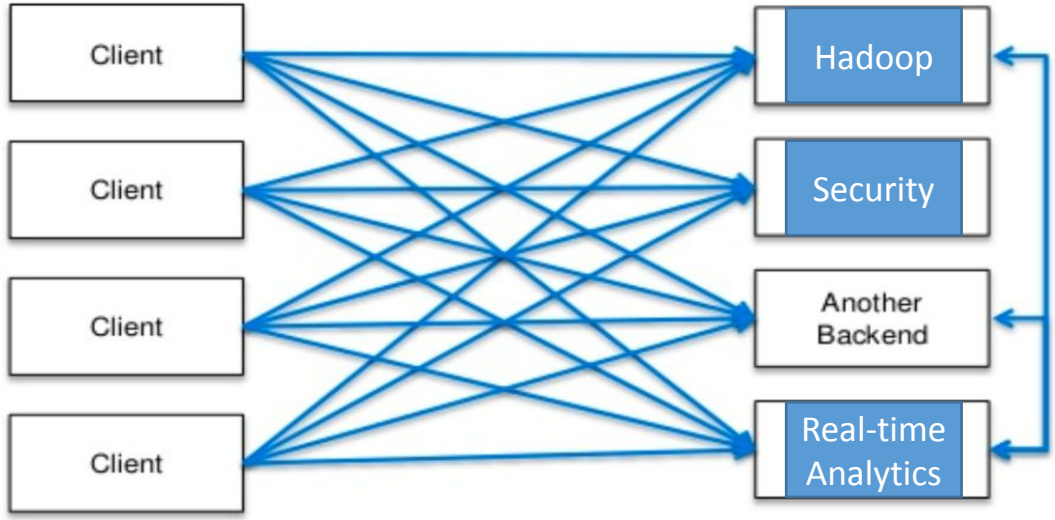
- Give an example of how data pipelines could be used
  - What are some examples of backends?



BIG DATA

Exercise 1 (5 minutes)

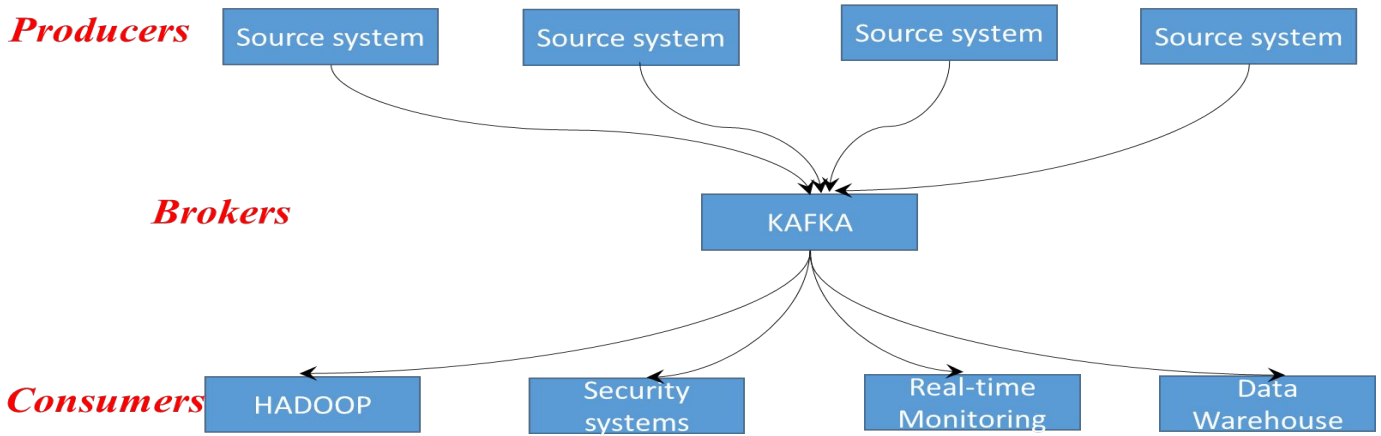
- Give an example of how data pipelines could be used
  - What are some examples of backends?



Kafka architecture

Why Kafka?

- Can we have an intermediary that connects
  - Different sources
  - Multiple backends
- It decouples the pipeline so that producers and consumers do not need to know about each other



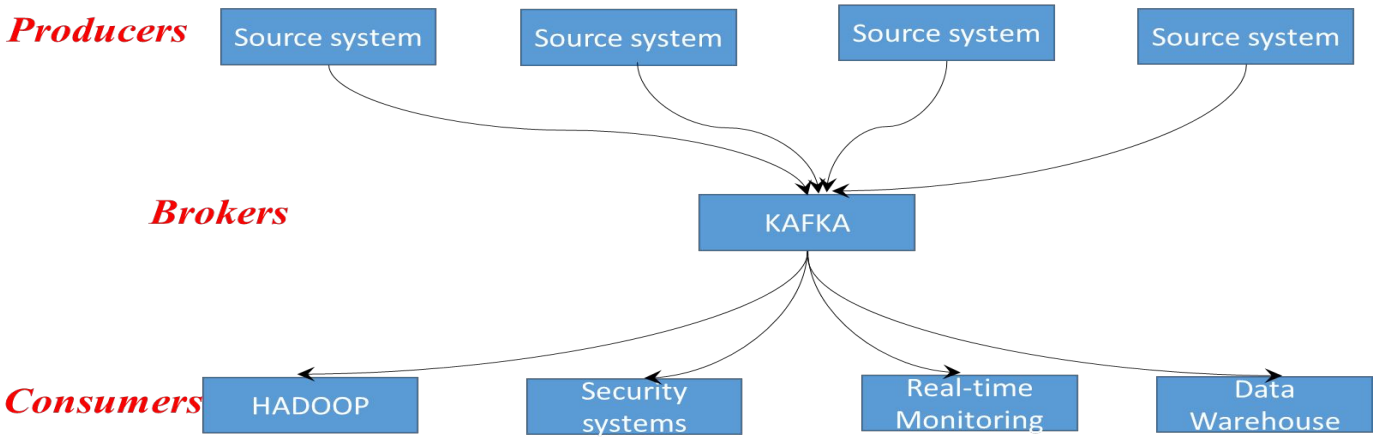
Kafka Decouples Data Pipelines

Kafka Components – Publish Subscribe Model

KAFKA components

The Three essential Elements /Components:

1. Publisher
2. Subscriber
3. Communication Infrastructure



Kafka Decouples Data Pipelines

PES UNIVERSITY
CELEBRATING 50 YEARS

Publishers and Subscribers

What does a Publisher do ..?	What does a Subscriber do ..?
It publishes messages to the Communication Infrastructure.	It subscribes to a category of messages.

The Routing Mystery



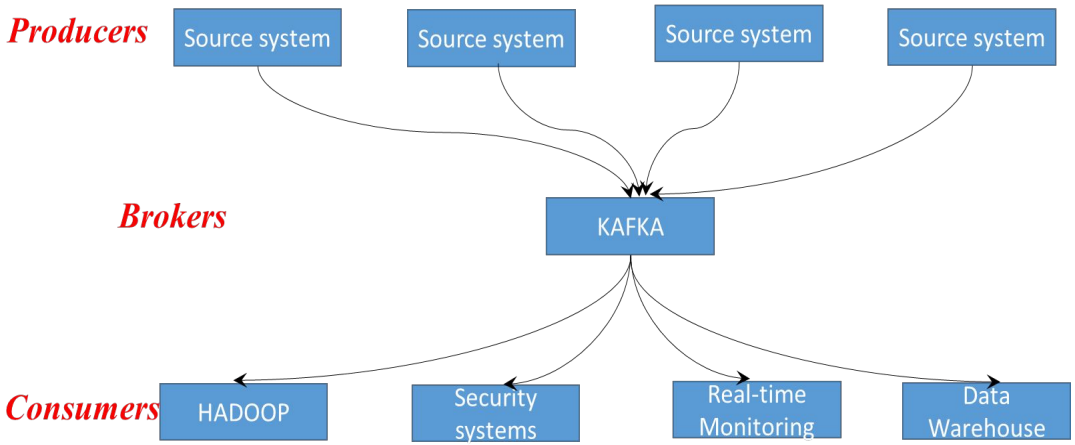
What If I tell you
I am the Publisher
and
I have no idea about
the Subscriber.



What If I tell you
I am the Subscriber
and
I have no idea about
the Publisher.

Role of a producer

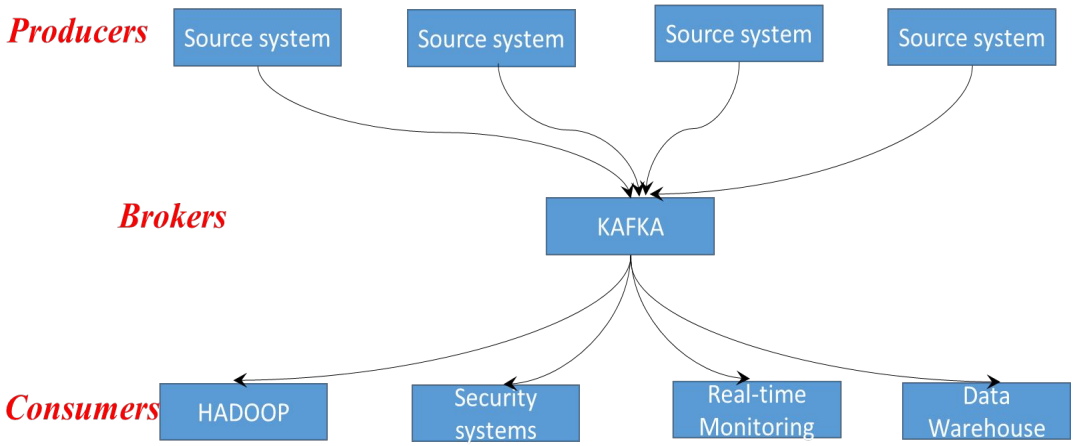
- A producer decides what data to send, and packages it into a message
- It publishes the messages on the communication infrastructure
- Simplest part of the system called the *publisher*



Kafka Decouples Data Pipelines

Role of a Consumer

- Tells the communication infrastructure about the type of messages it wants to receive
- Does not tell “who” to receive message from.
- Messages are delivered to the consumer by the communication infrastructure.
- Called the *subscriber*



Kafka Decouples Data Pipelines

Kafka Communication and Routing

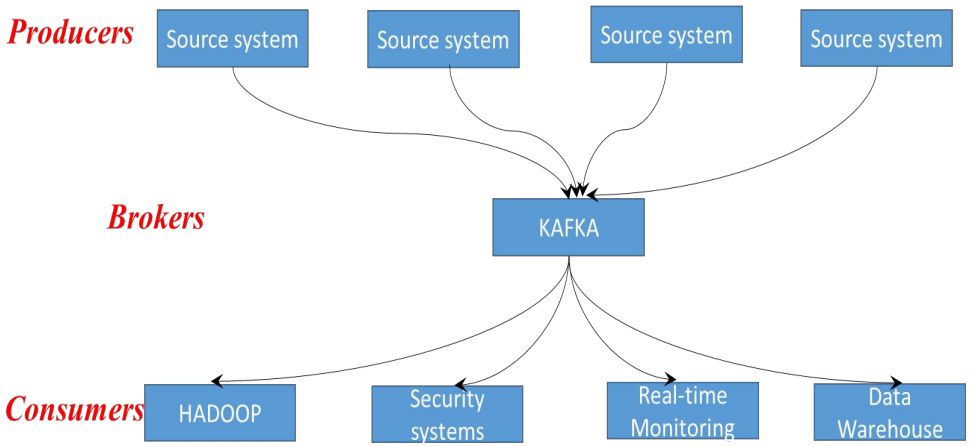
Which Messages get Delivered to Each Subscriber?

- Topic based system
- Content-based system



The Communication infrastructure

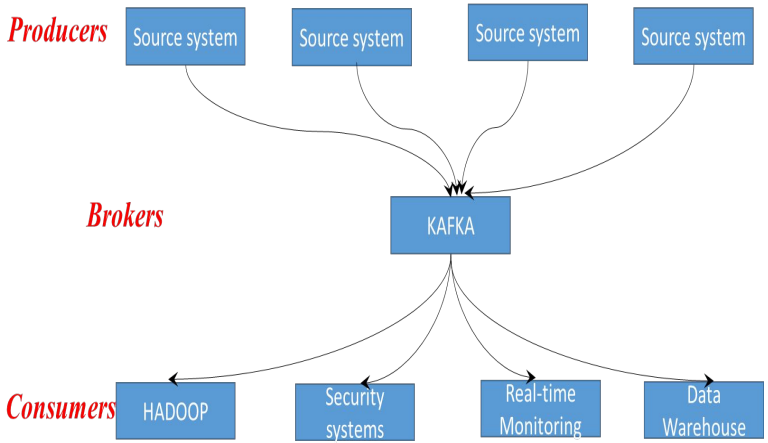
- The most complex part
- Determines which messages are delivered to each subscriber
- Two models of Routing
  - Topic based systems
  - Content based systems



Kafka Decouples Data Pipelines

Topic based routing

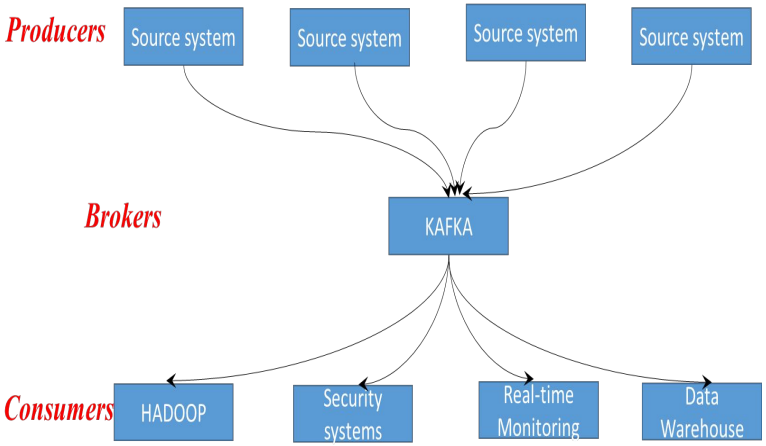
- Publishers send messages with topic labels
- Subscribers subscribe to topics and will receive all messages on that topic
- Example: Subscribe to all fire sensors in b block



Kafka Decouples Data Pipelines

Content based routing

- Subscribers define matching criteria and will receive all messages that match the criteria
- Example
  - Subscribe to advertisements that feature Virat Kohli - NOT supported by Kafka
- Pattern based
  - Kafka supports a simpler version called *pattern* based where we can give a wildcard expression for a topic
  - Example: Get all topics that have ipl*



Kafka Decouples Data Pipelines

Communication advantages/disadvantages

Pros

- No hard-wired connections between publishers and subscribers
- Flexible: Easy to add and remove publishers or subscribers

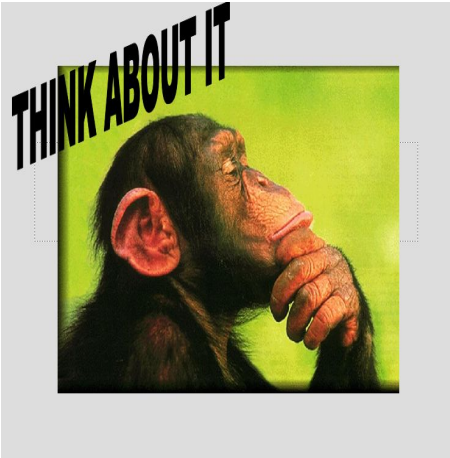
Cons

- Design and maintenance of topics
- Performance overhead due to communication infrastructure

BIG DATA

Exercise 2 (5 minutes)

- Consider a bookstore portal with various activities such as
  - Login
  - List books
  - Get book details
  - Buy book
  - Check status of order
  - Return book
  - Logout
- Assume we have 3 backend modules
  - Security
  - Order processing
  - Book information
- Would you use a topic-based or content-based system? What would be the topics / content..?



BIG DATA

Exercise 2 (Solution)

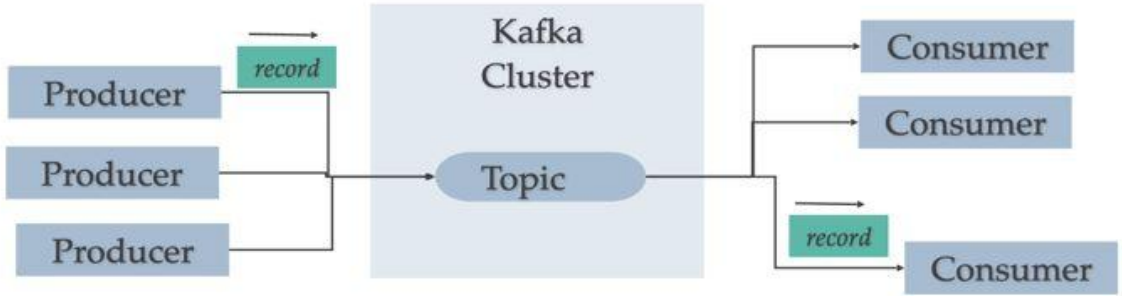
Would you use a topic-based or content-based system? What would be the topics / content..?

Probably topic-based, since each message type can be a topic



Kafka Messages

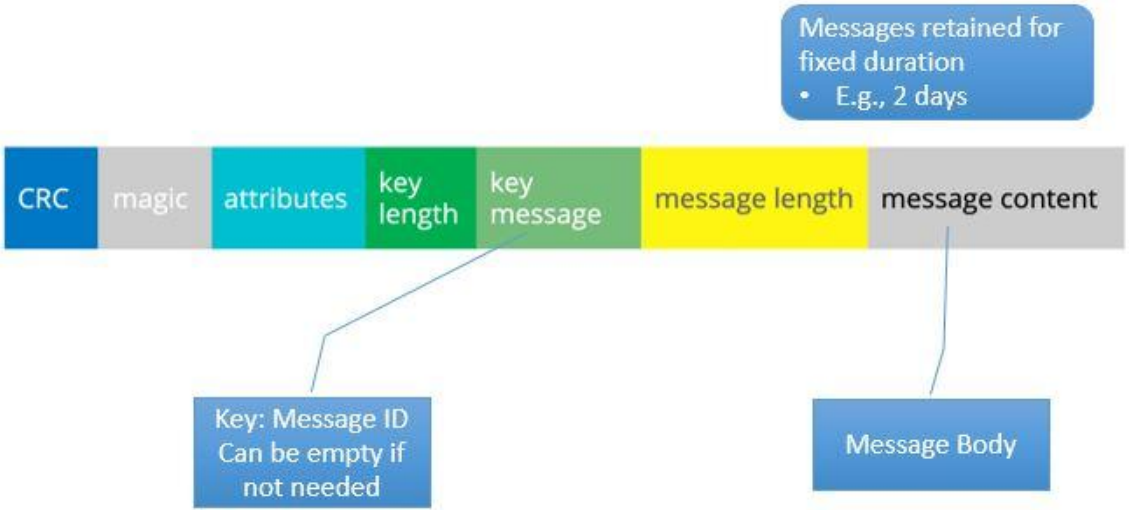
Topics, Producers and Consumers.



Topic name: “/shopping-cart-done“, “/user-signups”

Topic: log data structure. ordered, append-only sequence of records stored on disk

A Kafka Message



Key: used for partitioning
Value: the actual message payload
Offset: position of the message in the partition
Size: message length
Checksum: detects corruption
Magic byte: indicates message format version
Compression codec: Snappy, GZip, or LZ4
Timestamp: set by the producer at send time, or by the broker at arrival time, depending on config

Kafka Scaling

PES UNIVERSITY
CELEBRATING 50 YEARS

Scaling in Kafka - Partitions

Topics (Logical Category)

Named Category: Each topic has a unique name and is used to group related messages.

Multi-Tenant: Supports multiple concurrent producers writing and consumers reading.

Virtual Layer: Divided physically into underlying partitions across brokers.

Each topic is split into partitions.

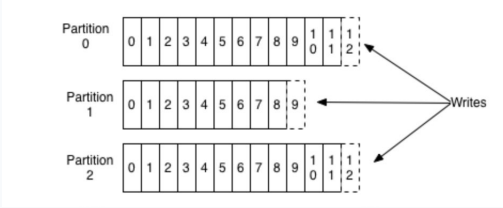
Partition (Physical Storage Unit)

Append-Only Log: An immutable sequence of incoming messages written on disk.

Scale Unit: Distributed across different broker nodes in the cluster.

Capacity Bypass: Overcomes single-server disk storage and I/O bottlenecks.

Each incoming message in a partition gets a unique offset.



Topic: Split into 3 independent physical partitions.

- Partition 0: Next Write at Offset 12
- Partition 1: Next Write at Offset 9
- Partition 2: Next Write at Offset 12

How Messages Are Assigned to Partitions

- **Key Provided in the message:** Hash of Key (Eg: hash(key) % N) ensures same key hits same partition.
- **No Key provided in the message:** Round-Robin / Sticky distribution balances load

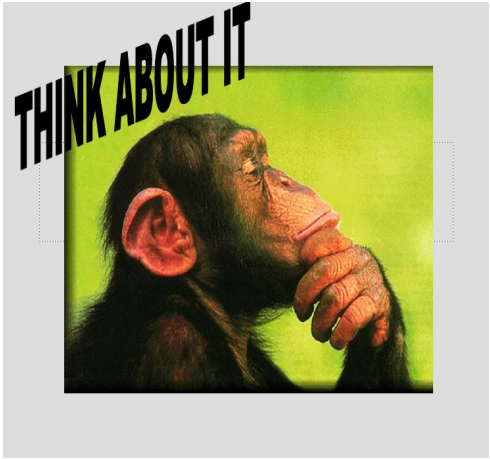
Note:

Consumers track the latest offset they've read. By comparing their current offset to the partition's newest offset, they instantly know if new messages are waiting to be read.

Exercise 3

How can reliability be guaranteed in Kafka?

- Hint: How does HDFS guarantee reliability?

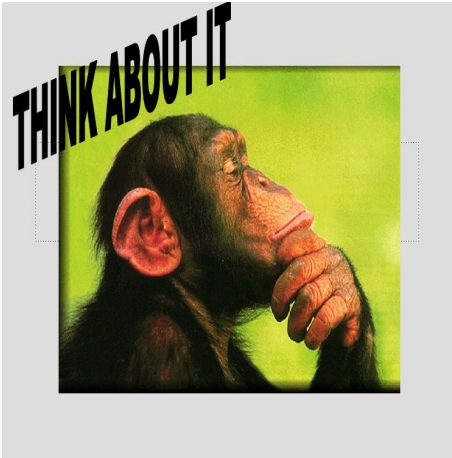


Exercise 3 (Solution)

How can reliability be guaranteed in Kafka?

- Hint: How does HDFS guarantee reliability?

Replication: each partition is copied across multiple brokers



Fault Tolerance in Kafka

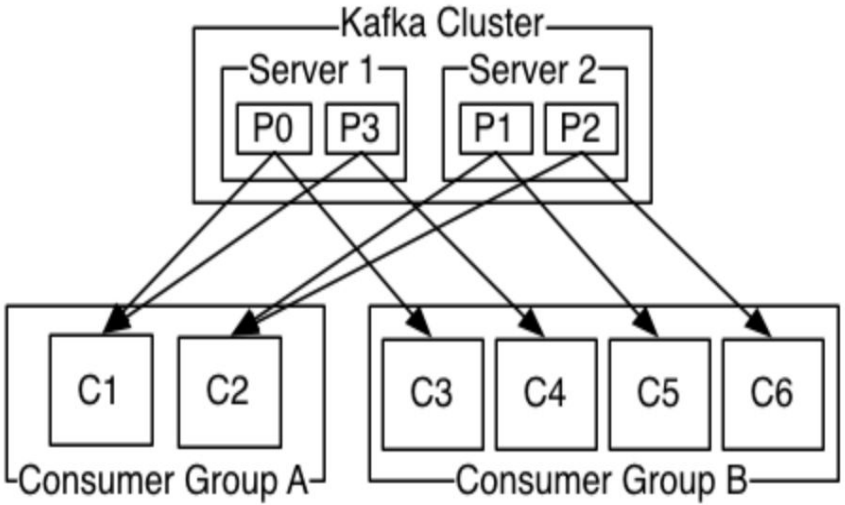
Fault-tolerance in Kafka (Replication)

- Partitions can be replicated
  - Leader: performs all reads, writes
  - Follower replicates the leader
  - New leader chosen if leader fails
- Durability levels (ack)
  - Sync: write confirmed only after ISR quorum (**ISR** - in-sync replicas)
  - Ack – configurable parameter
    - ack = 1: leader only (no follower ack)
    - ack = -1/all: leader waits for ack
- Recommended
  - Replicas = 3
  - Quorum = 2



Message delivery to consumers

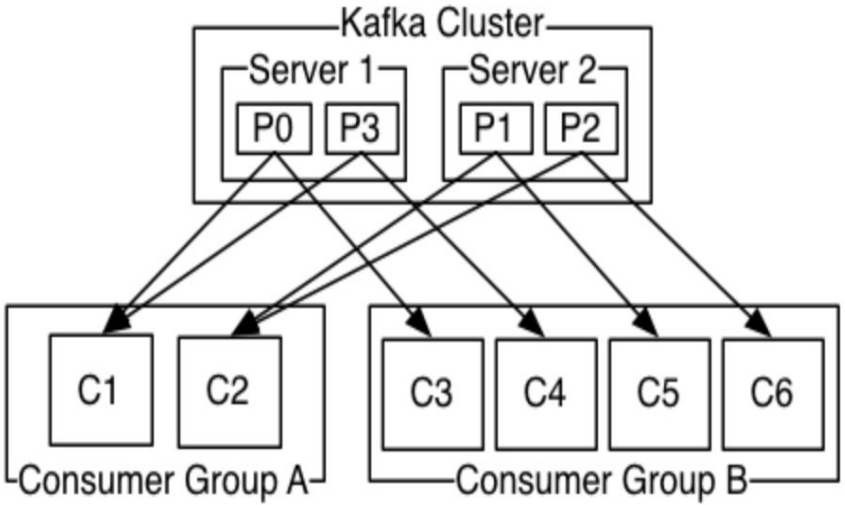
Topic with 4 partitions



- Consumer group
  - Typically multiple instances of an application
- Partition delivers message to ONE of the group members
  - Load balancing

Exercise 4

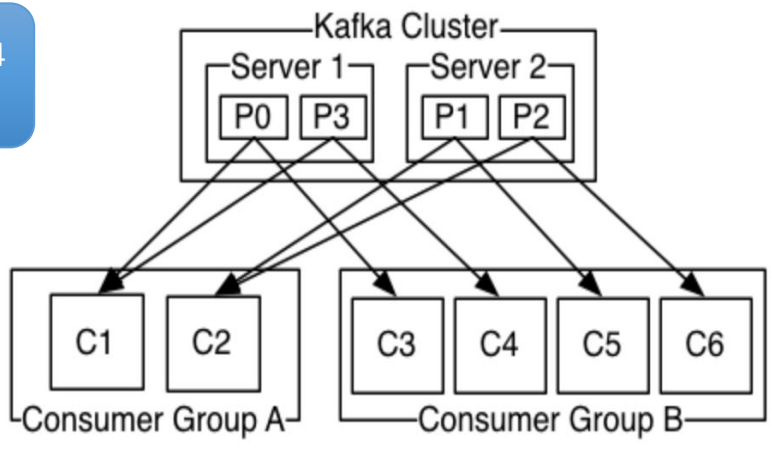
Topic with 4 partitions



- In the above configuration, how is the load balanced over all the instances?

Exercise 4 (Solution)

Topic with 4 partitions



- Group A
  - C1 is assigned P0, P3
  - C2 is assigned P1, P2

- Group B
  - C3 is assigned P0
  - C4 is assigned P3
  - …

- Each instance has the same number of partitions

BIG DATA

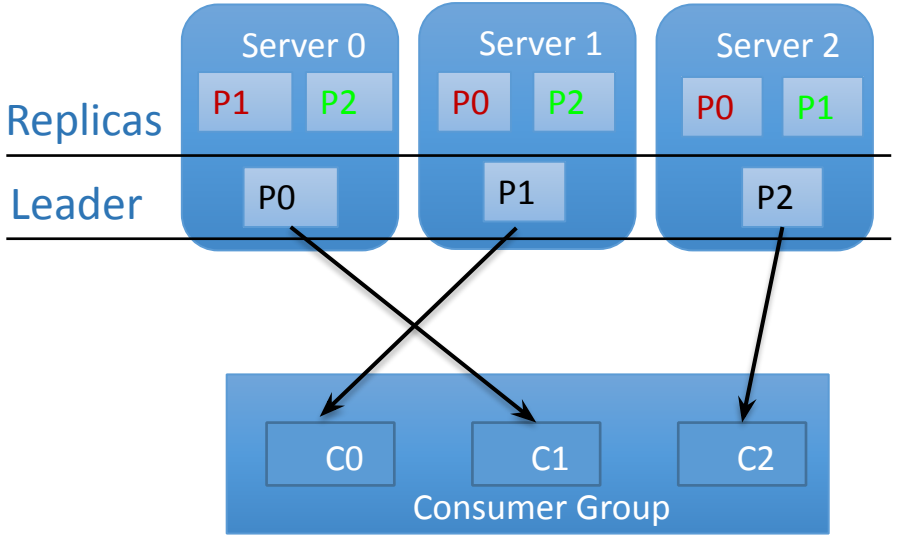
Exercise 5

- Suppose we have a Kafka system
  - 1 topic
  - 3 servers
  - 3 partitions
  - 3 replicas per partition
  - Consumer group with 3 instances
- Draw a diagram showing
  - Servers
  - Partitions
  - Consumer instances
  - Partition assignments



Exercise 5 (Solution)

- Suppose we have a Kafka system
  - 1 topic
  - 3 servers
  - 3 partitions
  - 3 replicas per partition
  - Consumer group with 3 instances
- Draw a diagram showing
  - Servers
  - Partitions
  - Consumer instances
  - Partition assignments



Kafka : Controllers

Controllers & Server Roles

Who decides which partition is the leader replica?

The **Controller** : a Kafka server responsible for electing partition leaders.

What is a Kafka Server?

A process or machine running Kafka.

Each server can be assigned **one or both** of the roles below:

Controller Role

- **Manages Metadata:** Controls cluster metadata & quorum state
- **Leader Elections:** Elects partition leaders across brokers
- **Cluster Leadership:** Acts as the administrative authority

Broker Role

- **Stores Data:** Persists and manages topic partitions
- **Serves Requests:** Handles produce (write) & consume (read) traffic
- **Partition Storage:** Hosts primary and replica partition logs

Who determines which Kafka server gets to be a controller?

Now this can be done two ways:

1. External Coordinator - ZooKeeper

Mechanism: A ZooKeeper can be brought in that is in-charge of maintaining which server in the cluster is the controller.

Version Usage: This mechanism was used in older versions of Kafka.

Requirement: Required a ZooKeeper mandatorily for this process.

(We will not go into the details of the mechanism of how ZooKeeper does it)

2. Self-Consensus - KRaft

Mechanism: The servers could themselves decide which one of them gets to become the controller through consensus (leader election algorithm).

Algorithm: Uses an election algorithm / consensus protocol - **KRaft**.

Version Usage: Newer versions of Kafka use KRaft.

Requirement: Does not require a ZooKeeper.

Source: https://kafka.apache.org/39/operations/kraft/

Who Uses Kafka

- ***LinkedIn***: Activity data and Operational metrics.
- ***Twitter***: Uses it as part of Storm – stream Processing infrastructure.
- ***Square***: Kafka as bus to move all system events to various Square data centers(logs, custom events, metrics, and so on). Outputs to Splunk, Gtaphite,Esper-like alerting systems.
- ***Spotify, Uber, Tumbler, Goldman Sachs, PayPal, Box, Cisco, Cloud Fatr, DataDog, LucidWorks, MailChimp, Netflix, etc.***

Additional Reading

- Introduction to Kafka
  - https://kafka.apache.org/intro
  - Watch the video on the introduction.
- Kafka in a nutshell
  - https://sookocheff.com/post/kafka/kafka-in-a-nutshell

[PDF] **Kafka**: A distributed messaging system for log processing [PDF] wisc.edu

J Kreps, N Narkhede, J Rao - Proceedings of the NetDB, 2011 - pages.cs.wisc.edu

… In Section 3, we describe the architecture of **Kafka** and its key design principles. We describe our deployment of **Kafka** at LinkedIn in Section 4 and the performance results of **Kafka** in …

☆ Save Cite Cited by 2542 Related articles All 29 versions »

As on Sept 7th, 2026



THANK YOU

Prafullata Kiran Auradkar
Department of Computer Science and Engineering
prafullatak@pes.edu



BIG DATA

Scala Programming Language

Prafullata Kiran Auradkar

Department of Computer Science and Engineering

prafullatak@pes.edu

Acknowledgements:

Significant information in the slide deck presented through the Unit 1 of the course have been created by **Dr. K V Subramaniam's** and would like to acknowledge and thank him for the same. There have been some information which I might have leveraged from **Dr. H L Phalachandra's** slide contents too. I may have supplemented the same with contents from books and other sources from Internet and would like to sincerely thank, acknowledge and reiterate that the credit/rights for the same remain with the original authors/publishers only. These are intended for classroom presentation only.

References

Programming in Scala
Martin Odersky, Lex Spoon, Bill Venners

https://data-flair.training/blogs/13-limitations-of-hadoop/

Issues with Hadoop

Issue with Small Files

- Hadoop does not suit for small data files due to its high capacity design
- A small file is significantly smaller than the HDFS block size (default 128MB)
- Too many small files cause problem for managing the Namenode.

Slow Processing Speed

- Data is distributed and processed over the cluster in MapReduce which increases the time and reduces processing speed.

Support for Batch Processing only and No Real-time Data Processing

- Hadoop supports batch processing only, it does not process streamed data, and hence overall performance is slower

No Delta Iteration

- Hadoop is not so efficient for iterative processing, as Hadoop does not support cyclic data flow(i.e. a chain of stages in which each output of the previous stage is the input to the next stage).

Apache Spark vs. Apache Hadoop

- Apache Hadoop and Apache Spark are both open-source frameworks for big data processing with some key differences.
- Hadoop uses the MapReduce to process data, while Spark uses resilient distributed datasets (RDDs).
- Hadoop has a distributed file system (HDFS), meaning that data files can be stored across multiple machines.
- The file system is scalable, since servers and machines can be added to accommodate increasing volumes of data. Spark does not provide a distributed file storage system, so it is mainly used for computation, on top of Hadoop.
- Spark does not need Hadoop to run, but can be used with Hadoop since it can create distributed datasets from files stored in the HDFS.

BIG DATA

What is Scala?

- SCAlable Language developed by Marvin Odersky at EPFL (Switzerland)
- Scala is a modern multi-paradigm programming language designed to express common programming patterns in a concise, elegant, and type-safe way.
- It seamlessly integrates features of object-oriented and functional languages.

Scala is object-oriented

- every value is an object. Types and behaviors of objects are described by classes and traits.
- Classes can be extended by subclassing, and by using a flexible mixin-based composition mechanism as a clean replacement for multiple inheritance.

Scala is functional

- every function is a value. Scala provides a lightweight syntax for defining anonymous functions, it supports higher-order functions, it allows functions to be nested, and it supports currying(transforms functions with multiple arguments into a single function arguments).
- Scala's case classes and its built-in support for pattern matching provide the functionality of algebraic types, which are used in many functional languages.
- Singleton objects provide a convenient way to group functions that aren't members of a class.
- ideal for developing applications like web services, data intensive applications.

What is Scala?

Scala is statically typed

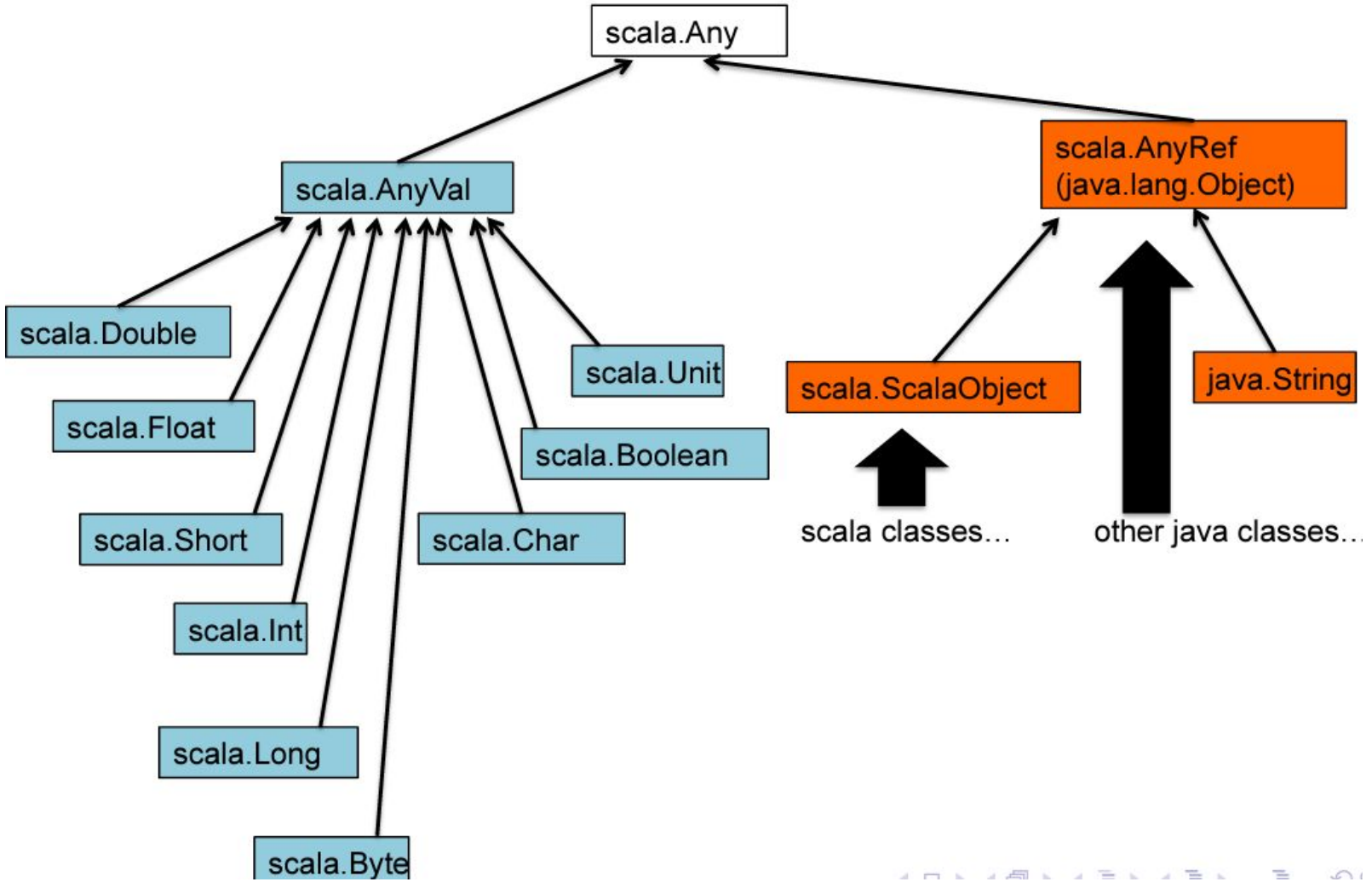
It supports:

- Generic classes
- Variance annotations
- Upper and lower type bounds
- Inner classes and abstract type members as object members
- Compound types
- Explicitly typed self references
- Implicit parameters and conversions
- Polymorphic methods – **EXAMPLES for function overloading are to be added with screenshots.**

Type inference means the user is not required to annotate code with redundant type information.

The Scala compiler can often infer the type of an expression so you don't have to declare it explicitly.

Scala class hierarchy



Apache Spark vs. Scala

- Spark is an open-source distributed general-purpose cluster-computing framework.
- Scala is a general-purpose programming language providing support for functional programming and a **strong static type system**.
- Spark is used to increase the Hadoop computational process.
- Scala can be used for web applications, streaming data, distributed applications and parallel processing.

BIG DATA

Spark with Python vs Spark with Scala

Criteria	Python with Spark	Scala with Spark
Performance Speed 	Python is comparatively slower than Scala when used with Spark, but programmers can do much more with Python than with Scala as Python provides an easier interface	Spark is written in Scala, so it integrates well with Scala. It is faster than Python
Learning Curve 	Python is known for its easy syntax and is a high-level language easier to learn. It is also highly productive even with its simple syntax	Scala has an arcane syntax making it hard to learn, but once you get a hold of it you will see that it has its own benefits
Data Science Libraries 	In Python API, you don't have to worry about the visualizations or Data Science libraries. You can easily port the core parts of R to Python as well	Scala lacks proper Data Science libraries and tools, and it does not have proper tools for visualization

Spark with Python vs Spark with Scala

Readability of Code 	Readability, maintenance, and familiarity of code are better in Python API	In Scala API, it is easy to make internal changes since Spark is written in Scala
Complexity 	Python API has an easy, simple, and comprehensive interface	Scala, in fact, produces verbose output, and hence it is considered a complex language
Machine Learning Libraries 	Python is preferred for implementing Machine Learning algorithms	Scala is preferred when you have to implement Data Engineer technologies rather than Machine Learning

BIG DATA

Java Vs Scala

- Scala is very similar to Java Language. It compiled to byte codes and use Java Virtual Machine(JVM) to execute code.
- Scala reduces the number of lines from a Java application by making clever use of type inference, treating everything as an object, function passing, and several other features.
- Scala has built-in lazy evaluation, which allows deferring time-consuming computation until absolutely needed and we can do this by using a keyword called “lazy”.
- Scala supports Operator overloading but Java does not support Operator overloading .
- Functions are treated as objects in Java but Scala treats any method or function as variables.
- Syntax in Scala is more complicated than Java.
- Java is backward compatible but Scala provides limited backward compatibility



Scala is interoperable

Scala programs interoperate seamlessly with Java class libraries:

- Method calls
- Field accesses
- Class inheritance
- Interface implementation

all work as in Java.

Scala programs compile to JV bytecodes.

Scala's but there differences.

```
object Example1 {
  def main(args: Array[String]) {
    val b = new StringBuilder()
    for (i ← 0 until args.length) {
      if (i > 0) b.append(" ")
      b.append(args(i).toUpperCase)
    }
    Console.println(b.toString)
  }
}
```

Array[String] instead of String[]

Scala's version of the extended **for** loop

Arrays are indexed args(i) instead of args[i]

Program to change the command line argument input to uppercase

Java and Scala: Spot the differences

Declaring variables:

```
var x: Int = 7
var x = 7 // type inferred
val y = "hi" // read-only
```

Java equivalent:

```
int x = 7;

final String y = "hi";
```

Functions:

```
def square(x: Int): Int = x*x
def square(x: Int): Int = {
  x*x
}
def announce(text: String) =
{
  println(text)
}
```

Java equivalent:

```
int square(int x) {
  return x*x;
}

void announce(String text) {
  System.out.println(text);
}
```

Java and Scala: Class

... in Java:

```
public class Person {
  public final String name;
  public final int age;
  Person(String name, int age) {
      this.name = name;
      this.age = age;
  }
}
```

... in Scala:

```
class Person(val name: String,
             val age: Int) {}
```

BIG DATA

Java and Scala: Usage of Class

... in Java:

```
import java.util.ArrayList;
...
Person[] people;
Person[] minors;
Person[] adults;
{  ArrayList<Person> minorsList = new ArrayList<Person>();
   ArrayList<Person> adultsList = new ArrayList<Person>();
   for (int i = 0; i < people.length; i++)
       (people[i].age < 18 ? minorsList : adultsList)
           .add(people[i]);
   minors = minorsList.toArray(people);
   adults = adultsList.toArray(people);
}
```

This program partition the data based on age. <18 is stored in minors list and >18 is stored in adults list.

A function value

An infix method call

... in Scala:

```
val people: Array[Person]
val (minors, adults) = people partition (_.age < 18)
```

A simple pattern match

Java and Scala: Definitions

Scala method definitions:

```
def fun(x: Int): Int = {
   result
}

def fun = result
```

Scala variable definitions:

```
      var x: int = expression
val x: String = expression
```

Java method definition:

```
int fun(int x) {
   return result
}
```

(no parameterless methods)

Java variable definitions:

```
int x = expression
final String x = expression
```

```
scala> val res =10
res: Int = 10

scala> def fun = res
fun: Int

scala> fun
res8: Int = 10
```

Java and Scala: Expressions

Scala method calls:

```
obj.meth(arg)
```

or: obj meth arg

Scala choice expressions:

```
if (cond) expr1 else expr2
```

```
expr match {
   case pat1 => expr1
   ....
          case patn => exprn
       }
```

Java method call:

```
obj.meth(arg)
```

(no operator overloading)

Java choice expressions, stats:

```
cond ? expr1 : expr2   // expression

if (cond) return expr1; // statement
else return expr2;
```

```
switch (expr) {
   case pat1 : return expr1;
   ...
   case patn : return exprn ;
} // statement only
```

Java and Scala: Traits

Scala Trait

```
trait T {
   def abstractMeth(x: String): String

   def concreteMeth(x: String) =
      x+field

   var field = "!"
}
```

Scala mixin composition:

```
class C extends Super with T
```

Java Interface

```
interface T {
   String abstractMeth(String x)

   (no concrete methods)


   (no fields)
}
```

Java extension + implementation:

```
class C extends Super implements T
```

Scala:

- Minimal Verbosity
- Referential Transparency – type inferencing
- Concurrency
- Functional Programming

Minimal Verbosity

- Java:
  - class Person {
    private String firstName;
    private String lastName;
    private int age;

    public Person(String firstName, String lastName, int age) {
    this.firstName = firstName;
    this.lastName = lastName;
    this.age = age;
    }

    public void setFirstName(String firstName) { this.firstName = firstName; }
    public void String getFirstName() { return this.firstName; }
    public void setLastName(String lastName) { this.lastName = lastName; }
    public void String getLastName() { return this.lastName; }
    public void setAge(int age) { this.age = age; }
    public void int getAge() { return this.age; }
    }
- Scala:
  - class Person(var firstName: String, var lastName: String, var age: Int)
- Source: http://blog.objectmentor.com/articles/2008/08/03/the-seductions-of-scala-part-i

BIG DATA

Type Inferencing

- Java is statically typed--a variable has a type, and can hold only values of that type
  - You must specify the type of every variable
  - Type errors are caught by the compiler, not at runtime--this is a big win
  - However, it leads to a lot of typing (pun intended)
- Languages like Ruby and Python don’t make you declare types
  - Easier (and more fun) to write programs
  - Less fun to debug, especially if you have even slightly complicated types

Type Inferencing

- Scala is also statically typed, but it uses type inferencing--that is, it figures out the types, so you don't have to
  - **The good news**: Less typing, more fun, type errors caught by the compiler
  - **The bad news**: More kinds of error messages to get familiar with

- Example
  - `val collegeName = "PES University"`
  - `def squareOf(x: Int) : x * x`

Source: https://docs.scala-lang.org/tour/type-inference.html

Type Inferencing

- Example

Inferred type

```
ubuntu@ubuntu2004:~$ scala
Welcome to Scala 2.11.12 (OpenJDK 64-Bit Server VM, Java 11.0.24).
Type in expressions for evaluation. Or try :help.

scala> val inferredValue = 42
inferredValue: Int = 42
```

```
scala> val explicitValue: Int = 42
explicitValue: Int = 42
```

Explicit type

Type Inferencing

- Example

```
scala> def calSquare(x: Int) = x * x
calSquare: (x: Int)Int

scala> calSquare(20)
res0: Int = 400
```

Inferred return type as Int

But parameters are to be explicitly typed – strict type safety

- Explicit return type

```
scala> def Div(x: Int, y: Int): Float = { x/y}
Div: (x: Int, y: Int)Float
```

Type Inferencing

- Example

```
scala> def Div(x: Int, y: Int): Float = { x/y}
Div: (x: Int, y: Int)Float
```

Explicit return type as Float

```
scala> Div(3,2)
res1: Float = 1.0
```

Otherwise – inferred return type would be Int !

Type Inferencing

Operations limited to the **Generic Type**

- Generic Function

```
scala> def isEqual[T] (a: T, b:T): Boolean= { a == b}
isEqual: [T](a: T, b: T)Boolean

scala> isEqual(23, 32)
res2: Boolean = false

scala> isEqual (23,43.3)
res3: Boolean = false

scala> isEqual( "abc","abc")
res4: Boolean = true

scala> isEqual(List(1,2), 32.4)
res5: Boolean = false
```

While types mismatch – returns False

PES UNIVERSITY
CELEBRATING 50 YEARS

Consistency

- In Java, every value is an object--unless it's a primitive
  - Numbers and booleans are primitives for reasons of efficiency, so we have to treat them differently (**you can't "talk" to a primitive**)
  - **In Scala, all values are objects.**
  - The **compiler turns them into primitives, so no efficiency is lost** (behind the scenes, there are objects like RichInt)

- Java has operators (+, <, ...) and methods, with different syntax
  - In Scala, operators are just methods, and in many cases you can use either syntax

Concurrency

- Broadly speaking, concurrency can be either:
  - Fine-grained: Frequent interactions between threads working closely together (extremely challenging to get right)
  - Coarse-grained: Infrequent interactions between largely independent sequential processes (much easier to get right)
- Java 5 and 6 provide reasonable support for traditional fine-grained concurrency
  - Threads
- Scala has total access to the Java API
  - Hence, it can do anything Java can do
  - And it can do much more (see next slide)
- Scala also has Actors for coarse-grained concurrency
  - Sending messages (use the send ! Abstraction)

Concurrency

- The best-known functional languages are ML, OCaml, and Haskell
- Functional languages are regarded as:
  - "Ivory tower languages," used only by academics (mostly but not entirely true)
  - Difficult to learn (mostly true)
  - The solution to all concurrent programming problems everywhere (exaggerated, but not entirely wrong)
- Scala is an "impure" functional language--you can program functionally, but it isn't forced upon you

BIG DATA

Functional Programming

- The big problem with concurrency is dealing with shared state--multiple threads all trying to read and maybe change the value of same variables
- If all data were immutable, then any thread could read it any time, no synchronization, no locks, no problem
- But if your program couldn’t ever change anything, then it couldn’t ever do anything, right?
- Wrong!
- There is an entire class of programming languages that use only immutable data, but work just fine: the functional languages

BIG DATA

Functional Programming

- Immutable – functional operations create new structures
  - Don’t modify existing structures
- Program implicitly captures data flow
- Order of operations not significant
- Functions
  - Are objects
  - can be passed as arguments to functions
  - can return functions
  - Can operate on collections

Val declares an immutable object. So, reassignment is not allowed

```
scala> val validString: String = "Hello World"
validString: String = Hello World

scala> validString = "Hi"
<console>:12: error: reassignment to val
       validString = "Hi"
                   ^
```

Var declares a mutable object.
So, reassignment is allowed

```
scala> var validString2: String = "Hello World"
validString2: String = Hello World

scala> validString2 = "Hi"
validString2: String = Hi
```

Type safety with generic parameters

```
scala> var validString2: String = "Hello World"
validString2: String = Hello World

scala> validString2 = "Hi"
validString2: String = Hi
```

Functional Programming Example

Functional Programming – differences with java

Quicksort program in Scala – java style

Observe

Focus on HOW?

Explicitly iterate

Determine when and how to swap()

```
def sort(xs: Array[Int]) {
  def swap(i: Int, j: Int) {
    val t = xs(i); xs(i) = xs(j); xs(j) = t
  }
  def sort1(l: Int, r: Int) {
    val pivot = xs((l + r) / 2)
    var i = l; var j = r
    while (i <= j) {
      while (xs(i) < pivot) i += 1
      while (xs(j) > pivot) j -= 1
      if (i <= j) {
        swap(i, j)
        i += 1
        j -= 1
      }
    }
    if (l < j) sort1(l, j)
    if (j < r) sort1(i, r)
  }
  sort1(0, xs.length - 1)
}
```

Functional Programming – differences with java

Focus on solving the problem

Observe

Focus on WHAT, not HOW

```
def sort(xs: Array[Int]): Array[Int] = {
  if (xs.length <= 1) xs
  else {
    val pivot = xs(xs.length / 2)
    Array.concat(
      sort(xs filter (pivot >)),
           xs filter (pivot ==),
      sort(xs filter (pivot <)))
  }
}
```

Pick a pivot

Sort values smaller than the pivot

Sort values larger than the pivot

Concatenate the result

BIG DATA

Self Learning Exercise

- Does this sort array in ascending order or descending order?
- Consider the program with array
  - xs=3,1,2,0,7,6,4,5
- Write a program to sort in the reverse order (if ascending, sort descending)
- How can we parallelize this?

```
def sort(xs: Array[Int]): Array[Int] = {
  if (xs.length <= 1) xs
  else {
    val pivot = xs(xs.length / 2)
    Array.concat(
      sort(xs filter (pivot >)),
           xs filter (pivot ==),
      sort(xs filter (pivot <)))
  }
}
```

Functional programming and collections

Original List is left unchanged

```
val list = List(1, 2, 3)
list.foreach(x => println(x)) // prints 1, 2, 3
list.foreach(println)         // same

list.map(x => x + 2)    // returns a new List(3, 4, 5)
list.map(_ + 2)         // same

list.filter(x => x % 2 == 1)// returns a new List(1, 3)
list.filter(_ % 2 == 1)     // same

list.reduce((x, y) => x + y) // => 6
list.reduce(_ + _)           // same
```

Functional Programming and Big Data

- Big data architectures leverage

  Parallel disk, memory and CPU in clusters

- Operations consist of independently parallel operations

  Similar to *map()* operator in a functional language

- Parallel operations have to be consolidated

  Similar to *aggregation()* operators in functional languages

- In order to achieve the desired efficiency for big data applications, Concurrency, Parallelism and Referential Transparency are key, and in a functional environment, all of them are naturally present.



THANK YOU

Prafullata Kiran Auradkar
Department of CSE
prafullatak@pes.edu